

Programsko inženjerstvo ak.god 2025./2026.

Sveučilište u Zagrebu
Fakultet elektrotehnike i računarstva

Room-Rently

Tim: <TG03.2>

Ime tima: OpenReservations

Članovi tima: Jakov Zekić – Josip Mrakovčić – Karlo
Živković – Mateo Cerčić –
Nino Strčić – Noa Rešetar

Nastavnik: Vlado Sruk

Asistent: Miljenko Krhen

Demos: Ivo Gabud

Sadržaj

1. Opis projektnog zadatka	7
Room-Rently – Modernizacija turizma u jednoj aplikaciji.....	7
Uvod.....	7
Cilj projekta	7
Problematika i motivacija.....	7
Potencijalna korist projekta.....	8
Za goste	8
Za hotel / iznajmljivača.....	8
Za administratore sustava	8
Postojeća slična rješenja	8
Skup korisnika	8
Mogućnost prilagodbe i daljnjeg razvoja	9
Opseg projektnog zadatka.....	9
Zaključak	9
2. Analiza zahtjeva	9
Funkcionalni zahtjevi	9
Nefunkcionalni zahtjevi.....	11
Zahtjevi performansi	11
Zahtjevi sigurnosti	11
Zahtjevi upotrebljivosti	12
Zahtjevi pouzdanosti i dostupnosti	12
- Zahtjevi održavanja i proširivosti.....	12
Zahtjevi kompatibilnosti	12
Zahtjevi skalabilnosti	13
Dionici sustava	13
Aktori i njihovi funkcionalni zahtjevi.....	13
Zaključak	14
3. Specifikacija zahtjeva sustava.....	14
Obrasci uporabe	14
Dijagrami obrazaca uporabe	15
Dijagram obrazaca uporabe općenito	15
Funkcionalnosti vlasnika/voditelja	16
Funkcionalnosti administratora.....	17
Opis obrazaca uporabe	17
UC1: Prijava i registracija korisnika	17
UC2: Upravljanje smještajem	18
UC3: Upravljanje rezervacijama	18
UC4: Generiranje i slanje potvrda rezervacije.....	18
UC5: Pregled izvještaja i statistike	19

UC6: Filtriranje i pretraga smještaja	19
UC7: Upravljanje dostupnošću i terminima	19
UC8: Upravljanje korisnicima	19
UC9: Slanje e-mail obavijesti	20
UC10: Optimizacija zauzeća smještaja	20
UC11: Integracija s Google Maps API-jem	20
Sekvencijski dijagrami	21
Sekvencijski dijagram za registraciju i prijavu korisnika (UC1)	21
Sekvencijski dijagram za upravljanje smještajnih jedinica i brisanje korisnika (UC2, UC8)	23
Provjera uključenosti funkcionalnosti u obrasce uporabe	24
4. Arhitektura sustava	25
Arhitektura sustava – Room-Rently	25
Cilj poglavlja	25
Opis arhitekture	25
Stil arhitekture	25
Podsustavi	25
Preslikavanje na radnu platformu	26
Spremišta podataka	26
Mrežni protokoli	26
Globalni upravljački tok	26
Sklopovskoprogramski zahtjevi	26
Obrazloženje odabira arhitekture	27
Principi oblikovanja	27
Razmatrane alternative	27
Organizacija sustava na visokoj razini	27
Organizacija aplikacije	27
Frontend i Backend slojevi	27
MVC Arhitektura	27
MVC model	28
Ključne komponente	29
Interakcija između frontenda i backenda	29
Prijava i autentifikacija (OAuth 2.0)	29
Zaštita ruta	29
Baza podataka	30
Opis tablica	30
Person	30
Unit	31
UnitReservation	33
UnitImg	34
ER Dijagram baze podataka	35

Dijagram razreda	36
Dinamičko ponašanje aplikacije	37
UML dijagram stanja	38
UML dijagram aktivnosti	39
5. Arhitektura komponenata i razmještaja	40
Dijagram Komponenti	41
Dijagram komponenti Frontenda	41
Dijagram komponenti Backenda.....	41
Dijagram razmještaja	42
6. Ispitivanje programskog rješenja	43
Ispitivanje komponenti	43
Ispit komponente 1 – dodavanje korisnika	44
Ulaz očekivanog ponašanja:	44
Ulaz rubnog slučaja:.....	44
Ulaz nepodržanog ponašanja:	44
Izvorni kod ispitnog slučaja	44
Ispit komponente 2 – dohvat trenutnog korisnika	45
Ulaz očekivanog ponašanja:	45
Ulaz rubnog slučaja:.....	45
Ulaz neočekivanog ponašanja:	45
Izvorni kod ispitnog slučaja	45
Ispit komponente 3 – dodavanje smještajne jedinice	46
Ulaz očekivanog ponašanja:	47
Ulaz rubnog slučaja:.....	47
Ulaz nepodržanog ponašanja:	47
Izvorni kod ispitnog slučaja	47
Ispit komponente 4 – ažuriranje smještajne jedinice	48
Ulaz očekivanog ponašanja:	48
Ulaz rubnog slučaja:.....	49
Ulaz nepodržanog ponašanja:	49
Izvorni kod ispitnog slučaja	49
Ispit komponente 5 – kreiranje rezervacije	50
Ulaz očekivanog ponašanja:	51
Ulaz rubnog slučaja:.....	51
Ulaz nepodržanog ponašanja:	51
Izvorni kod ispitnog slučaja	51
Ispit komponente 6 – dohvat vlastitih rezervacija	53
Ulaz očekivanog ponašanja:	53
Ulaz rubnog slučaja:.....	53
Ulaz neočekivanog ponašanja:	53

Izvorni kod ispitnog slučaja	53
Ispitivanje sustava	55
1. Redovni slučaj - Rezervacija smještaja(Gost)	56
Ulazi	56
Koraci ispitivanja.....	56
Očekivani izlaz	56
Dobiveni izlaz.....	56
Izvorni kod	56
2. Rubni uvjet - Rezervacija smještaja zauzetog datuma(Gost)	58
Ulazi	58
Koraci ispitivanja.....	58
Očekivani izlaz	59
Dobiveni izlaz.....	59
Izvorni kod	59
3. Rubni uvjet - Izvoz statistike za period bez rezervacija(Owner).....	60
Ulazi	61
Koraci ispitivanja.....	61
Očekivani izlaz	61
Dobiveni izlaz.....	61
Izvorni kod	61
4. Poziv nepostojeće funkcionalnosti - Pokušaj korištenja /admin rute(Gost)	63
Ulazi	63
Koraci ispitivanja.....	63
Očekivani izlaz	63
Dobiveni izlaz.....	63
Izvorni kod	63
7. Tehnologije za implementaciju aplikacije	65
Korištene tehnologije i alati	65
1. Programski jezici	65
2. Radni okviri i biblioteke.....	65
3. Baza podataka	65
4. Razvojni alati.....	66
5. Alati za ispitivanje	66
6. Alati za razmjestaj	66
7. Cloud platforma.....	67
8. Upute za puštanje u pogon.....	67
1. Instalacija	67
Preduvjeti	67
Instalacija ovisnosti	67
2. Postavke	68

Konfiguracijske datoteke	68
Priprema Baze i Bucketa (Supabase)	69
3. Pokretanje aplikacije	70
Razvojno okruženje	70
Produkcijsko okruženje (Priprema):	70
4. Upute za administratore	71
Pristup podacima:	71
Redovito održavanje:	71
Ažuriranje aplikacije: Kada napravite promjene u kodu:	71
5. Postavljanje na Cloud (AWS App Runner i Netlify)	71
6. Opis pristupa aplikaciji na javnom poslužitelju	72
Ograničenja:	72
9. Zaključak i budući rad	72
Osvrt na izradu projektnog zadatka	72
Tehnički izazovi	73
Stečena i potrebna znanja	74
Iako smo ponosni na ostvareno, prepoznavamo područja u kojima bi nam dodatno znanje omogućilo još brži i kvalitetniji razvoj:	74
Perspektive za nastavak rada	74
Realizacija funkcionalnosti	75
Ključni izazovi i rješenja	75
Opis izazova:	75
Primijenjena rješenja i naučene lekcije	75
Zaključak	76
Smjernice za budući razvoj:	76
Završni osvrt voditelja tima	76
A. Popis literature	77
Baza podataka	77
Backend	77
Frontend	77
B. Prikaz Aktivnosti Grupe	77
Tablica aktivnosti	82
Dijagrami pregleda promjena	83
C. Dnevnik promjena dokumentacije	85
Za praćenje promjena koristimo Trello workspace (https://trello.com/w/roomrently).	85
A. Dnevnik promjena dokumentacije	85
Praćenje promjena	88

1. Opis projektnog zadatka

Room-Rently – Modernizacija turizma u jednoj aplikaciji

Uvod

Room-Rently je web aplikacija razvijena s ciljem modernizacije procesa rezervacije i upravljanja smještajem unutar jednog hotela ili objekta.

Ideja projekta je stvoriti **jednostavan, pouzdan i pregledan sustav** koji povezuje goste s osobljem hotela te olakšava svakodnevno poslovanje kroz digitalizaciju rezervacija i evidencija.

Aplikacija omogućuje **brzu i sigurnu rezervaciju soba**, jednostavno upravljanje dostupnošću te automatsko vođenje statistike i izvještaja.

Room-Rently tako predstavlja digitalnu alternativu klasičnim načinima vođenja rezervacija, uz naglasak na **učinkovitost, preglednost i profesionalno korisničko iskustvo**.

Cilj projekta

Glavni cilj **Room-Rently** sustava je **pojednostaviti i unaprijediti način** na koji hotel ili iznajmljivač upravlja svojim smještajem i rezervacijama.

Aplikacija gostima omogućuje jednostavnu pretragu i rezervaciju soba, dok osoblju pruža pregledan alat za upravljanje smještajnim jedinicama i praćenje zauzeća.

Osnovne ideje projekta su: - digitalizirati proces rezervacija unutar hotela ili objekta,

- smanjiti mogućnost grešaka i preklapanja termina,
- poboljšati komunikaciju između gostiju i osoblja,
- omogućiti analitički uvid u poslovanje i zauzeće soba.

Problematika i motivacija

Mnogi manji hoteli i privatni iznajmljivači još uvijek koriste ručne metode rezervacija putem telefona, e-maila ili bilježnica, što često dovodi do zabuna, dvostrukih rezervacija i gubitka informacija.

Globalne platforme poput Booking.com-a ili Airbnb-a nude napredna rješenja, ali su često prekompleksna, uz visoke provizije i nepotrebne funkcionalnosti za objekte koji žele samostalno upravljati svojim kapacitetima.

Room-Rently je zamišljen kao **interni sustav za jedan hotel**, koji omogućuje potpunu kontrolu nad rezervacijama i smještajem bez posrednika i dodatnih troškova.

Potencijalna korist projekta

Za goste

- Jednostavna pretraga i online rezervacija soba.
- Brza i sigurna potvrda rezervacije putem e-maila.
- Mogućnost prijave putem Google računa bez dodatne registracije.

Za hotel / iznajmljivača

- Centralizirano upravljanje svim sobama i rezervacijama.
- Automatska evidencija zauzeća i dolazaka gostiju.
- Izrada izvještaja i statistika o poslovanju (u PDF ili Excel formatu).
- Manje administracije i bolja organizacija recepcijskog osoblja.

Za administratore sustava

- Kontrola pristupa i ovlasti korisnika.
- Uvid u sve rezervacije i korisničke aktivnosti.
- Generiranje analitičkih izvještaja o radu sustava.

Postojeća slična rješenja

Na tržištu postoje brojni sustavi za hotelsko poslovanje (PMS – Property Management System), no **Room-Rently** se razlikuje po: - jednostavnom i modernom sučelju prilagođenom manjim objektima,

- potpunoj neovisnosti o vanjskim platformama,
- lokalnoj prilagodbi i podršci,
- mogućnosti integracije s Google Maps API-jem i lokalnim platnim servisima.

Ovakav pristup omogućuje hotelima i iznajmljivačima da imaju **vlastiti sustav rezervacija**, bez dodatnih troškova i provizija.

Skup korisnika

U sustavu **Room-Rently** definirane su tri osnovne korisničke uloge:

- **Korisnik / Gost** – može pregledavati dostupne sobe, izvršiti rezervaciju, primiti potvrdu putem e-maila te upravljati vlastitim rezervacijama.

- **Vlasnik objekta** – upravlja smještajnim jedinicama (unos, uređivanje, brisanje), pregledava rezervacije, zauzeće i izvještaje o poslovanju.
- **Administrator** – ima puni pristup sustavu, nadzire sve korisnike i rezervacije, dodjeljuje ovlasti te održava stabilnost i sigurnost aplikacije.

Mogućnost prilagodbe i daljnjeg razvoja

Sustav je razvijen modularno, što omogućuje jednostavno proširenje funkcionalnosti.

U budućnosti se može nadograditi s: - mobilnom aplikacijom za goste i osoblje,

- dinamičnim određivanjem cijena na temelju potražnje,
- integracijom s pametnim bravama (IoT) za automatsku prijavu gostiju,
- naprednim AI preporukama i analitikom poslovanja.

Opseg projektnog zadatka

Projekt obuhvaća razvoj web aplikacije koja omogućuje: - prijavu korisnika i upravljanje korisničkim ulogama,

- pregled, unos i uređivanje smještajnih jedinica,
- online rezervaciju i slanje potvrda putem e-maila,
- prikaz statistika zauzeća i izvještaja,
- responzivan dizajn i prikaz lokacije hotela putem Google Maps servisa.

Zaključak

Room-Rently donosi modernizaciju hotelskog poslovanja kroz jednostavnu, učinkovitu i prilagodljivu aplikaciju.

Kao interni sustav za jedan hotel ili iznajmljivača, omogućuje digitalizaciju rezervacija, bolju organizaciju i profesionalniji pristup gostima bez potrebe za vanjskim platformama ili provizijama.

2. Analiza zahtjeva

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Kriteriji prihvaćanja
F-001	Sustav omogućuje autentifikaciju korisnika putem OAuth2 protokola (Google Sign-In).	Visok	Korisnik se može prijaviti i registrirati putem Google računa te uspješno pristupiti sustavu.
F-002	Sustav razlikuje tri uloge korisnika: Gost, Voditelj hotela i Administrator.	Visok	Nakon prijave, korisniku se dodjeljuje odgovarajuća uloga i prikazuju funkcionalnosti prema ovlastima.
F-003	Middleware sloj	Visok	Neovlašteni korisnici nemaju pristup

ID zahtjeva	Opis	Prioritet	Kriteriji prihvaćanja
	provjerava korisničku ulogu i pristup API rutama.		zaštićenim funkcijama.
F-004	Sustav omogućuje unos, izmjenu, dohvat i brisanje smještajnih jedinica (CRUD operacije).	Visok	Voditelj hotela može uspješno unositi i uređivati smještajne jedinice.
F-005	Svaka smještajna jedinica sadrži attribute: naziv, opis, broj kreveta, cijenu, tip, dostupnost i dodatne usluge.	Visok	Podaci se ispravno prikazuju i spremaju u bazu.
F-006	Sustav validira unos podataka (npr. cijena > 0, broj kreveta ≥ 1).	Visok	Pogrešni podaci ne mogu biti spremljeni.
F-007	API omogućuje filtriranje smještajnih jedinica prema cijeni, tipu i dostupnosti.	Srednji	Rezultati pretrage odgovaraju odabranim filtrima.
F-008	Sustav omogućuje stvaranje, pregled, izmjenu i otkazivanje rezervacija (CRUD operacije).	Visok	Gost može uspješno rezervirati i otkazati smještaj.
F-009	Sustav provjerava dostupnost smještaja i sprječava preklapanje termina.	Visok	Nije moguće izvršiti rezervaciju za već zauzet termin.
F-010	Sustav generira potvrdu rezervacije u PDF formatu i šalje e-mail korisniku.	Visok	Nakon potvrde rezervacije, korisnik prima e-mail s PDF dokumentom.
F-011	Sustav omogućuje pregled i izvoz statistike o zauzeću, gostima i popularnosti usluga.	Srednji	Administrator i voditelj hotela mogu generirati izvještaje u PDF, XLSX ili XML formatu.
F-012	Sustav omogućuje prikaz smještaja na interaktivnoj karti putem	Srednji	Lokacija smještaja ispravno se prikazuje na mapi.

ID zahtjeva	Opis	Prioritet	Kriteriji prihvatanja
	Google Maps integracije.		
F-013	Sustav prikazuje dinamičnu naslovnu stranicu s tražilicom, kategorijama i istaknutim smještajima.	Srednji	Glavna stranica prikazuje ažurirane podatke i omogućuje pretragu.
F-014	Gost može putem tražilice filtrirati ponudu po lokaciji, datumu, cijeni i broju osoba.	Visok	Pretraga prikazuje relevantne rezultate.
F-015	Sustav šalje e-mail obavijesti o novim rezervacijama i promjenama statusa.	Visok	Korisnici dobivaju pravovremene obavijesti putem e-maila.

Nefunkcionalni zahtjevi

Zahtjevi performansi

ID zahtjeva	Opis	Prioritet
NF-1.1	Stranica treba učitati u razumnom roku, maksimalno do 10 sekundi.	Visok
NF-1.2	Sustav mora podržavati više istovremenih korisnika bez usporavanja rada.	Visok
NF-1.3	Pretraga smještajnih jedinica mora se izvršiti unutar nekoliko sekundi.	Srednji

Zahtjevi sigurnosti

ID zahtjeva	Opis	Prioritet
NF-2.1	Prijava i registracija provode se putem sigurnog OAuth2 protokola.	Visok
NF-2.2	Komunikacija između klijenta i poslužitelja mora se odvijati putem HTTPS protokola.	Visok
NF-2.3	Samo ovlašteni korisnici imaju pristup administracijskom dijelu sustava.	Visok
NF-2.4	Podaci o korisnicima i rezervacijama moraju biti šifrirani i zaštićeni od	Visok

ID zahtjeva	Opis	Prioritet
	neovlaštenog pristupa.	

Zahtjevi upotrebljivosti

ID zahtjeva	Opis	Prioritet
NF-3.1	Sučelje aplikacije mora biti jednostavno i pregledno.	Visok
NF-3.2	Sustav mora biti responzivan i ispravno se prikazivati na svim uređajima.	Visok
NF-3.3	Navigacija mora biti intuitivna i laka za korištenje.	Visok

Zahtjevi pouzdanosti i dostupnosti

ID zahtjeva	Opis	Prioritet
NF-4.1	Sustav treba biti dostupan korisnicima bez čestih prekida rada.	Visok
NF-4.2	Podaci o rezervacijama moraju se redovito arhivirati.	Visok
NF-4.3	U slučaju greške, sustav se mora brzo oporaviti i vratiti u stabilno stanje.	Visok

Zahtjevi održavanja i proširivosti

ID zahtjeva	Opis	Prioritet
NF-5.1	Sustav mora biti razvijen modularno radi lakšeg dodavanja novih funkcionalnosti.	Srednji
NF-5.2	Programska dokumentacija mora biti uredna i ažurna.	Visok
NF-5.3	Kôd treba biti pisan prema dogovorenim standardima i dobro komentiran.	Visok

Zahtjevi kompatibilnosti

ID zahtjeva	Opis	Prioritet
NF-6.1	Aplikacija mora raditi na najčešće korištenim preglednicima (Chrome, Firefox, Edge, Opera GX).	Visok

ID zahtjeva	Opis	Prioritet
NF-6.2	Sustav mora podržavati izvoz podataka u PDF, XML i XLSX formatu.	Visok
NF-6.3	Integracija s Google Maps servisom mora omogućiti prikaz lokacije hotela.	Visok

Zahtjevi skalabilnosti

ID zahtjeva	Opis	Prioritet
NF-7.1	Sustav mora omogućiti dodavanje novih korisnika i smještajnih jedinica bez pada performansi.	Visok
NF-7.2	Baza podataka mora biti pripremljena za povećanje količine zapisa u budućnosti.	Srednji

Dionici sustava

Dionik	Opis uloge
Gost	Krajnji korisnik koji pretražuje i rezervira smještaj.
Voditelj hotela	Upravlja smještajnim jedinicama, pregledavaju rezervacije i statistiku.
Administrator	Nadgleda cijeli sustav, kontrolira korisnike i osigurava sigurnost i stabilnost rada.
Razvojni tim	Odgovoran za implementaciju, održavanje i daljnji razvoj aplikacije.

Aktori i njihovi funkcionalni zahtjevi

ID aktora	Uloga i opis	Funkcionalnosti
A-1	Gost (inicijator)	Registracija i prijava putem Google Sign-In (F-001), pregled i filtriranje smještaja (F-007, F-014), rezervacija i otkazivanje (F-008, F-009), primanje e-mail potvrda (F-010).
A-2	Voditelj hotela (inicijator)	Upravljanje smještajnim jedinicama (F-004, F-005, F-006), pregled i potvrda rezervacija (F-008, F-009), generiranje izvještaja (F-011).
A-3	Administrator (sudionik)	Upravljanje korisnicima i nadzor nad sustavom (F-002, F-003, F-011, F-015), pristup svim podacima i statistici.

Zaključak

Analiza zahtjeva obuhvaća sve funkcionalne i nefunkcionalne potrebe Room-Rently sustava.

Funkcionalni zahtjevi definiraju osnovne procese i interakcije korisnika sa sustavom, dok nefunkcionalni zahtjevi određuju standarde kvalitete, sigurnosti i izvedbe koji su nužni za pouzdan i skalabilan rad aplikacije.

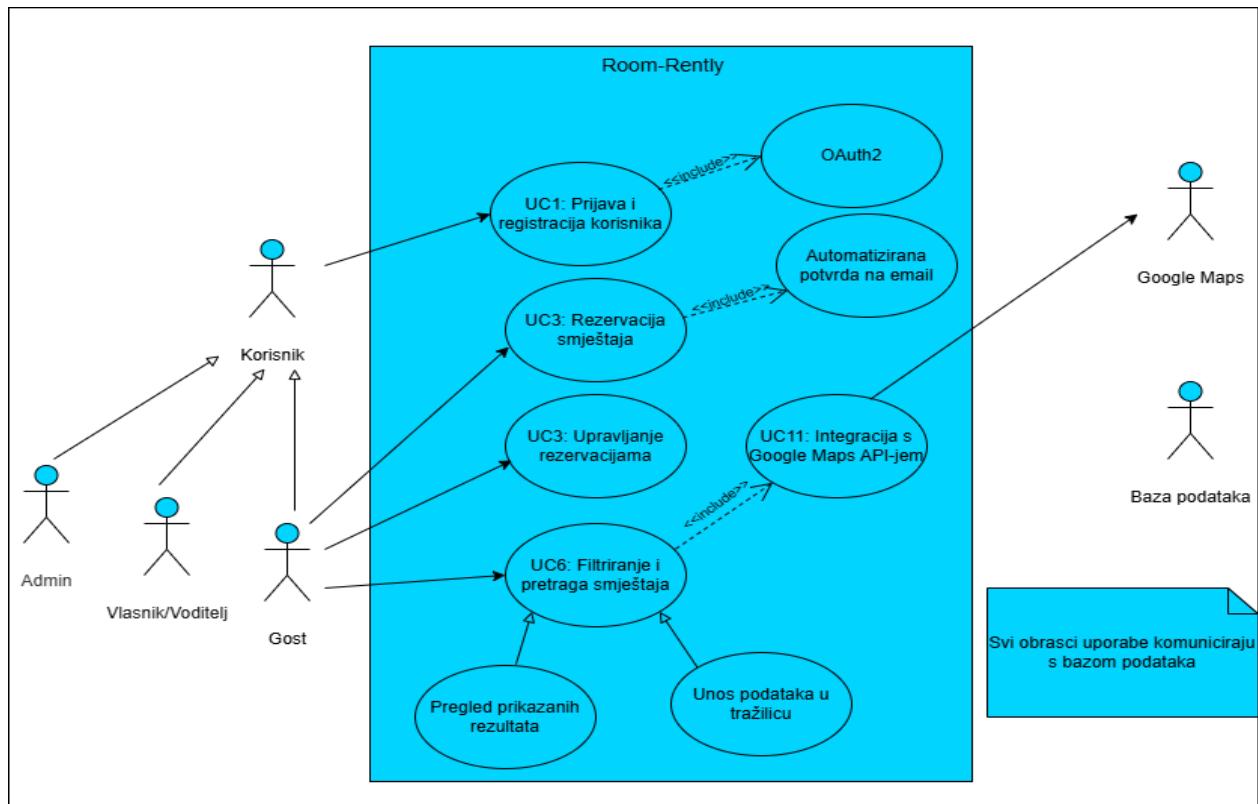
3. Specifikacija zahtjeva sustava

Obrasci uporabe

Kategorija	Use Caseovi
1. Visokorazinski dijagram obrazaca uporabe cijelog sustava	UC1: Prijava i registracija korisnikaUC2: Upravljanje smještajemUC3: Upravljanje rezervacijamaUC4: Generiranje i slanje potvrda rezervacijeUC5: Pregled izvještaja i statistike
2. Dijagram obrazaca uporabe za ključne funkcionalnosti	UC6: Filtriranje i pretraga smještajaUC7: Upravljanje dostupnošću i terminimaUC8: Upravljanje korisnicimaUC9: Slanje e-mail obavijestiUC10: Optimizacija zauzeća smještaja
3. Dijagram obrazaca uporabe za korisničke uloge	UC1: Prijava i registracija korisnikaUC2: Upravljanje smještajem (Administrator)UC3: Upravljanje rezervacijama (Gost)UC5: Pregled izvještaja i statistike (Vlasnik/Voditelj)UC7: Upravljanje dostupnošću i terminima (Vlasnik/Voditelj)UC8: Upravljanje korisnicima (Administrator)
4. Dijagram obrazaca uporabe za osnovne poslovne procese	UC3: Upravljanje rezervacijamaUC4: Generiranje i slanje potvrda rezervacijeUC5: Pregled izvještaja i statistikeUC10: Optimizacija zauzeća smještaja
5. Dijagram obrazaca uporabe za kritične sustave i integracije	UC1: Prijava i registracija korisnikaUC4: Generiranje i slanje potvrda rezervacijeUC11: Integracija s Google Maps API-jem

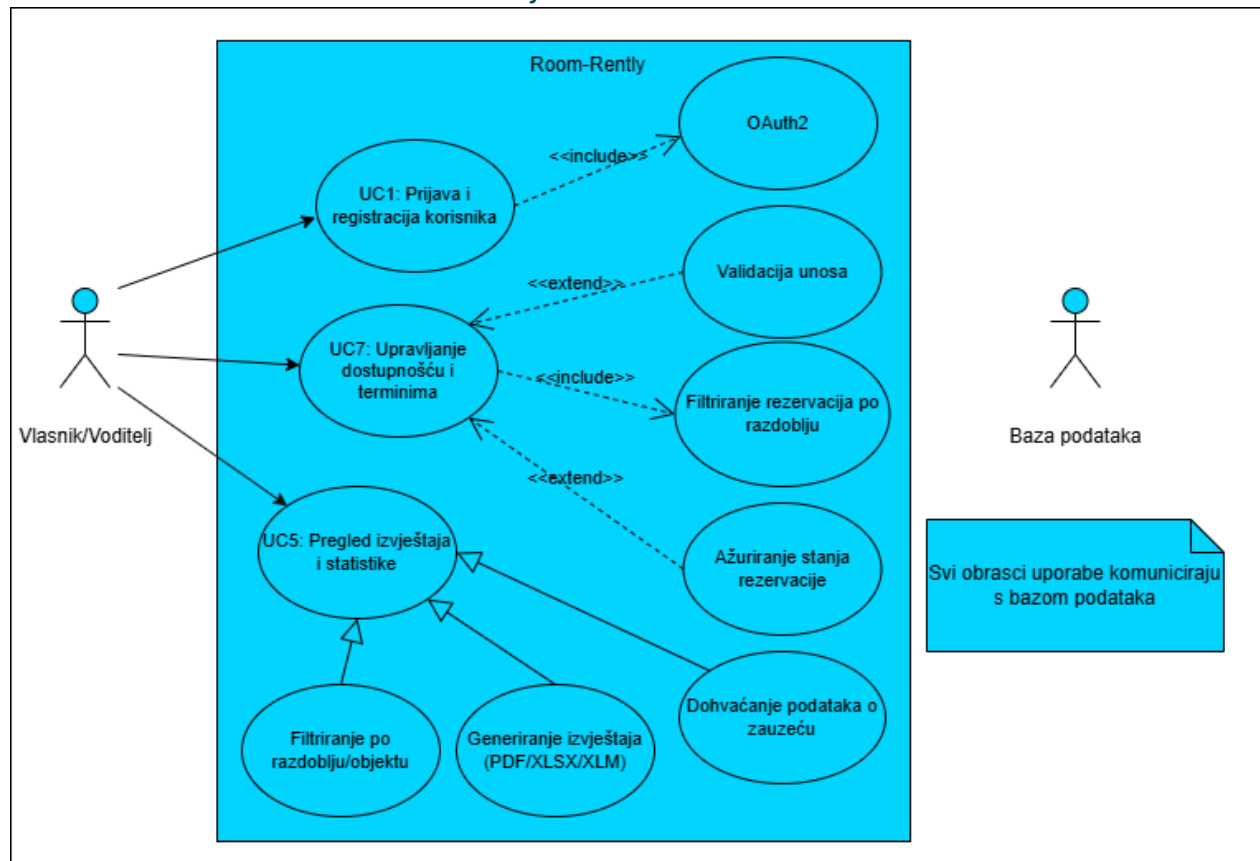
Dijagrami obrazaca uporabe

Dijagram obrazaca uporabe općenito



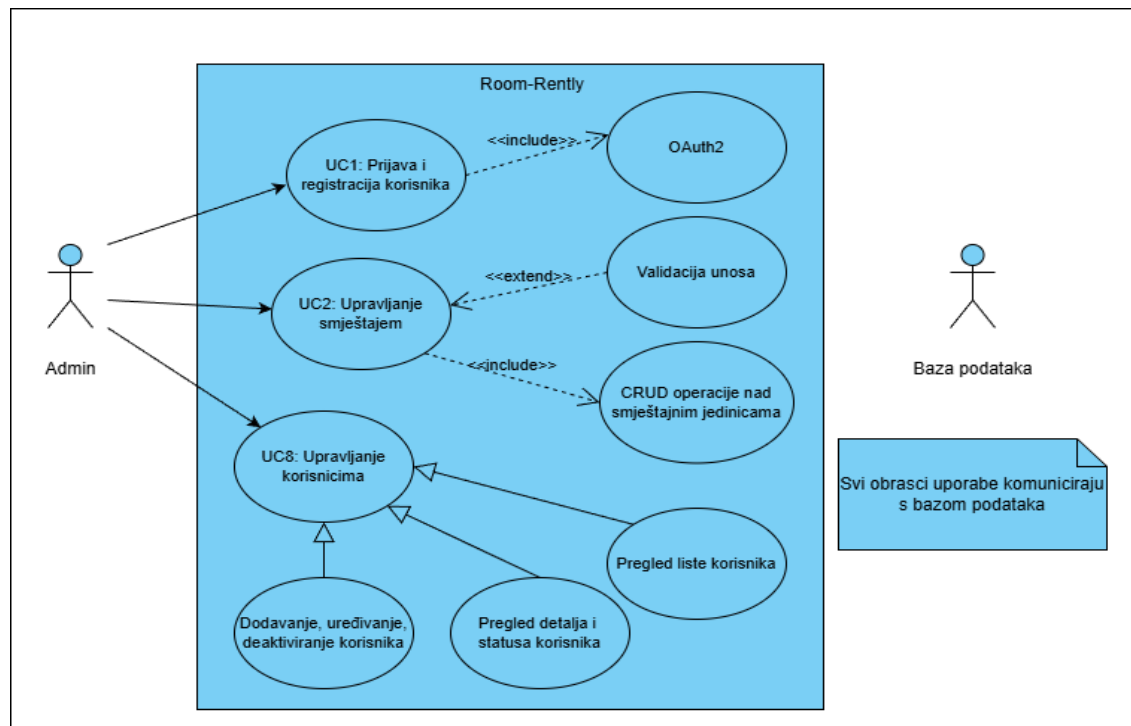
Slika 3.1 Dijagram obrazaca uporabe općenito

Funkcionalnosti vlasnika/voditelja



Slika 3.2 Funkcionalnosti Vlasnika/Voditelja

Funkcionalnosti administratora



Slika 3.3 Funkcionalnosti administratora

Opis obrazaca uporabe

UC1: Prijava i registracija korisnika

Glavni sudionik: Korisnik (Gost, Vlasnik/Voditelj, Administrator)

Cilj: Omogućiti korisniku siguran pristup sustavu putem OAuth2 (Google Sign-In).

Sudionici: Korisnik, Sustav, Google API

Preduvjet: Korisnik posjeduje Google račun.

Opis osnovnog tijeka: 1. Korisnik otvara početnu stranicu i bira opciju "Prijava" ili "Registracija" (F-001).

2. Sustav otvara OAuth2 autentifikacijski prozor.

3. Korisnik potvrđuje prijavu svojim Google računom.

4. Sustav provjerava postoje li podaci u bazi i kreira novi račun ako ne postoji.

5. Korisniku se dodjeljuje uloga (Gost, Vlasnik/Voditelj, Administrator) i preusmjerava ga na pripadajuće sučelje (F-002).

Moguća odstupanja:

- Ako autentifikacija ne uspije, korisnik dobiva obavijest o grešci i može pokušati ponovo.
- Ako Google API nije dostupan, sustav nudi privremenu lokalnu prijavu.

UC2: Upravljanje smještajem

Glavni sudionik: Administrator

Cilj: Omogućiti unos, izmjenu, brisanje i pregled smještajnih jedinica.

Sudionici: Administrator, Sustav

Preduvjet: Administrator je prijavljen u sustav.

Opis osnovnog tijeka: 1. Administrator otvara panel “Smještajne jedinice” (F-004).

2. Sustav prikazuje popis postojećih smještajnih jedinica.

3. Administrator dodaje novu jedinicu i unosi podatke (naziv, opis, broj kreveta, cijena, tip, usluge) (F-005).

4. Sustav validira unos (F-006) i pohranjuje ga u bazu.

5. Administrator može uređivati ili brisati postojeće jedinice.

Moguća odstupanja:

- Ako unos nije valjan, sustav javlja poruku o pogrešci.

- Ako jedinica ima aktivne rezervacije, brisanje nije moguće.

UC3: Upravljanje rezervacijama

Glavni sudionik: Gost

Cilj: Omogućiti stvaranje, izmjenu i otkazivanje rezervacija.

Sudionici: Gost, Sustav

Preduvjet: Gost je prijavljen i odabrao smještaj.

Opis osnovnog tijeka: 1. Gost odabire smještaj i klikne “Book Now” (F-008).

2. Sustav prikazuje formu za odabir datuma i unosa osobnih podataka.

3. Sustav provjerava dostupnost (F-009).

4. Ako je smještaj slobodan, rezervacija se pohranjuje u bazu.

5. Sustav generira potvrdu rezervacije i šalje e-mail korisniku (F-010).

Moguća odstupanja:

- Ako termin nije dostupan, sustav nudi alternativne datume.

- Ako korisnik ne potvrdi rezervaciju, ona se ne pohranjuje.

UC4: Generiranje i slanje potvrda rezervacije

Glavni sudionik: Sustav

Cilj: Automatski generirati i poslati potvrdu rezervacije e-mailom.

Sudionici: Sustav, Korisnik

Preduvjet: Rezervacija je uspješno kreirana.

Opis osnovnog tijeka: 1. Sustav prepoznaje novu potvrđenu rezervaciju (F-010).

2. Generira PDF dokument s detaljima (hotel, period, cijena, usluge).

3. Šalje e-mail korisniku s privitkom.

4. Pohranjuje potvrdu u bazu podataka.

Moguća odstupanja:

- Ako slanje e-maila ne uspije, sustav pokušava ponovno ili obavještava administratora.

UC5: Pregled izvještaja i statistike

Glavni sudionik: Vlasnik/Voditelj

Cilj: Pregledati podatke o zauzeću, gostima i uslugama te izvoziti izvještaje.

Sudionici: Vlasnik/Voditelj, Sustav

Preduvjet: Korisnik ima ulogu Vlasnika/Voditelja.

Opis osnovnog tijeka: 1. Korisnik otvara modul "Statistika" (F-011).

2. Sustav dohvaća podatke o rezervacijama i zauzeću jedinica.

3. Korisnik filtrira izvještaj po periodu.

4. Sustav prikazuje rezultate i nudi izvoz u PDF/XLSX/XML.

Moguća odstupanja:

- Ako nema dostupnih podataka, sustav prikazuje obavijest.

UC6: Filtriranje i pretraga smještaja

Glavni sudionik: Gost

Cilj: Pretražiti smještaj prema odabranim kriterijima.

Sudionici: Gost, Sustav

Preduvjet: Gost se nalazi na glavnoj stranici sustava.

Opis osnovnog tijeka: 1. Gost unosi lokaciju, datume i broj osoba u tražilicu (F-007, F-014).

2. Sustav filtrira dostupne jedinice.

3. Gost pregledava prikazane rezultate.

Moguća odstupanja:

- Ako nema rezultata, sustav prikazuje poruku o nedostupnosti smještaja.

UC7: Upravljanje dostupnošću i terminima

Glavni sudionik: Voditelj hotela

Cilj: Ažurirati dostupnost smještajnih jedinica.

Sudionici: Voditelj, Sustav

Preduvjet: Voditelj je prijavljen.

Opis osnovnog tijeka: 1. Voditelj otvara kalendar dostupnosti (F-009).

2. Odabire datume i označava zauzete termine.

3. Sustav sprema promjene u bazu.

Moguća odstupanja:

- Ako termin ima postojeću rezervaciju, izmjena nije dopuštena.

UC8: Upravljanje korisnicima

Glavni sudionik: Administrator

Cilj: Upravlјati korisničkim računima i pristupom sustavu.

Sudionici: Administrator, Sustav

Preduvjet: Administrator je prijavljen.

Opis osnovnog tijeka: 1. Administrator otvara listu korisnika (F-002, F-003).

2. Pregledava detalje i statuse korisnika.
3. Dodaje, uređuje ili deaktivira korisnike.

Moguća odstupanja:

- Ako korisnik ima aktivne rezervacije, sustav blokira brisanje.

UC9: Slanje e-mail obavijesti

Glavni sudionik: Sustav

Cilj: Obavijestiti korisnike o promjenama rezervacija.

Sudionici: Sustav, Korisnik

Preduvjet: Postoji događaj koji zahtijeva obavijest.

- Opis osnovnog tijeka:**
1. Sustav detektira događaj (potvrda, promjena, otkaz).
 2. Generira e-mail s relevantnim informacijama (F-015).
 3. Šalje obavijest korisniku.

Moguća odstupanja:

- Ako slanje ne uspije, sustav ponavlja pokušaj.

UC10: Optimizacija zauzeća smještaja

Glavni sudionik: Sustav

Cilj: Predložiti optimalan raspored rezervacija radi smanjenja praznih dana.

Sudionici: Sustav, Voditelj hotela

Preduvjet: Postoje aktivne rezervacije.

- Opis osnovnog tijeka:**
1. Sustav analizira raspored rezervacija (F-009).
 2. Predlaže optimizirane datume zauzeća.
 3. Voditelj može prihvatiti prijedlog i ažurirati kalendar.

Moguća odstupanja:

- Ako nema alternativnih termina, sustav zadržava postojeći raspored.

UC11: Integracija s Google Maps API-jem

Glavni sudionik: Sustav

Cilj: Prikazati lokaciju smještaja na interaktivnoj karti.

Sudionici: Sustav, Korisnik, Google Maps API

Preduvjet: Smještaj ima spremljene koordinate.

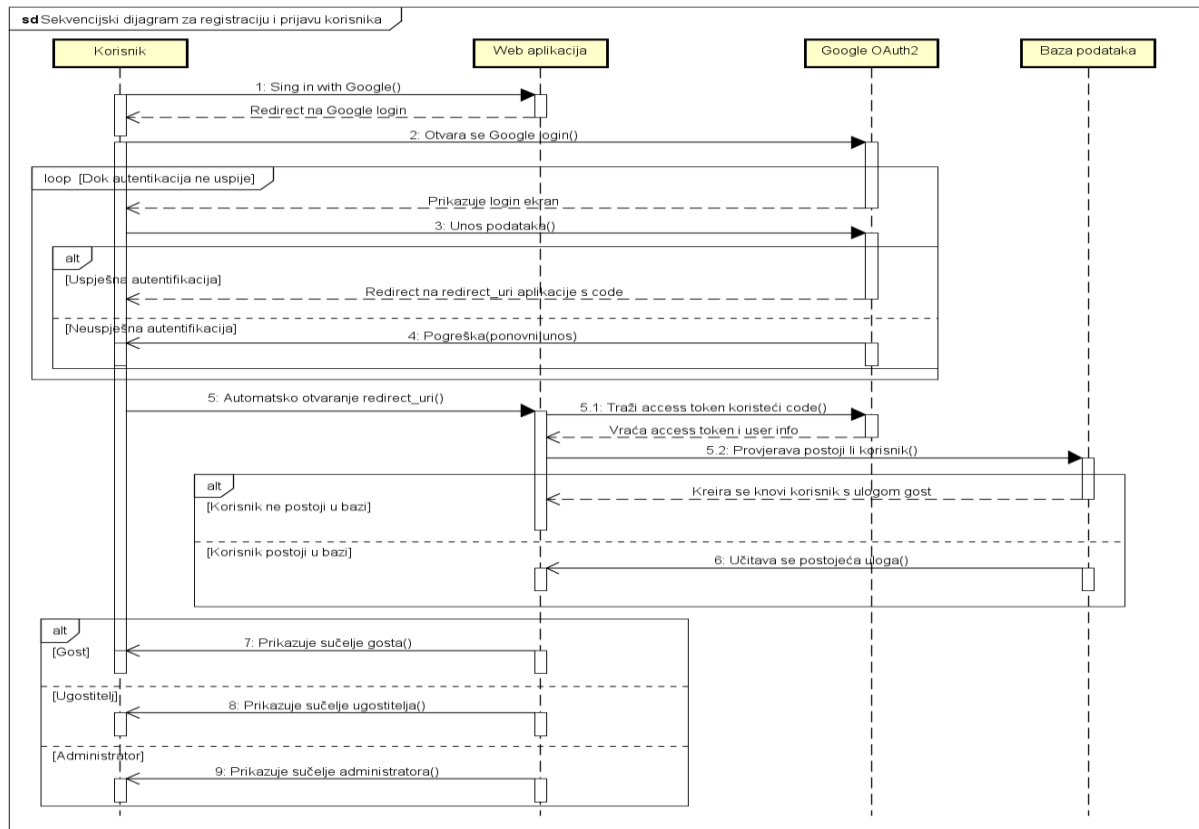
- Opis osnovnog tijeka:**
1. Korisnik otvara stranicu smještaja (F-012).
 2. Sustav dohvaća koordinate iz baze.
 3. Prikazuje lokaciju putem Google Maps API integracije.

Moguća odstupanja:

- Ako API ne odgovara, prikazuje se statična slika karte.

Sekvencijski dijagrami

Sekvencijski dijagram za registraciju i prijavu korisnika (UC1)



Slika 3.4 Sekvencijski dijagram za registraciju i prijavu korisnika

Ovaj sekvencijski dijagram prikazuje proces registracije i prijave korisnika putem Google OAuth2 autentifikacije. U njemu sudjeluju četiri glavna sudionika (lifeline-a): korisnik, Web aplikacija, Google OAuth2 i baza podataka.

Korisnik pokreće prijavu

Korisnik na web aplikaciji klikne “Sign in with Google” te ga aplikacija preusmjerava (redirect) na Googleovu stranicu za prijavu.

Google login stranica

Otvora se Google login ekran u kojemu korisnik unosi svoje Google podatke za prijavu (email, lozinku). Ovaj dio je unutar petlje (loop) te ako autentifikacija ne uspije, korisnik ponovno unosi podatke dok ne uspije.

Autentifikacija (alt grana)

Dijagram ima mogućnosti uspješne i neuspješne autentifikacije. Kod uspješne autentifikacije Google vraća aplikaciji autorizacijski kod (authorization code) putem

redirect_uri, dok se kod neuspješne autentifikacije korisniku prikazuje poruka o grešci te nudi ponovno pokušavanje.

Dobivanje access tokena i korisničkih informacija

Kada se redirect_uri automatski otvori, web aplikacija šalje zahtjev Google OAuth2 servisu odnosno traži access token koristeći code. Google vraća access token i informacije o korisniku (npr. ime, email).

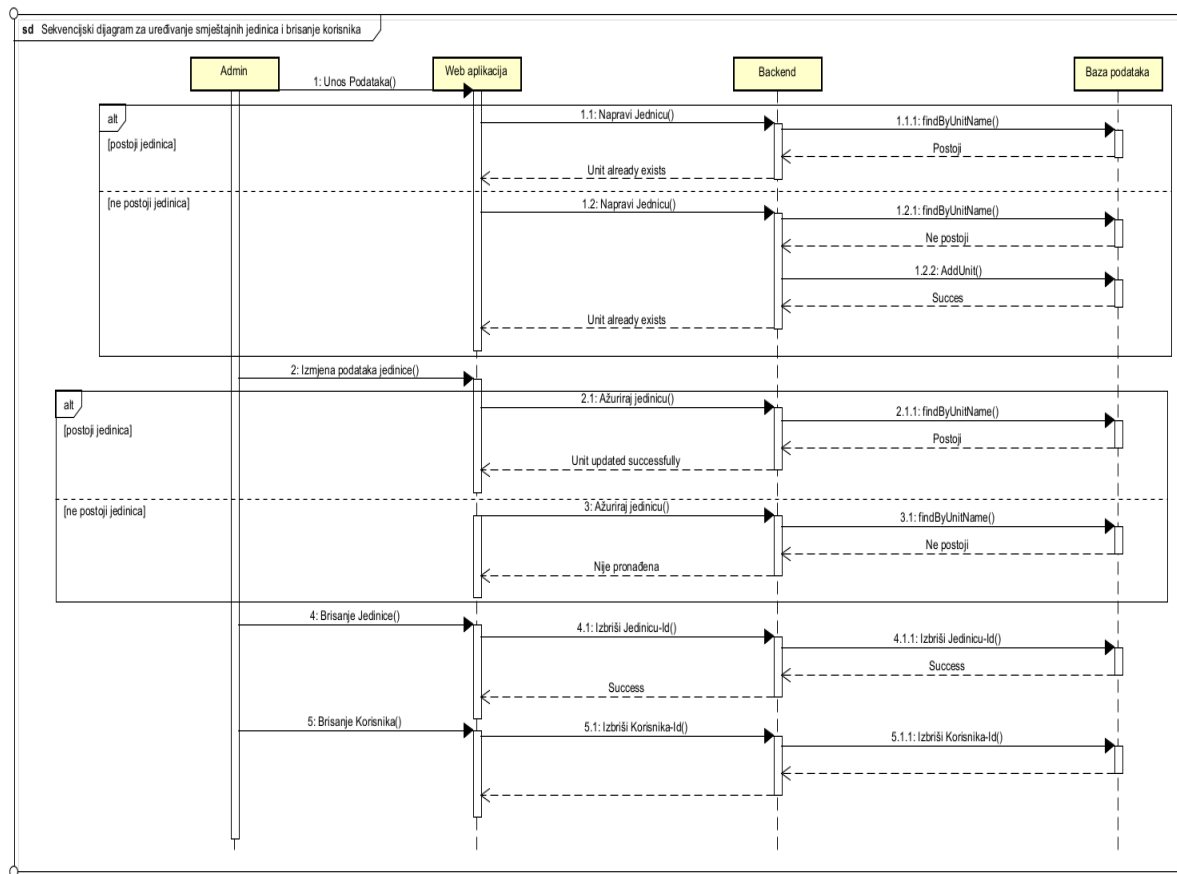
Provjera korisnika u bazi podataka

Web aplikacija provjerava postoji li korisnik u bazi podataka. Ako korisnik ne postoji, aplikacija kreira novi korisnički zapis i dodjeljuje mu početnu ulogu “Gost”, a ako korisnik već postoji, učitavaju se njegovi postojeći podaci i uloga.

Prikaz korisničkog sučelja prema ulozi

Dijagram dalje koristi alt blok da prikaže različite scenarije ovisno o ulozi korisnika. Za svakog korisnika, ovisno o tome je li gost, ugostitelj ili administrator postoji odgovarajuće sučelje.

Sekvencijski dijagram za upravljanje smještajnih jedinica i brisanje korisnika (UC2, UC8)



Slika 3.5 Sekvencijski dijagram za upravljanje smještajnih jedinica i brisanje korisnika

Ovaj sekvencijski dijagram prikazuje procese dodavanja, ažuriranja i uklanjanja smještajnih jedinica te brisanja korisnika. U dijagramu sudjeluju četiri glavna sudionika (lifeline-a): Admin, Web aplikacija, Backend i Baza podataka.

Admin pokreće akciju

Admin u web aplikaciji odabire jednu od dostupnih akcija:

- dodavanje smještajne jedinice
- ažuriranje postojeće smještajne jedinice
- uklanjanje smještajne jedinice
- brisanje korisnika Web aplikacija prikazuje odgovarajuću formu ili potvrdu akcije.

Provjera postojanja naziva (alt grana — samo za dodavanje i ažuriranje smještaja)

Kada admin unese podatke za novu smještajnu jedinicu ili ažurira postojeću, web aplikacija šalje zahtjev Backend-u. Backend zatim provjerava u Bazi podataka postoji li smještajna jedinica s istim nazivom.

Dodavanje i ažuriranje smještajne jedinice

U slučaju da naziv ne postoji ili se nije mijenjao, Backend upisuje podatke u Bazu:

- kod dodavanja: sprema novi zapis smještajne jedinice
- kod ažuriranja: modificira postojeće podatke. Nakon uspješne operacije Baza podataka vraća potvrdu Backend-u, koji prosljeđuje Web aplikaciji poruku o uspjehu.

Uklanjanje smještajne jedinice

Web aplikacija šalje zahtjev Backend-u, koji zatim upućuje naredbu Bazi podataka za brisanje zapisa. Nakon završetka operacije, Backend šalje Web aplikaciji poruku o uspješnom brisanju.

Brisanje korisnika

Proces je isti kao kod brisanja smještajne jedinice. Admin odabire korisnika kojeg želi izbrisati. Zatim Web aplikacija šalje zahtjev Backend-u te Backend šalje naredbu Bazi podataka kako bi izbrisala korisnika. Nakon uspjeha, Web aplikacija prikazuje poruku adminu.

Provjera uključenosti funkcionalnosti u obrasce uporabe

Funkcionalni zahtjev	Obrazac uporabe
F-001	UC1
F-002	UC1, UC8
F-003	UC8
F-004	UC2
F-005	UC2
F-006	UC2
F-007	UC6
F-008	UC3
F-009	UC3, UC7, UC10
F-010	UC4, UC9
F-011	UC5
F-012	UC11
F-013	UC6

Funkcionalni zahtjev	Obrazac uporabe
F-014	UC6
F-015	UC9

4. Arhitektura sustava

Arhitektura sustava – Room-Rently

Cilj poglavlja

Cilj ovog poglavlja je pružiti jasan, sažet i strukturiran pregled arhitekture sustava Room-Rently, uz razrađivanje ključnih komponenata i njihovih međusobnih odnosa. Dokumentacija omogućuje svim sudionicima projekta potpuno razumijevanje arhitekture, podjele odgovornosti i načina funkcioniranja sustava kao cjeline.

Opis arhitekture

Stil arhitekture

Room-Rently je razvijen koristeći **klijent-poslužitelj arhitekturu** s jasno odvojenim frontend i backend slojevima.

- **Frontend:** Web aplikacija (React.js) odgovorna za interakciju s korisnikom.
- **Backend:** HTTP endpoint servis (Spring Boot) koji upravlja poslovnom logikom, autentifikacijom i pristupom podacima.
- **Razlozi odabira:**
 - Jednostavnost implementacije i održavanja
 - Jasna podjela odgovornosti između korisničkog sučelja i poslovne logike
 - Skalabilnost za buduće proširenje (mobilna aplikacija, integracija IoT uređaja)

Podsustavi

1. **Autentifikacija i autorizacija** – upravljanje prijavom, autentifikacijom i pristupom sustavu prema ulogama (Gost, Voditelj hotela, Administrator).
2. **Upravljanje smještajem** – CRUD operacije nad smještajnim jedinicama, validacija i filtriranje.
3. **Upravljanje rezervacijama** – kreiranje, izmjena i otkazivanje rezervacija, provjera dostupnosti i sprječavanje preklapanja termina.
4. **Notifikacije i potvrde** – automatsko generiranje PDF potvrda i slanje e-mail obavijesti korisnicima.

5. **Statistika i izvještaji** – pregled i izvoz podataka o zauzeću, gostima i popularnosti usluga.

Preslikavanje na radnu platformu

- Sustav se može implementirati na **lokalne servere** ili **cloud infrastrukturu** (npr. AWS, Azure) radi skalabilnosti i lakše dostupnosti.
- Cloud rješenje omogućuje jednostavno horizontalno skaliranje u slučaju povećanog broja korisnika i rezervacija.

Spremišta podataka

- **Relacijska baza podataka (PostgreSQL)** za pohranu korisničkih podataka, smještajnih jedinica i rezervacija.
- Svaka glavna komponenta može imati vlastitu bazu radi izolacije podataka i sigurnosti.

Mrežni protokoli

- **HTTPS** za siguran prijenos podataka između klijenta i poslužitelja.
- **HTTP endpoint komunikacija** između frontend i backend slojeva, gdje frontend šalje zahtjeve na definirane HTTP rute, a backend vraća odgovarajuće odgovore (JSON za podatke, PDF ili statične datoteke po potrebi).

Globalni upravljački tok

1. Korisnik šalje zahtjev putem web aplikacije.
2. Backend provjerava autentifikaciju i autorizaciju.
3. Sustav obrađuje zahtjev, komunicira s bazom podataka i generira odgovor.
4. Rezultat se šalje korisniku u obliku prikaza na web sučelju, PDF potvrde ili e-mail obavijesti.

Sklopovskoprogramski zahtjevi

- Server: minimalno 2 CPU jezgre, 4 GB RAM.
- Podržani OS: Windows Server.
- Web preglednici: Chrome, Firefox, Edge, Opera GX.

Obrazloženje odabira arhitekture

Principi oblikovanja

- **Visoka kohezija** – funkcionalnosti su grupirane prema podsustavima.
- **Niska povezanost** – frontend i backend su odvojeni, olakšava održavanje i nadogradnju.
- **Fleksibilnost i skalabilnost** – modularna arhitektura omogućuje buduće proširenje (mobilna aplikacija, IoT integracija).
- **Sigurnost** – autentifikacija putem OAuth2 i HTTPS protokol.

Razmatrane alternative

Organizacija sustava na visokoj razini

- **Klijent-poslužitelj:** Frontend (React) komunicira s backend HTTP endpoint servisom.
- **Baza podataka:** PostgreSQL relacijska baza za sve ključne podatke.
- **Datotečni sustav:** Pohrana PDF potvrda rezervacija i privitaka.
- **Grafičko sučelje:** Web aplikacija povezana s backendom putem HTTP endpoint komunikacije, responzivna i intuitivna.

Organizacija aplikacije

Frontend i Backend slojevi

- **Frontend:** prezentacijski sloj, prikazuje podatke korisniku, šalje zahtjeve backendu.
- **Backend:** poslovna logika, kontrola pristupa, obrada podataka i komunikacija s bazom.

MVC Arhitektura

Arhitektura sustava Room-Rently temelji se na MVC (Model–View–Controller) obrascu, koji omogućuje jasno odvajanje podataka, poslovne logike i korisničkog sučelja. Ovakav pristup olakšava razvoj, održavanje i nadogradnju aplikacije jer se promjene u jednom sloju ne odražavaju izravno na druge.

Model

Model sloj predstavlja temelj aplikacije, zadužen za upravljanje podacima i implementaciju poslovnih pravila. U ovom sloju definiraju se klase koje odgovaraju entitetima u bazi

podataka primjerice korisnik, rezervacija, smještajna jedinica, kategorija i dodatna usluga. Model komunicira s PostgreSQL bazom podataka te omogućuje sigurno dohvaćanje, pohranu i ažuriranje informacija.

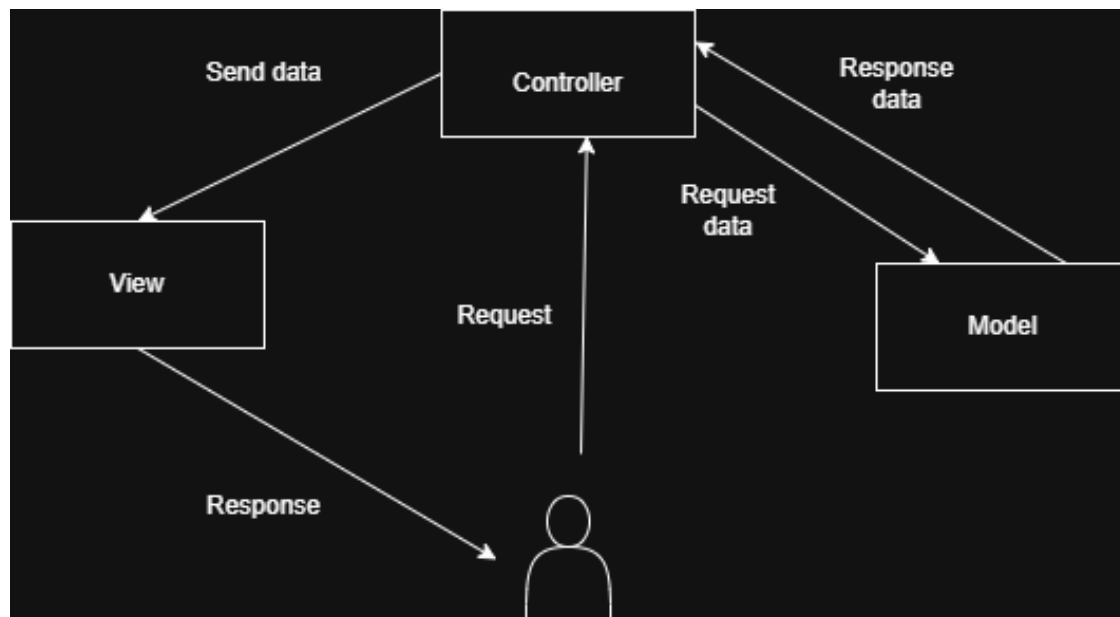
View

View sloj odgovoran je za vizualni prikaz podataka korisniku i interakciju s aplikacijom. U sustavu Room-Rently ovu ulogu preuzima frontend aplikacija izrađena pomoću React okvira. Frontend koristi React Router za navigaciju između različitih stranica i komponenti aplikacije, što omogućuje jednostraničnu strukturu s brzim i dinamičnim prijelazima bez ponovnog učitavanja stranice. Cilj View sloja je omogućiti korisnicima intuitivan pregled dostupnih soba, stvaranje i upravljanje novih jedinica te pregled profila bez potrebe za izravnim kontaktom s poslužiteljem ili bazom podataka.

Upravljač

Controller sloj djeluje kao posrednik između View i Model sloja. Njegova je zadaća primati korisničke zahtjeve s frontend aplikacije (npr. prijava, rezervacija, dodavanje nove jedinice,...), prosljeđivati ih servisima u model sloju te vraćati rezultate natrag View sloju. U sustavu Room-Rently, kontroleri su implementirani unutar Spring Boot okvira i definiraju REST API rute koje React frontend koristi za komunikaciju. Ovakva struktura omogućuje jasnu razdvojenost poslovne logike od prezentacijskog sloja, što znatno pojednostavljuje testiranje, održavanje i buduće nadogradnje aplikacije.

MVC model



Slika 4.1: Prikaz MVC modela sustava Room-Rently

Ključne komponente

- **Autentifikacija i autorizacija**
- **Upravljanje smještajem**
- **Upravljanje rezervacijama**
- **Notifikacije i potvrde**
- **Statistika i izvještaji**
- **Baza podataka** – PostgreSQL
- **Vanjski servisi** – Google OAuth2, e-mail servisi, Google Maps API

Interakcija između frontenda i backenda

- Frontend (React) komunicira s backendom (Spring Boot) putem HTTP zahtjeva poslanih pomoću Axios biblioteke. Axios omogućuje slanje GET, POST, PUT i DELETE zahtjeva prema REST API endpointima na backendu. Svi zahtjevi sadrže odgovarajuće zaglavlje s autentifikacijskim tokenom (Authorization: Bearer), čime se osigurava sigurna komunikacija i provjera korisničkog identiteta.
- Backend obrađuje zahtjeve unutar Spring Boot kontrolera (npr. PersonController.java), koji su mapirani na odgovarajuće rute pomoću anotacija poput @PostMapping, @GetMapping i @RequestMapping. Backend zatim poziva servisni sloj i komunicira s bazom podataka kako bi vratio ili pohranio tražene informacije.

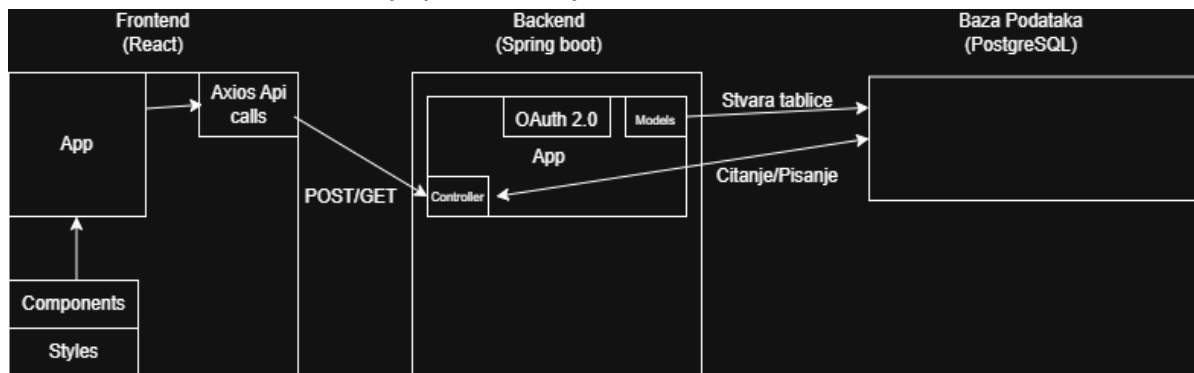
Prijava i autentifikacija (OAuth 2.0)

- Autentifikacija korisnika odvija se pomoću Google OAuth 2.0. Korisnik se prijavljuje putem Google računa, a Google API vraća ID token (JWT) koji se prosljeđuje backendu. Backend validira taj token pomoću sigurnosnog mehanizma OAuth 2.0 i: provjerava autentičnost korisnika, po potrebi pohranjuje korisnika u bazu podataka, i vraća status zahtjeva frontend aplikaciji.
- Na frontend strani, token se lokalno pohranjuje (localStorage) kako bi korisnik ostao prijavljen i nakon ponovnog učitavanja stranice.

Zaštita ruta

- Na temelju prisutnosti i valjanosti tokena, frontend kontrolira pristup određenim komponentama aplikacije. Korisnici koji nisu prijavljeni ne mogu pristupiti zaštićenim rutama poput korisničkog profila ili stranice za rezervaciju smještaja.

Ovo se ostvaruje pomoću React Router-a, koji preusmjerava korisnika na login stranicu ako token nije prisutan ili je nevažeći.



Slika 4.2 Dijagram organizacije aplikacije

Baza podataka

Baza podataka predstavlja temelj našeg sustava jer nam omogućuje strukturiranu pohranu, dodavanje, promjenu i pretraživanje podataka te kontrolirani pristup. Temeljni element naše baze je entitet koji posjeduje skup svojstava ili atributa koji ga karakteriziraju. U našoj bazi definirali smo 4 entiteta za potrebe uspješnog vođenja evidencije poslovanja i upravljanja podacima. Ti atributi su redom: + Person + Unit + UnitImg + UnitReservation

Opis tablica

Person

Entitet Person sadrži sve važne informacije o korisniku aplikacije. Sadrži sljedeće atribute: id, email, is_admin, is_user, is_owner, name.

Povezan je One-to-many vezom s entitetom UnitReservation, gdje se veza ostvaruje preko atributa id (Person može imati više rezervacija).

Funkcionalno, entitet Person predstavlja sve korisnike sustava i omogućuje razlikovanje uloga pomoću atributa is_admin, is_user i is_owner. Rezervacije koje korisnik stvara ili upravlja pohranjuju se u tablicu UnitReservation.

Atribut	Tip podataka	Opis atributa
id	BIGINT	jedinstveni identifikator osobe (primarni ključ)
email	TEXT	jedinstvena email adresa osobe

Atribut	Tip podataka	Opis atributa
is_admin	BOOLEAN	oznaka je li osoba administrator
is_user	BOOLEAN	oznaka je li osoba gost
is_owner	BOOLEAN	oznaka je li osoba vlasnik/voditelj
name	TEXT	ime osobe

Unit

Entitet Unit sadrži sve važne informacije o smještajnim jedinicama unutar aplikacije. Sadrži sljedeće attribute: id_unit, price, num_rooms, cap_adults, cap_children, num_beds, rating, has_parking, has_wifi, has_breakfast, has_towels, has_shampoo, has_hair_dryer, has_heater, has_air_conditioning, apartment, unit_name, location, main_desc_name, main_desc_content, sec_desc_name, sec_desc_content.

Entitet ima One-to-many veze s entitetima UnitImg i UnitReservation, gdje se veze ostvaruju preko atributa id_unit.

Funkcionalno, entitet Unit omogućuje pohranu svih relevantnih podataka o smještajnim jedinicama, uključujući kapacitete, pogodnosti, detaljne opise i slike, te povezuje jedinicu s rezervacijama koje korisnici kreiraju u sustavu.

Atribut	Tip podataka	Opis atributa
id_unit	BIGINT	jedinstveni identifikator smještajne jedinice (primarni ključ)
price	INTEGER	cijena smještajne jedinice
num_rooms	INTEGER	broj soba u jedinici

Atribut	Tip podataka	Opis atributa
cap_adults	INTEGER	smještajni kapacitet za odrasle osobe
cap_children	INTEGER	smještajni kapacitet za odrasle djecu
num_beds	INTEGER	broj kreveta u jedinici
rating	INTEGER	ocjena jedinice
has_parking	BOOLEAN	oznaka postoji li parking
has_wifi	BOOLEAN	oznaka postoji li Wi-Fi
has_breakfast	BOOLEAN	oznaka uključuje li doručak
has_towels	BOOLEAN	oznaka uključuju li se ručnici
has_shampoo	BOOLEAN	oznaka uključuje li šampon
has_hair_dryer	BOOLEAN	oznaka uključuje li sušilo za kosu
has_heater	BOOLEAN	oznaka uključuje li grijač
has_air_conditioning	BOOLEAN	oznaka uključuje li klima uređaj
apartment	BOOLEAN	oznaka je li jedinica apartman
unit_name	TEXT	naziv smještajne jedinice

Atribut	Tip podataka	Opis atributa
location	TEXT	lokacija smještajne jedinice
main_desc_name	TEXT	naziv glavnog opisa jedinice
main_desc_content	TEXT	sadržaj glavnog opisa jedinice
sec_desc_name	TEXT	naziv sekundarnog opisa jedinice
sec_desc_content	TEXT	sadržaj sekundarnog opisa jedinice

UnitReservation

Entitet UnitReservation sadrži sve važne informacije o rezervacijama smještajnih jedinica unutar aplikacije. Sadrži sljedeće atribute: id_unit_reservation, start_date, end_date, status, person, unit.

Entitet ima Many-to-one veze s entitetima Person i Unit, gdje se veze ostvaruju preko atributa id (Person) i id_unit (Unit).

Funkcionalno, entitet UnitReservation omogućuje pohranu svih rezervacija koje korisnici kreiraju u sustavu. Povezuje korisnika i smještajnu jedinicu te čuva podatke o datumu početka, završetka i statusu rezervacije, što omogućuje daljnju obradu poput generiranja potvrda i provjere zauzetosti jedinica.

Atribut	Tip podataka	Opis atributa
id_unit_reservation	BIGINT	jedinstveni identifikator rezervacije (primarni ključ)
start_date	DATE	datum početka rezervacije
end_date	DATE	datum završetka rezervacije
status	TEXT	status rezervacije
id	BIGINT	identifikator korisnika koji je kreirao rezervaciju (strani ključ prema Person)

Atribut	Tip podataka	Opis atributa
id_unit	BIGINT	identifikator smještajne jedinice koja je rezervirana (strani ključ prema Unit)

UnitImg

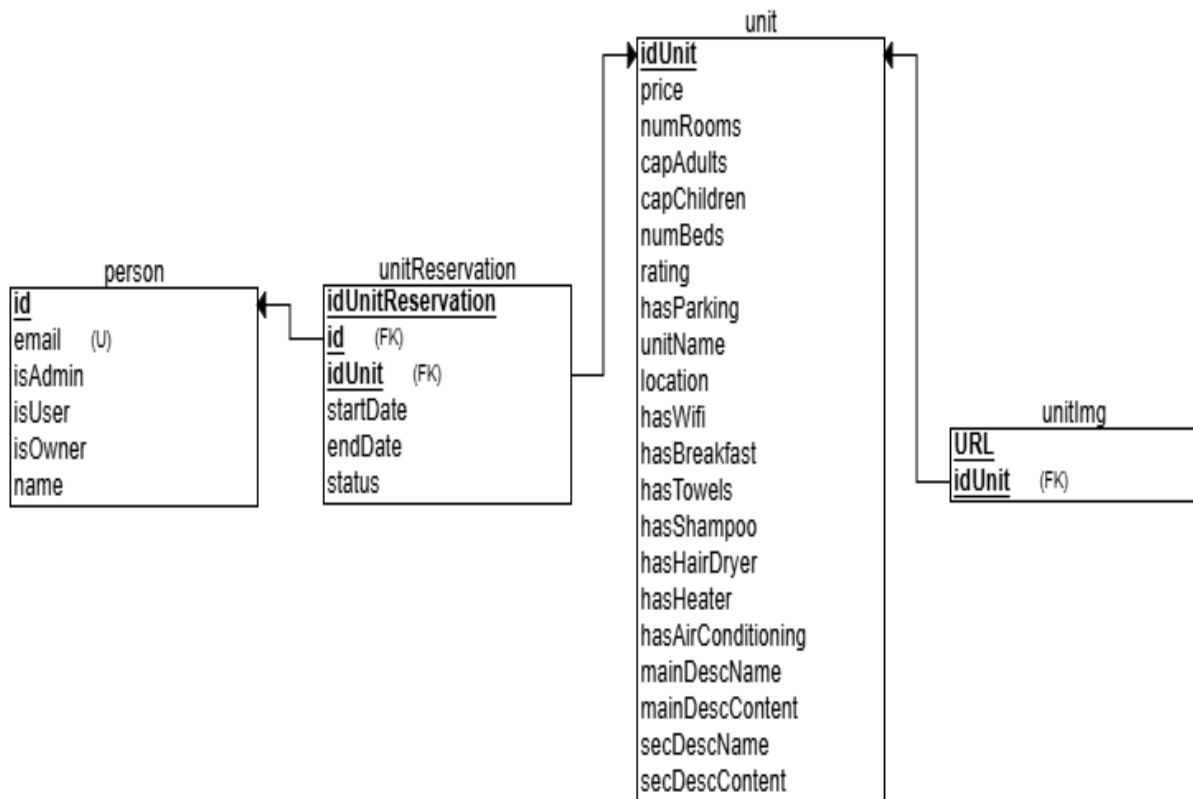
Entitet UnitImg sadrži sve važne informacije o slikama smještajnih jedinica unutar aplikacije. Sadrži sljedeće attribute: url, unit.

Entitet ima Many-to-one vezu s entitetom Unit, gdje se veza ostvaruje preko atributa id_unit.

Funkcionalno, entitet UnitImg omogućuje pohranu URL-ova slika koje pripadaju određenoj smještajnoj jedinici.

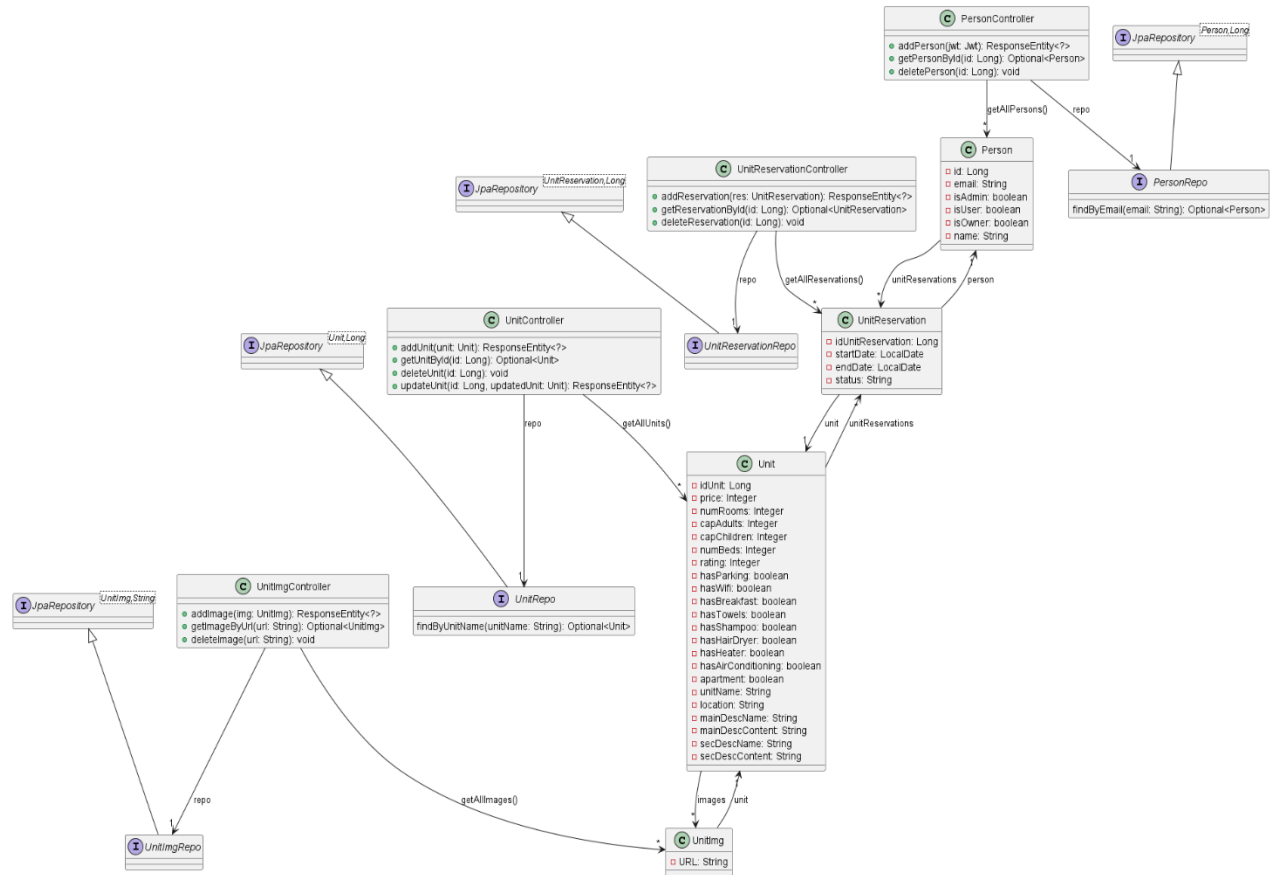
Atribut	Tip podataka	Opis atributa
url	TEXT	jedinstveni URL slike (primarni ključ)
id_unit	BIGINT	identifikator smještajne jedinice kojoj slika pripada (strani ključ prema Unit)

ER Dijagram baze podataka



Slika 4.3 ER Dijagram

Dijagram razreda



Slika 4.4 Dijagram razreda

Dijagram prikazuje skup klasa koje čine osnovu sustava za upravljanje smještajem, rezervacijama i pripadajućim podacima o korisnicima i jedinicama smještaja.

Model korisnika predstavljen je klasom **Person**, koja sadrži osnovne informacije kao što su **id**, **name**, **email**, te boolean polja **isAdmin**, **isUser** i **isOwner** koja definiraju ulogu korisnika unutar sustava. Za pristup podacima o korisnicima koristi se **PersonRepo**, dok **PersonController** omogućuje dodavanje, dohvat i brisanje korisnika.

Smještajne jedinice modelirane su klasom **Unit**, koja sadrži detalje poput broja soba (**numRooms**), kapaciteta odraslih i djece (**capAdults**, **capChildren**), cijene (**price**) te obilježja kao što su dostupnost Wi-Fi-ja, parkinga, doručka i drugih pogodnosti. Za rad s podacima o smještajnim jedinicama koristi se **UnitRepo**, dok **UnitController** omogućuje dodavanje, dohvat i brisanje smještajne jedinice.

Rezervacije smještajnih jedinica modelirane su klasom **UnitReservation**, koja sadrži **id**, **startDate**, **endDate** i **status** rezervacije. Kontroler **UnitReservationController** omogućuje dodavanje, dohvat i brisanje rezervacija, dok **UnitReservationRepo** služi za rad s bazom podataka.

Slike smještajnih jedinica predstavljene su klasom `UnitImg` sa Stringom URL koji predstavlja relativnu putanju slike. `UnitImgController` omogućuje dodavanje, dohvat i brisanje slika, dok `UnitImgRepo` komunicira s bazom podataka.

Kontroleri sustava (`UnitController`, `UnitReservationController`, `UnitImgController`, `PersonController`) koriste pripadajuće repozitorije za dohvat i manipulaciju podacima. Svaki kontroler nudi metode za osnovne CRUD operacije, a repozitoriji nasljeđuju `JpaRepository`, što omogućuje jednostavan rad s bazom podataka.

Ovim modelom osigurana je jasna povezanost između korisnika, smještajnih jedinica i rezervacija. Svaka klasa je modularno odvojena, a korištenje repozitorija omogućuje jednostavno proširenje sustava, primjerice dodavanje novih vrsta rezervacija, dodatnih informacija o korisnicima ili proširenih značajki smještajnih jedinica.

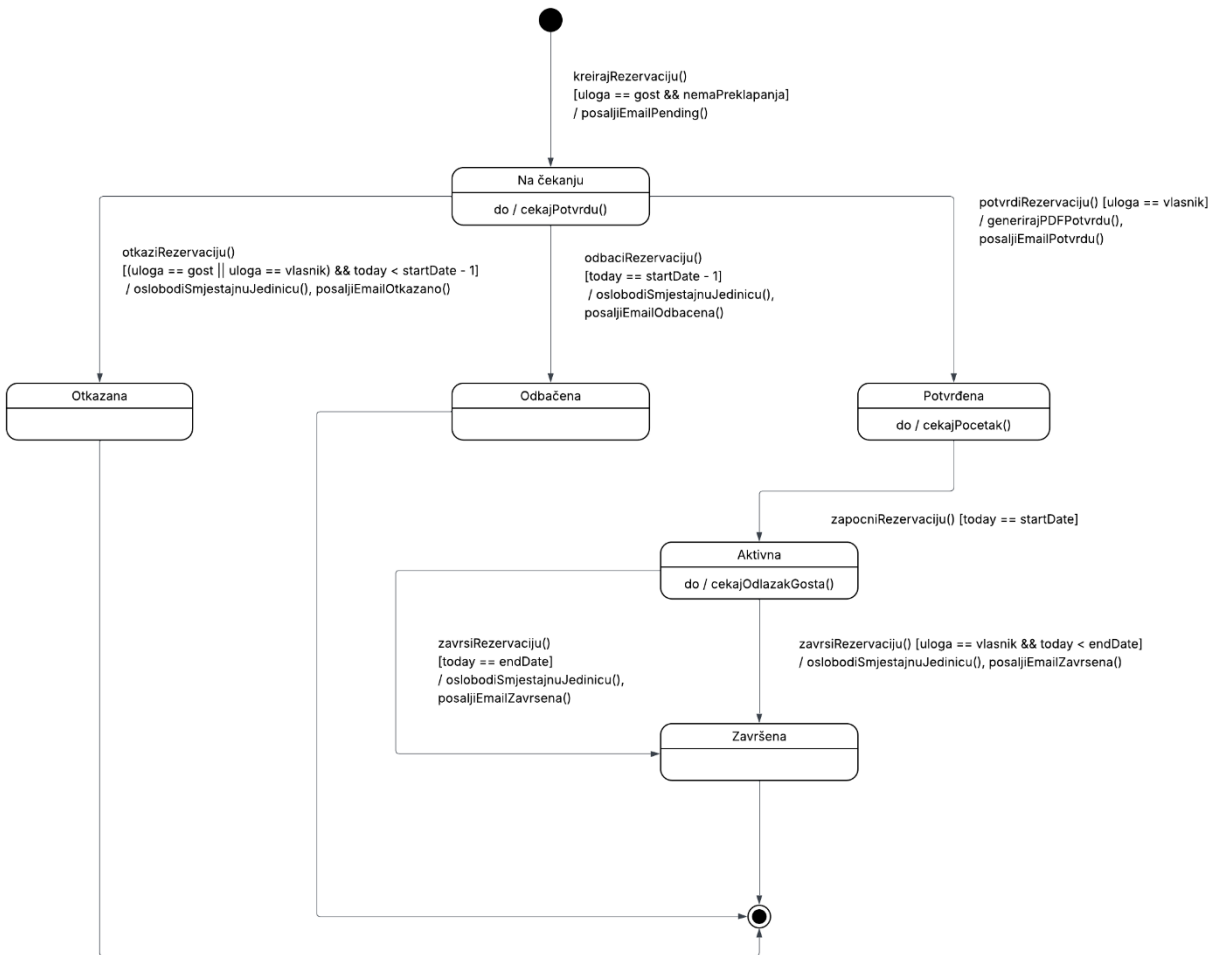
Dinamičko ponašanje aplikacije

Dinamičko ponašanje aplikacije odnosi se na način na koji objekti u sustavu evoluiraju kroz vrijeme, uključujući prijelaze između različitih stanja. To uključuje aktivnosti, događaje, odluke i interakcije unutar aplikacije. UML dijagrami stanja omogućuju vizualizaciju tih promjena i olakšavaju razumijevanje dinamike sustava.

Razumijevanje promjena stanja neophodno je za pravilno funkcioniranje aplikacije jer pruža uvid u interakcije među objektima, komponentama i korisnicima tijekom rada sustava. Korištenjem UML dijagrama stanja i aktivnosti moguće je vizualizirati prijelaze i stanja objekata, identificirati potencijalne probleme, osigurati točnu implementaciju te poboljšati komunikaciju među članovima tima.

Za dijagram stanja odabrali smo najvažniji objekt sustava, a to je rezervacija. Sukladno tome, za dijagram aktivnosti, odabrali smo proces kreiranja i potvrde rezervacije smještaja prijavljenog gosta s unesenom nacionalnosti jer prijavljivanje gosta i unos nacionalnosti nisu ključni procesi u ovom kontekstu.

UML dijagram stanja



Slika 4.5 Dijagram stanja rezervacije

UML dijagram stanja rezervacije prikazuje životni vijek objekta rezervacije. Kada gost potvrdi rezervaciju koja nema preklapanja datuma s drugim rezervacijama onda rezervacija prelazi u stanje čekanja da vlasnik potvrdi rezervaciju i šalje se e-mail potvrda da je rezervacija na čekanju. Gost može otkazati rezervaciju koja još nije potvrđena, a vlasnik ju može potvrditi ili otkazati. To je moguće do dva dana prije početka odsjedanja. Ako vlasnik ne potvrdi niti otkaže rezervaciju(kao i korisnik) do dana prije odsjedanja, onda se rezervacija odbacuje. U slučaju odbacivanja i otkazivanja rezervacije, oslobađa se smještajna jedinica za druge rezervacije, šalje se adekvatan e-mail gostu te završava životni vijek rezervacije. Kada vlasnik potvrdi rezervaciju onda se generira PDF potvrda i šalje se e-mail obavijest potvrde gostu. Kada je rezervacija potvrđena, čeka se početak rezervacije. Na dan startDate započinje trajanje rezervacije odnosno ona postaje aktivna i čeka se odlazak gosta. Gost ima opciju da se odjavi na dan isteka rezervacije ili ranije. Ako se odjavi ranije, onda vlasnik mora završiti rezervaciju, a inače se rezervacija sama završava na dan endDate. Po završetku rezervacije oslobađa se smještajna jedinica, šalje se adekvatan e-mail gostu te završava životni vijek rezervacije.

Slika 4.6 Dijagram aktivnosti rezervacije



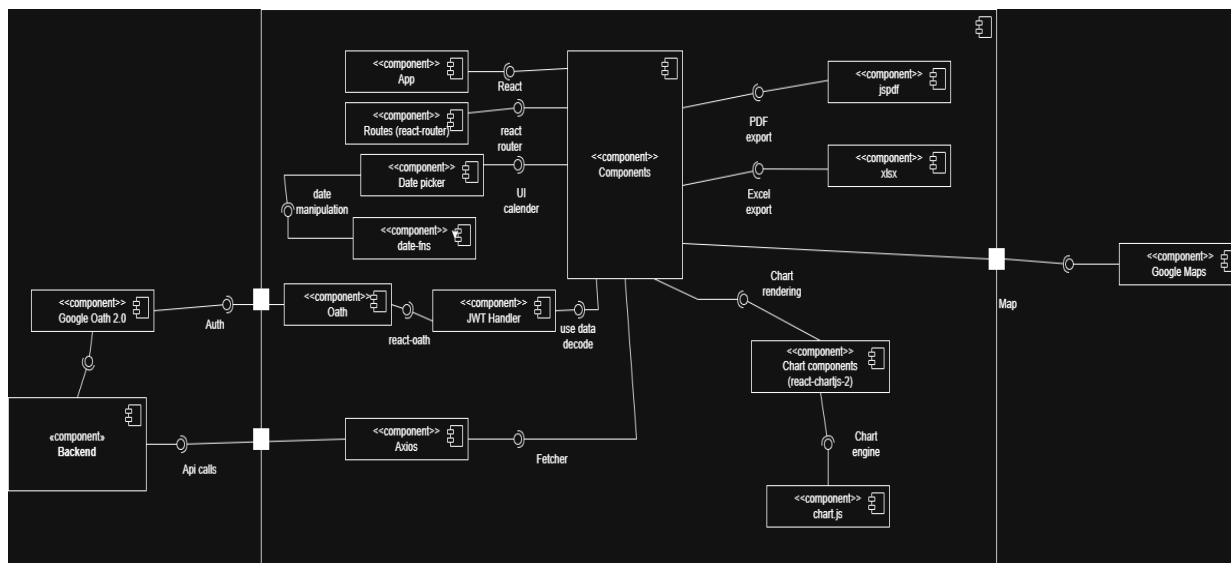
UML dijagram aktivnosti rezervacije prikazuje kreiranje i potvrdu rezervacije smještaja prijavljenog gosta s unesenom nacionalnosti. Gost prvo mora unijeti željeni raspon datuma rezervacije te ostale postavke kao što su broj odraslih osoba te tip sobe. Web aplikacija zatim od baze podataka dobiva i prikazuje popis odgovarajućih smještajnih jedinica (tipskih soba ili apartmana) za odabrane postavke. Gost odabire jednu od ponuđenih jedinica i potvrđuje odabir klikom na gumb “Reserve or Book Now”. Web aplikacija prikazuje gostu formu u kojoj on odabire specifičniji raspon datuma te dodatne usluge (WiFi, ručnici i sl.). Klikom na gumb “Confirm Reservation”, gost potvrđuje rezervaciju te Web aplikacija kreira rezervaciju. Stvara objekt rezervacije te paralelno šalje informacije bazi podataka i Brevo API-ju. Baza podataka pohrani rezervaciju kao pending, a Brevo API pošalje e-mail potvrdu da je rezervacija na čekanju te ju tako Web aplikacija i prikaže gostu. Web aplikacija zatim obavijesti vlasnika te vlasnik pregledava rezervaciju i odlučuje hoće li ju prihvatiti ili odbaciti. Osim toga, dok vlasnik nije odlučio, gost ima opciju da otkáže rezervaciju. Ako je gost otkazao rezervaciju ili je vlasnik odlučio, status rezervacije se prikazuje gostu i vlasniku. Za sva tri slučaja, Web aplikacija šalje informaciju bazi podataka kako bi ona mogla zabilježiti status rezervacije. Na poslijetku, ako je status prihvaćen, Web aplikacija generira PDF potvrdu te Brevo API šalje e-mail gostu. Inače, ako status nije prihvaćen, ne generira se PDF, nego Brevo API automatski šalje adekvatan e-mail gostu. Time završava navedena aktivnost.

5. Arhitektura komponenata i razmjesta

Frontend je razvijen korištenjem React biblioteke, a ovisnosti i korištene biblioteke definirane su u package.json datoteci. Od značajnijih paketa koriste se biblioteke za autentifikaciju putem Google računa (@react-oauth/google), rad s datumima (date-fns), rukovanje JWT tokenima (jwt-decode), te navigaciju (react-router-dom). Frontend komunicira s backendom putem HTTP zahtjeva koristeći Axios biblioteku, dok se podaci razmjenjuju u JSON formatu. Pored toga, frontend je izravno povezan s Google Maps API-jem, koji omogućuje prikaz karata, lokacija i ruta unutar korisničkog sučelja.

Dijagram Komponenti

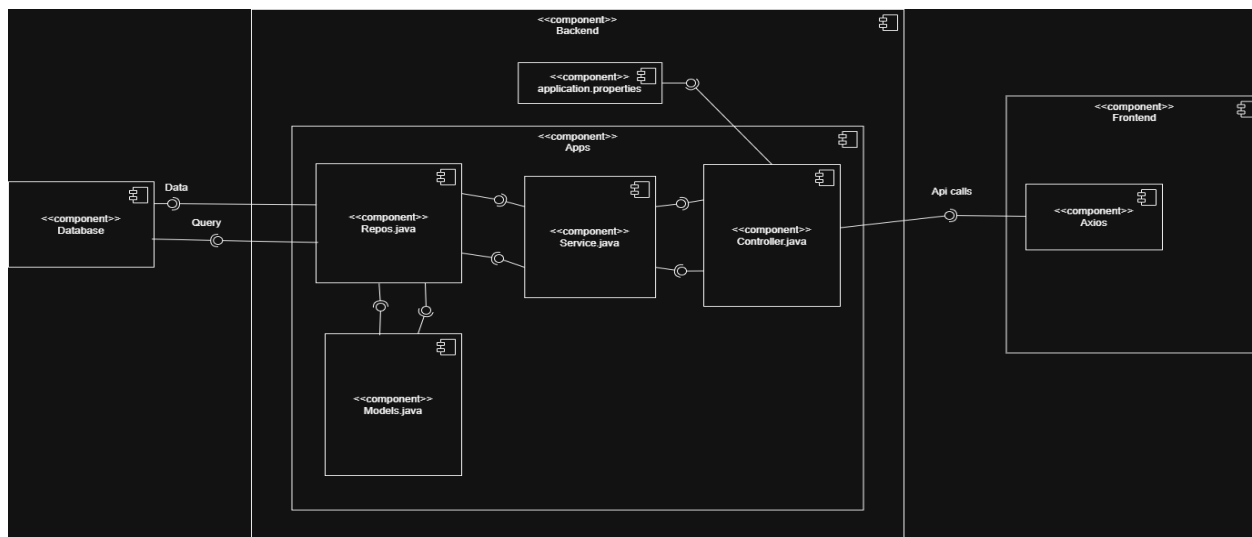
Dijagram komponenti Frontenda



Slika 5.1 Dijagram komponenti Frontenda

Na slici 5.1 prikazan je dijagram komponenti za Frontend te na slici 5.2 prikazan je dijagram komponenti za Backend. Dijagram prikazuje arhitekturu frontenda i backenda te sučelja kojima oni međusobno komuniciraju, kao i način na koji backend komunicira s bazom podataka. Backend je implementiran korištenjem Spring Boot frameworka te se sastoji od više slojeva koji doprinose modularnosti sustava. U backendu čuvaju se API ključevi potrebni za komunikaciju s bazom podataka i vanjskim servisima, primjerice sa sustavom za autentifikaciju putem OAuth 2.0.

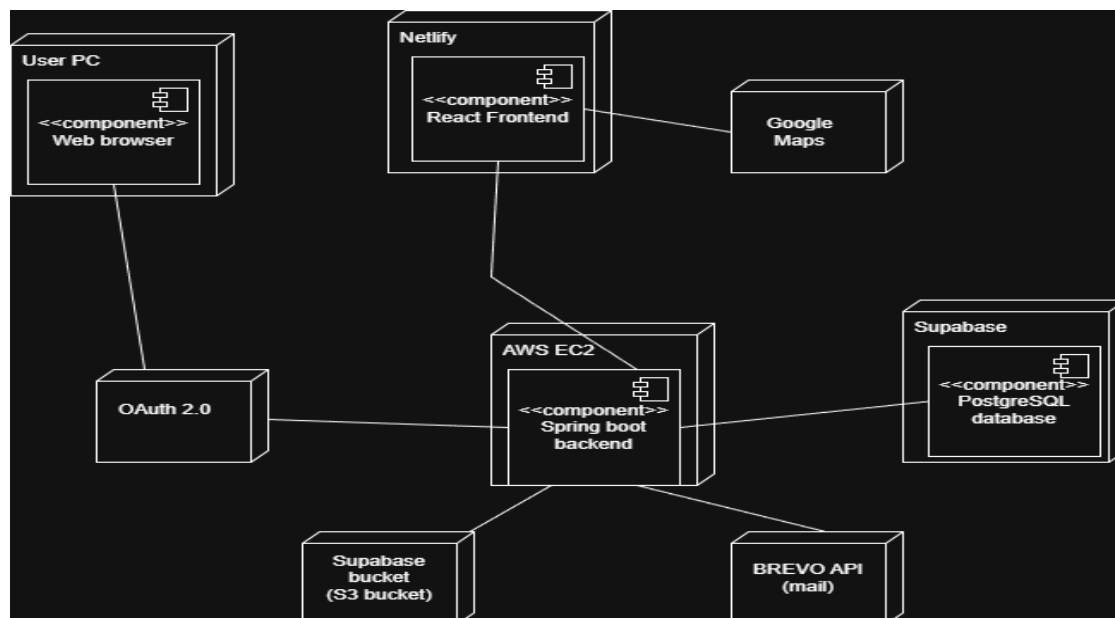
Dijagram komponenti Backenda



Slika 5.2 Dijagram komponenti Backenda

Na slici 5.2 je prikazan dijagram komponenti backenda implementiranog korištenjem Spring Boot frameworka. Dijagram ilustrira unutarnju strukturu backenda, njegove glavne slojeve te način komunikacije između pojedinih komponenti, kao i interakciju s frontendom i bazom podataka. Backend je organiziran u višeslojnu arhitekturu s ciljem postizanja modularnosti, lakšeg održavanja i jasne razdvojenosti odgovornosti. Središnji dio sustava čini komponenta Apps, unutar koje se nalaze osnovni slojevi aplikacije. Controller sloj (Controller.java) zadužen je za obradu HTTP zahtjeva pristiglih s frontenda putem API poziva. Repozitориjski sloj (Repos.java) služi za komunikaciju s bazom podataka te sadrži metode za dohvat, spremanje i obradu podataka. On koristi model sloj (Models.java), koji definira domenske entitete i strukture podataka koje se mapiraju na zapise u bazi podataka. Konfiguracijska komponenta application.properties sadrži postavke potrebne za rad aplikacije, uključujući podatke za povezivanje s bazom podataka, API ključeve te konfiguraciju sigurnosnih mehanizama. U backendu se pohranjuju osjetljivi podaci poput API ključeva i vjerodajnica potrebnih za komunikaciju s vanjskim servisima, primjerice za autentifikaciju i autorizaciju putem OAuth 2.0 protokola. Backend komunicira s bazom podataka, dok je komunikacija s frontendom ostvarena putem REST API sučelja. Ovakva arhitektura omogućuje skalabilnost, sigurnost i jasno odvajanje poslovne logike od prezentacijskog sloja. Servisni sloj (Service.java) predstavlja središnji dio poslovne logike aplikacije te djeluje kao posrednik između Controller i Repozitориjskog sloja. Njegova je uloga obrada poslovnih pravila, validacija podataka te koordinacija operacija koje uključuju rad s više repozitorija. Controller sloj delegira obradu zahtjeva servisnom sloju, čime se osigurava jasno razdvajanje odgovornosti i smanjenje kompleksnosti kontrolera. Servisni sloj omogućuje ponovnu upotrebu poslovne logike, jednostavnije testiranje aplikacije te lakše održavanje i proširivanje sustava.

Dijagram razmještaja



Slika 5.3 Dijagram razmještaja

Na slici 5.3 prikazan je dijagram razmjesta sustava koji koristi klijent–poslužitelj arhitekturu. Sustav je podijeljen na frontend, backend i bazu podataka, pri čemu je svaka komponenta razmještena na zasebnoj infrastrukturi. Frontend aplikacija razvijena u Reactu hostana je na platformi Netlify te joj korisnici pristupaju putem web preglednika. Backend aplikacija implementirana je u okviru Spring Boot tehnologije i pokrenuta na AWS EC2 poslužitelju. Baza podataka koristi PostgreSQL i smještena je na Supabase serveru.

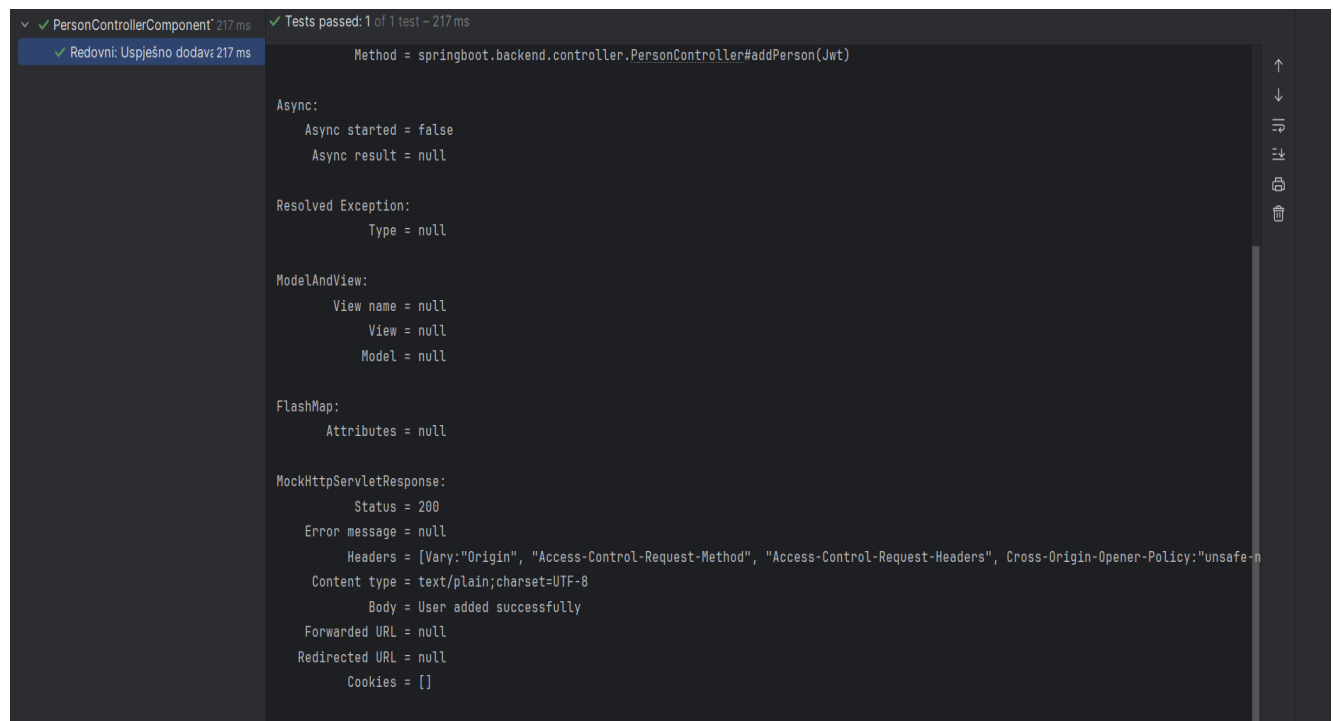
Sustav podržava autentifikaciju korisnika putem OAuth 2.0 protokola. Za prikaz i obradu geografskih podataka integriran je Google Maps API. Backend komunicira i s vanjskim servisima, uključujući Supabase bucket (S3 kompatibilnu pohranu) za spremanje datoteka te Brevo API za slanje elektroničke pošte.

6. Ispitivanje programskog rješenja

Ispitivanje komponenti

Programsko testiranje (JUnit & MockMvc): U repozitoriju se nalaze automatizirane testne klase (PersonControllerComponentTest, UnitReservationControllerComponentTest i UnitControllerComponentTest). Ovi testovi: - Simuliraju HTTP zahtjeve koristeći MockMvc. - Izoliraju logiku kontrolera od baze podataka koristeći @MockitoBean. - Provjeravaju ispravnost odgovora (Status kodove, JSON strukturu).

Izvršavanje skripti: Testovi se mogu pokrenuti izravno iz razvojnog okruženja (IDE).



Slika 6.1 Izlaz iz JUnita u IntelliJ

Ispit komponente 1 – dodavanje korisnika

Endpoint: - POST /addPerson

Preduvjet: - Korisnik mora biti ulogiran.

Ulaz očekivanog ponašanja:

- JWT token s emailom i imenom
- Korisnik ne postoji u bazi

Izlaz: - status: 200 OK - "User added successfully"

Ulaz rubnog slučaja:

- JWT token s emailom
- Korisnik već postoji u bazi

Izlaz: - status: 409 CONFLICT - "User already exists"

Ulaz nepodržanog ponašanja:

- JWT token bez email claima

Izlaz: - status: 400 BAD REQUEST - "No email in token."

Izvorni kod ispitnog slučaja

```
@Test
@DisplayName("Redovni: Uspješno dodavanje novog korisnika")
public void testAddPerson_Success() throws Exception {
    String email = "novi@test.com";
    Mockito.when(repo.findByEmail(email)).thenReturn(Optional.empty());
    Mockito.when(repo.count()).thenReturn(1L);

    mockMvc.perform(post("/addPerson")
        .with(jwt().jwt(j -> j.claim("email", email).claim("name", "Test User"))))
        .contentType(MediaType.APPLICATION_JSON))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(content().string("User added successfully"));
}

@Test
@DisplayName("Rubni: Pokušaj dodavanja korisnika koji već postoji (409 Conflict)")
public void testAddPerson_AlreadyExists() throws Exception {
    String email = "postojeci@test.com";
    Mockito.when(repo.findByEmail(email)).thenReturn(Optional.of(new Person()));
}
```

```

        mockMvc.perform(post("/addPerson")
            .with(jwt().jwt(j -> j.claim("email", email)))
            .contentType(MediaType.APPLICATION_JSON))
            .andDo(print())
            .andExpect(status().isConflict())
            .andExpect(content().string("User already exists"));
    }

    @Test
    @DisplayName("Izazivanje pogreške: Provjera RuntimeException-a")
    public void testAddPerson_ThrowsException() {
        org.junit.jupiter.api.Assertions.assertThrows(Exception.class, () ->
        {
            mockMvc.perform(post("/addPerson")
                .with(jwt().jwt(j -> j.claim("name", "Bez Emaila")))).and
Do(print());
        });
    }
}

```

Ispit komponente 2 – dohvat trenutnog korisnika

Endpoint: - GET /me

Preduvjet: - Korisnik mora biti ulogiran (valjani JWT token).

Ulaz očekivanog ponašanja:

- JWT token s claimom email = user@test.com
- Korisnik postoji u bazi podataka

Izlaz: - status: 200 OK - Vraćen objekt korisnika (Person)

Ulaz rubnog slučaja:

- JWT token s valjanim emailom
- Korisnik ne postoji u bazi

Izlaz: - status: 404 NOT FOUND - "User not found"

Ulaz neočekivanog ponašanja:

- JWT token nije poslan

Izlaz: - status: 400 BAD REQUEST - "Invalid token"

Izvorni kod ispitnog slučaja

```

@Test
@DisplayName("Redovni: Dohvat postojećeg korisnika (/me)")
public void testGetCurrentUser_Success() throws Exception {
    String email = "postojeci@test.com";
}

```

```

    Person p = new Person();
    p.setEmail(email);
    p.setName("Ivan Horvat");

    // Simuliramo da repo pronalazi korisnika
    Mockito.when(repo.findByEmail(email)).thenReturn(Optional.of(p));

    mockMvc.perform(get("/me")
        .with(jwt().jwt(j -> j.claim("email", email)))
        .contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.email").value(email))
        .andExpect(jsonPath("$.name").value("Ivan Horvat"));
}

@Test
@DisplayName("Rubni: Korisnik s ispravnim tokenom ne postoji u bazi")
public void testGetCurrentUser_NotFound() throws Exception {
    String email = "nepoznat@test.com";

    // Simuliramo da repo NE pronalazi korisnika
    Mockito.when(repo.findByEmail(email)).thenReturn(Optional.empty());

    mockMvc.perform(get("/me")
        .with(jwt().jwt(j -> j.claim("email", email)))
        .contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isNotFound())
        .andExpect(content().string("User not found"));
}

@Test
@DisplayName("Izazivanje pogreške: Token bez email claima")
public void testGetCurrentUser_InvalidToken() throws Exception {
    // Šaljemo JWT bez "email" claima
    mockMvc.perform(get("/me")
        .with(jwt().jwt(j -> j.claim("name", "Only Name")))
        .contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isBadRequest())
        .andExpect(content().string("Invalid token"));
}

```

Ispit komponente 3 – dodavanje smještajne jedinice

Endpoint: - POST /unit/add

Preduvjet: - Korisnik mora biti ulogiran i imati admin ulogu.

Ulaz očekivanog ponašanja:

```
{
  "unitName": "Hotel Central",
  "price": 100,
  "capAdults": 2,
  "apartment": false
  "numSameRooms": 2
}
```

Izlaz: - status: 200 OK - Vraćen spremljeni objekt Unit

Ulaz rubnog slučaja:

```
{
  "unitName": "Postojeci"
}
```

Izlaz: - status: 409 CONFLICT - "Unit already exists"

Ulaz nepodržanog ponašanja:

```
{
  "price": 0
}
```

Izlaz: - status: 400 BAD REQUEST - "Price must be a positive number."

Izvorni kod ispitnog slučaja

```
@Test
@DisplayName("Redovni: Uspješno dodavanje sobe s generiranjem pod-soba")
public void testAddUnit_Success() throws Exception {
    Unit unit = new Unit();
    unit.setUnitName("Hotel Central");
    unit.setPrice(100);
    unit.setCapAdults(2);
    unit.setApartment(false);
    unit.setNumSameRooms(2);

    Mockito.when(unitRepo.findByUnitName("Hotel Central")).thenReturn(Optional.empty());
    Mockito.when(unitRepo.save(Mockito.any(Unit.class))).thenAnswer(i -> i.getArguments()[0]);

    mockMvc.perform(post("/unit/add")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(unit)))
        .andDo(print())
        .andExpect(status().isOk())
```

```

        .andExpect(jsonPath("$.unitName").value("Hotel Central"));
    }

    @Test
    @DisplayName("Rubni: Naziv jedinice već postoji")
    public void testAddUnit_DuplicateName() throws Exception {
        Unit unit = new Unit();
        unit.setUnitName("Postojeci");

        Mockito.when(unitRepo.findByUnitName("Postojeci")).thenReturn(Optional
1.of(new Unit()));

        mockMvc.perform(post("/unit/add")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(objectMapper.writeValueAsString(unit)))
                .andDo(print())
                .andExpect(status().isConflict())
                .andExpect(content().string("Unit already exists"));
    }

    @Test
    @DisplayName("Izazivanje pogreške: Neispravna cijena")
    public void testAddUnit_InvalidPrice() throws Exception {
        Unit unit = new Unit();
        unit.setPrice(0); // Prema kodu, cijena mora biti > 0 (price < 1 vrać
a badRequest)

        mockMvc.perform(post("/unit/add")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(objectMapper.writeValueAsString(unit)))
                .andDo(print())
                .andExpect(status().isBadRequest())
                .andExpect(content().string("Price must be a positive number.
"));
    }
}

```

Ispit komponente 4 – ažuriranje smještajne jedinice

Endpoint: - PUT /unit/update/{id}

Preduvjet: - Korisnik mora biti admin.

Ulaz očekivanog ponašanja:

- Postojeći ID jedinice
- Valjani podaci za ažuriranje

Izlaz: - status: 200 OK - Ažurirani objekt Unit

Ulaz rubnog slučaja:

- Smanjenje broja soba blokirano rezervacijom

Izlaz: - status: 400 BAD REQUEST - "Some rooms have active reservations. You cannot reduce the number of rooms below 5."

Ulaz nepodržanog ponašanja:

- ID jedinice ne postoji u bazi

Izlaz: - status: 500 INTERNAL SERVER ERROR - "Greška: Unit can't be found."

Izvorni kod ispitnog slučaja

```
@Test
@DisplayName("Redovni: Uspješno ažuriranje cijene")
public void testUpdateUnit_Success() throws Exception {
    Unit existing = new Unit();
    existing.setIdUnit(1L);
    existing.setPrice(50);

    Unit payload = new Unit();
    payload.setPrice(80);

    Mockito.when(unitRepo.findById(1L)).thenReturn(Optional.of(existing));
    Mockito.when(unitRepo.save(Mockito.any(Unit.class))).thenAnswer(i ->
i.getArguments()[0]);

    mockMvc.perform(put("/unit/update/1")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(payload)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.price").value(80.0));
}

@Test
@DisplayName("Rubni: Smanjenje broja soba blokirano rezervacijom")
public void testUpdateUnit_RoomsWithReservations() throws Exception {
    // 1. Pripremamo postojeću jedinicu koja ima 5 soba u bazi
    Unit existingUnit = new Unit();
    existingUnit.setIdUnit(1L);
    existingUnit.setNumSameRooms(5);
    existingUnit.setApartment(false);

    // Dodajemo barem jednu sobu u listu kako bi currentRooms.size() bio
    // veći od 2
    List<Unit> rooms = new java.util.ArrayList<>();
    for(int i=0; i<5; i++) {
        Unit r = new Unit();
```

```

        r.setUnitName("Soba " + i);
        rooms.add(r);
    }
    existingUnit.setListOfRooms(rooms);

    // 2. Payload koji šaljemo (želimo smanjiti na 2 sobe)
    Unit payload = new Unit();
    payload.setNumSameRooms(2);
    payload.setApartment(false);

    // 3. Mockiranje
    Mockito.when(unitRepo.findById(1L)).thenReturn(Optional.of(existingUn
it));

    // Simuliramo da postoji aktivna rezervacija za sobu koju pokušavamo
obrisati
    Mockito.when(reservationRepo.existsByUnitAndStatusInAndEndDateAfter(
        Mockito.any(Unit.class), Mockito.anyList(), Mockito.a
ny(java.time.LocalDate.class)))
        .thenReturn(true);

    mockMvc.perform(put("/unit/update/1")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(payload)))
        .andDo(print())
        .andExpect(status().isBadRequest())
        .andExpect(content().string(org.hamcrest.Matchers.containsStr
ing("active reservations"))));
    }

    @Test
    @DisplayName("Izazivanje pogreške: Nepostojeći ID jedinice")
    public void testUpdateUnit_NotFound() throws Exception {
        Mockito.when(unitRepo.findById(99L)).thenReturn(Optional.empty());

        mockMvc.perform(put("/unit/update/99")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(new Unit())))
            .andDo(print())
            .andExpect(status().isInternalServerError())
            .andExpect(content().string(org.hamcrest.Matchers.containsStr
ing("Unit can't be found.")));
    }

```

Ispit komponente 5 – kreiranje rezervacije

Endpoint: - POST /unitReservation/add

Preduvjet: - Korisnik mora biti ulogiran i imati ulogu user.

Ulaz očekivanog ponašanja:

```
{
  "unitId": 10,
  "startDate": "2026-06-10",
  "endDate": "2026-06-15",
  "adults": 2
}
```

Izlaz: - status: 200 OK - "Reservation successfully created. A confirmation inquiry has been sent to: gost@example.com"

Ulaz rubnog slučaja:

- JWT token korisnika koji nema ulogu user

Izlaz: - status: 403 FORBIDDEN - "Only regular users can make reservations."

Ulaz nepodržanog ponašanja:

```
{
  "unitId": 5,
  "startDate": "2020-01-01",
  "endDate": "2020-01-05"
}
```

Izlaz: - status: 400 BAD REQUEST - "Departure date must be at least one day after arrival."

Izvorni kod ispitnog slučaja

```
@Test
@DisplayName("Redovni: Uspješno kreiranje rezervacije za običnog korisnik
a")
public void testAddReservation_Success() throws Exception {
    String email = "gost@example.com";
    ReservationRequest req = new ReservationRequest();
    req.setUnitId(10L);
    req.setStartDate(LocalDate.now().plusDays(5));
    req.setEndDate(LocalDate.now().plusDays(10));
    req.setAdults(2);

    // Postavljamo Person prema tvom modelu
    Person gost = new Person();
    gost.setEmail(email);
    gost.setUser(true); // Ključno za prolaz if(person.isUser())
    gost.setOwner(false);
    gost.setName("Gost Korisnik");

    Unit smjestaj = new Unit();
    smjestaj.setIdUnit(10L);

    Mockito.when(personRepo.findByEmail(email)).thenReturn(Optional.of(go
```

```

st));
    Mockito.when(reservationRepo.existsOverlapping(10L, req.getStartDate(
), req.getEndDate())).thenReturn(false);
    Mockito.when(unitRepo.findById(10L)).thenReturn(Optional.of(smjesta
j));

    mockMvc.perform(post("/unitReservation/add")
        .with(jwt().jwt(j -> j.claim("email", email)))
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(req)))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(content().string(org.hamcrest.Matchers.containsStri
ing("Reservation successfully created"))));
    }

    @Test
    @DisplayName("Rubni: Korisnik s ulogom Owner pokušava rezervirati")
    public void testAddReservation_ForbiddenForOwner() throws Exception {
        String email = "vlasnik@example.com";
        ReservationRequest req = new ReservationRequest();
        req.setStartDate(LocalDate.now().plusDays(1));
        req.setEndDate(LocalDate.now().plusDays(2));

        Person vlasnik = new Person();
        vlasnik.setEmail(email);
        vlasnik.setUser(false); // Owner nije "regular user" u tvom kontroler
u
        vlasnik.setOwner(true);

        Mockito.when(personRepo.findByEmail(email)).thenReturn(Optional.of(vl
asnik));

        mockMvc.perform(post("/unitReservation/add")
            .with(jwt().jwt(j -> j.claim("email", email)))
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andDo(print())
            .andExpect(status().isForbidden())
            .andExpect(content().string("Only regular users can make rese
rvations.")));
    }

    @Test
    @DisplayName("Izazivanje pogreške: Krajnji datum prije početnog")
    public void testAddReservation_InvalidDates() throws Exception {
        String email = "gost@example.com";
        ReservationRequest req = new ReservationRequest();
        req.setStartDate(LocalDate.now().plusDays(5));

```

```

        req.setEndDate(LocalDate.now().plusDays(4)); // GREŠKA: Kraj je prije
        početka

        Person gost = new Person();
        gost.setEmail(email);
        gost.setUser(true);

        Mockito.when(personRepo.findByEmail(email)).thenReturn(Optional.of(gost));

        mockMvc.perform(post("/unitReservation/add")
                .with(jwt().jwt(j -> j.claim("email", email)))
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(req)))
                .andDo(print())
                .andExpect(status().isBadRequest())
                .andExpect(content().string("Departure date must be at least
one day after arrival.")));
    }

```

Ispit komponente 6 – dohvat vlastitih rezervacija

Endpoint: - GET /unitReservation/my-reservations

Preduvjet: - Korisnik mora biti ulogiran.

Ulaz očekivanog ponašanja:

- JWT token korisnika koji ima rezervacije

Izlaz: - status: 200 OK - Lista rezervacija korisnika

Ulaz rubnog slučaja:

- JWT token korisnika bez rezervacija

Izlaz: - status: 200 OK - Prazna lista []

Ulaz neočekivanog ponašanja:

- JWT token nije poslan

Izlaz: - status: 401 UNAUTHORIZED

Izvorni kod ispitnog slučaja

```

@Test
@DisplayName("Redovni: Uspješan dohvat liste rezervacija")
public void testGetMyReservations_Success() throws Exception {
    String email = "test@example.com";
    UnitReservation res = new UnitReservation();
    res.setStatus("Confirmed");
}

```

```

Mockito.when(reservationRepo.findByPersonEmail(email))
    .thenReturn(Collections.singletonList(res));

mockMvc.perform(get("/unitReservation/my-reservations")
    .with(jwt().jwt(j -> j.claim("email", email))))
    .andDo(print())
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.length()").value(1))
    .andExpect(jsonPath("$[0].status").value("Confirmed"));
}

@Test
@DisplayName("Rubni: Dohvat kada korisnik nema rezervacija")
public void testGetMyReservations_Empty() throws Exception {
    String email = "novi@example.com";
    Mockito.when(reservationRepo.findByPersonEmail(email)).thenReturn(Collections.emptyList());

    mockMvc.perform(get("/unitReservation/my-reservations")
        .with(jwt().jwt(j -> j.claim("email", email))))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.length()").value(0));
}

@Test
@DisplayName("Izazivanje pogreške: Pristup bez prijave")
public void testGetMyReservations_NoJwt() throws Exception {
    // Ne dodajemo .with(jwt())
    mockMvc.perform(get("/unitReservation/my-reservations"))
        .andDo(print())
        .andExpect(status().isUnauthorized());
}

```

Ispitivanje sustava

Pokretanje kodova: - Potrebno je imati instalirao: Java JDK, IntelliJ i Chrome browser. - U IntelliJ-u stvoriti novi projekt koristeći maven. - Nazvati projekt npr. system-tests. - Za archetype staviti org.apache.maven.archetypes:maven-archetype-quickstart. - Te kliknuti na create. - U pom.xml potrebno je dodati:

```
<dependencies>
  <!-- JUnit 5 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>

  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.16.1</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- Maven plugin za pokretanje JUnit testova -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <includes>
          <include>**/*Test.java</include>
        </includes>
      </configuration>
    </plugin>
  </plugins>
</build>
```

- Te kliknuti da maven to importa.
- U src/test/java/org/example/ napraviti klasu za test i kopirati kod.
- Sada bi se testovi trebali moći pokrenuti.
- Ako ima problema, pogledajte pomoću view -> tool windows -> maven jesu li se dependencies importali.

- Nakon pokretanja prvoga testa, potrebno je zaustaviti test i ulogirati se sa Google računom te ponovno pokrenuti test.
- Za testiranje s ownerom, potrebno je preko admina staviti useru role ownera.

1. Redovni slučaj - Rezervacija smještaja(Gost)

Preduvjeti: - Korisnik mora biti ulogiran kao gost. - Ne smije biti otvoren niti jedan drugi Google Chrome.

Ulazi

1. Datum (16. do 17.).
2. Ponovni odabir datuma(potvrda datuma).
3. Prve dvije dodatne usluge(npr. WiFi i Shampoo).

Koraci ispitivanja

1. Otvoriti aplikaciju u pregledniku.
2. Kliknuti na gumb "Find a Place to Stay".
3. Kliknuti na datum kako bi se otvorio kalendar.
4. Kliknuti na startDate i endDate(u našem slučaju 16. do 17.).
5. Kliknuti na gumb "Search".
6. Kliknuti na prvi gumb "See availability".
7. Kliknuti na gumb "Reserve or Book Now!".
8. Kliknuti na startDate i endDate(isto kao i prije).
9. Kliknuti na prve dvije dodatne usluge.
10. Kliknuti na gumb "Confirm Reservation".
11. Kliknuti na gumb "Close".
12. Kliknuti na gumb "Booked Reservations".
13. Verificirati da je rezervacija pending.

Očekivani izlaz

Pod "Booked Reservations" vidljiva je napravljena rezervacija s oznakom pending.

Dobiveni izlaz

Video Selenium testa rezervacije smještaja za gosta

<https://github.com/user-attachments/assets/33505375-1574-448c-9b24-42aa9323ba4c>

Izvorni kod

```
package org.example;
```

```
import org.junit.jupiter.api.*;
```

```

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import java.time.Duration;
import java.util.Collections;

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class ReservationTest {

    WebDriver driver;

    @BeforeEach
    void setup() {
        ChromeOptions options = new ChromeOptions();

        // novi profil dostupan samo Seleniumu
        // zamijeni usera stvarnim userom računala
        options.addArguments("--user-data-dir=C:\\Users\\User\\Desktop\\SeleniumProfile");

        // ovo maskira Selenium da ga Google ne prepozna kao bota
        options.addArguments("--disable-blink-features=AutomationControlled");
        options.setExperimentalOption("excludeSwitches", Collections.singletonList("enable-automation"));
        options.setExperimentalOption("useAutomationExtension", false);

        driver = new ChromeDriver(options);
        // Selenium će automatski čekati do 10 sekundi za svaki element u kodu
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
    }

    @AfterEach
    void teardown() {
        //driver.quit();
    }

    @Test
    @DisplayName("Redovni slučaj: Uspješna rezervacija")
    void testSuccessfulReservation() {
        driver.get("https://room-rently-v2.netlify.app");
        driver.findElement(By.tagName("button")).click();
        driver.findElement(By.cssSelector(".headerSearchItem:nth-child(2) > .headerSearchText")).click();

        // 16. dolazak
        driver.findElement(By.cssSelector(".rdrDay:nth-child(20) > .rdrDayNumber > span")).click();
    }
}

```

```

        // 17. odlazak
        driver.findElement(By.cssSelector(".rdrDay:nth-child(21) > .rdrDayNum
ber > span")).click();

        driver.findElement(By.cssSelector(".headerBTN")).click();

        WebElement checkButton = driver.findElement(By.className("sicheckbutt
on"));
        ((JavascriptExecutor) driver).executeScript("arguments[0].click();",
checkButton);

        driver.findElement(By.cssSelector(".booknow")).click();

        // isto kao i prije
        driver.findElement(By.cssSelector(".rdrDay:nth-child(20) > .rdrDayNum
ber > span")).click();
        driver.findElement(By.cssSelector(".rdrDay:nth-child(21) > .rdrDayNum
ber > span")).click();

        driver.findElement(By.cssSelector(".optionCard:nth-child(1)")).click
();
        driver.findElement(By.cssSelector(".optionCard:nth-child(2)")).click
();

        driver.findElement(By.cssSelector(".confirmBtn")).click();
        driver.findElement(By.cssSelector(".closeBtn")).click();
        driver.findElement(By.cssSelector(".navButton:nth-child(3)")).click();

        // provjera pending statusa
        String statusNaEkranu = driver.findElement(By.className("reservationS
tatus")).getText();
        Assertions.assertEquals("PENDING", statusNaEkranu);
    }
}

```

2. Rubni uvjet - Rezervacija smještaja zauzetog datuma(Gost)

Preduvjeti: - Korisnik mora biti ulogiran kao gost. - Ne smije biti otvoren niti jedan drugi Google Chrome. - Sve sobe tog tipa su zauzete za taj datum.

Ulazi

1. Datum (17. do 18.).
2. Ponovni odabir datuma(sada pokušavamo odabrati 16. koji je zauzet).

Koraci ispitivanja

1. Otvoriti aplikaciju u pregledniku.
2. Kliknuti na gumb “Find a Place to Stay”.

3. Kliknuti na datum kako bi se otvorio kalendar.
4. Kliknuti na startDate i endDate(17 do 18.).
5. Kliknuti na gumb "Search".
6. Kliknuti na prvi gumb "See availability".
7. Kliknuti na gumb "Reserve or Book Now!".
8. Kliknuti na startDate i endDate(sada pokušavamo odabrati 16. koji je zauzet).
9. Verificirati da je datum zasivljen/onemogućen, odnosno nije ga moguće odabrati.

Očekivani izlaz

Datum je zasivljen i ne može se odabrati.

Dobiveni izlaz

Video Selenium testa rezervacije smještaja zauzetog datuma za gosta

<https://github.com/user-attachments/assets/68f493d5-fcba-445d-a0cc-d9df630c8c57>

Izvorni kod

```
package org.example;

import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import java.time.Duration;
import java.util.Collections;

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class AvailabilityTest {

    WebDriver driver;

    @BeforeEach
    void setup() {
        ChromeOptions options = new ChromeOptions();

        // novi profil dostupan samo Seleniumu
        // zamijeni usera stvarnim userom računala
        options.addArguments("--user-data-dir=C:\\Users\\User\\Desktop\\SeleniumProfile");

        // ovo maskira Selenium da ga Google ne prepozna kao bota
        options.addArguments("--disable-blink-features=AutomationControlled");
        options.setExperimentalOption("excludeSwitches", Collections.singletonList("enable-automation"));
        options.setExperimentalOption("useAutomationExtension", false);
    }
}
```

```

        driver = new ChromeDriver(options);
        // Selenium će automatski čekati do 10 sekundi za svaki element u kod
u
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
    }

    @AfterEach
    void teardown() {
        //driver.quit();
    }

    @Test
    @DisplayName("Rubni uvjet: Rezervacija smještaja zauzetog datuma")
    void testSuccessfulReservation() {
        driver.get("https://room-rently-v2.netlify.app");
        driver.findElement(By.tagName("button")).click();
        driver.findElement(By.cssSelector(".headerSearchItem:nth-child(2) > .
headerSearchText")).click();

        // 17. dolazak
        driver.findElement(By.cssSelector(".rdrDay:nth-child(21) > .rdrDayNum
ber > span")).click();
        // 18. odlazak
        driver.findElement(By.cssSelector(".rdrDay:nth-child(22) > .rdrDayNum
ber > span")).click();

        driver.findElement(By.cssSelector(".headerBTN")).click();

        WebElement checkButton = driver.findElement(By.className("scheckbutt
on"));
        ((JavascriptExecutor) driver).executeScript("arguments[0].click();",
checkButton);

        driver.findElement(By.cssSelector(".booknow")).click();

        // provjera je li datum onemogućen
        WebElement zauzetiDan = driver.findElement(By.cssSelector(".rdrDay:nt
h-child(20)"));
        String klase = zauzetiDan.getAttribute("class");
        Assertions.assertTrue(klase.contains("rdrDayDisabled"), "Dan bi treba
o biti zasivljen/onemogućen!");
    }
}

```

3. Rubni uvjet - Izvoz statistike za period bez rezervacija(Owner)

Preduvjeti: - Korisnik mora biti ulogiran kao owner. - Ne smije biti otvoren niti jedan drugi Google Chrome. - Za odabrani mjesec testiranja ne postoji niti jedna rezervacija.

Ulazi

Odabir mjeseca za kojeg ne postoji niti jedna rezervacija.

Koraci ispitivanja

1. Otvoriti aplikaciju u pregledniku.
2. Kliknuti na gumb "Find a Place to Stay".
3. Kliknuti na gumb "Dashboard".
4. Kliknuti na All Months kako bi se otvorio padajući izbornik za biranje mjeseca.
5. Kliknuti na mjesec koji nema niti jednu rezervaciju(u našem testu je to April).
6. Kliknuti na gumb "Export PDF Report".
7. Kliknuti na gumb "Export Excel".
8. Kliknuti na gumb "Export XML".
9. Kliknuti na gumb "Reservations".
10. Kliknuti na gumb "Export PDF".
11. Kliknuti na gumb "Export Excel".
12. Kliknuti na gumb "Export XML".
13. Kliknuti na gumb "Return".
14. Verificirati da je sustav generirao pdf, xlsx i xml statistike te nastavio s radom(nalazimo se na /main ruti).

Očekivani izlaz

Generirani pdf, xlsx i xml statistike za "Hotel Full Analytics Report" i "Reservations List" bez crashanja stranice.

Dobiveni izlaz

Video Selenium testa izvoza statistike za period bez rezervacija za ownera

<https://github.com/user-attachments/assets/e2a0c33d-08fe-47ec-86c9-3d222fac6415>

Izvorni kod

```
package org.example;

import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import java.time.Duration;
import java.util.Collections;

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class ExportTest {

    WebDriver driver;
```

```

@BeforeEach
void setup() {
    ChromeOptions options = new ChromeOptions();

    // novi profil dostupan samo Seleniumu
    // zamijeni usera stvarnim userom računala
    options.addArguments("--user-data-dir=C:\\Users\\User\\Desktop\\SeleniumProfile");

    // ovo maskira Selenium da ga Google ne prepozna kao bota
    options.addArguments("--disable-blink-features=AutomationControlled");
    options.setExperimentalOption("excludeSwitches", Collections.singletonList("enable-automation"));
    options.setExperimentalOption("useAutomationExtension", false);

    driver = new ChromeDriver(options);
    // Selenium će automatski čekati do 10 sekundi za svaki element u kodu
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    driver.manage().window().maximize();
}

@AfterEach
void teardown() {
    //driver.quit();
}

@Test
@DisplayName("Rubni uvjet: Izvoz statistike za period bez rezervacija")
void testSuccessfulReservation() {
    driver.get("https://room-rently-v2.netlify.app");
    driver.findElement(By.tagName("button")).click();
    driver.findElement(By.cssSelector(".navButton:nth-child(3)")).click();
    // Dashboard
    driver.findElement(By.cssSelector(".month-dropdown")).click();
    driver.findElement(By.cssSelector("option:nth-child(5)")).click(); // April
    driver.findElement(By.cssSelector(".export-pdf-btn")).click();
    driver.findElement(By.cssSelector(".export-xlsx-btn")).click();
    driver.findElement(By.cssSelector(".export-xml-btn")).click();
    driver.findElement(By.cssSelector(".tab-btn:nth-child(2)")).click();
    // Reservations
    driver.findElement(By.cssSelector(".export-pdf-btn")).click();
    driver.findElement(By.cssSelector(".export-xlsx-btn")).click();
    driver.findElement(By.cssSelector(".export-xml-btn")).click();
    driver.findElement(By.cssSelector(".return-btn")).click();

    // provjera URL-a

```

```

        String trenutniUrl = driver.getCurrentUrl();
        Assertions.assertEquals("https://room-rently-v2.netlify.app/main", trenutniUrl, "URL se ne podudara!");
    }
}

```

4. Poziv nepostojeće funkcionalnosti - Pokušaj korištenja /admin rute(Gost)

Preduvjeti: - Korisnik mora biti ulogiran kao gost. - Ne smije biti otvoren niti jedan drugi Google Chrome.

Ulazi

Unos URL-a admin rute(<https://room-rently-v2.netlify.app/admin>).

Koraci ispitivanja

1. Otvoriti aplikaciju u pregledniku.
2. Kliknuti na gumb “Find a Place to Stay”.
3. Promijeniti URL na rutu /admin.
4. Verificirati da se nalazimo na ruti /main.

Očekivani izlaz

Redirect natrag na /main rutu.

Dobiveni izlaz

Video Selenium testa pokušaja korištenja /admin rute za gosta

<https://github.com/user-attachments/assets/4b99c00c-2cff-42ff-8fbd-2b2604e00ad4>

Izvorni kod

```

package org.example;

import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import java.time.Duration;
import java.util.Collections;

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
public class AccessControlTest {

```

```

    WebDriver driver;

```

```

    @BeforeEach

```

```

void setup() {
    ChromeOptions options = new ChromeOptions();

    // novi profil dostupan samo Seleniumu
    // zamijeni usera stvarnim userom računala
    options.addArguments("--user-data-dir=C:\\Users\\User\\Desktop\\Selen
iumProfile");

    // ovo maskira Selenium da ga Google ne prepozna kao bota
    options.addArguments("--disable-blink-features=AutomationControlled");
    options.setExperimentalOption("excludeSwitches", Collections.singleto
nList("enable-automation"));
    options.setExperimentalOption("useAutomationExtension", false);

    driver = new ChromeDriver(options);
    // Selenium će automatski čekati do 10 sekundi za svaki element u kod
u
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    driver.manage().window().maximize();
}

@AfterEach
void teardown() {
    //driver.quit();
}

@Test
@DisplayName("Poziv nepostojeće funkcionalnosti: Pokušaj korištenja /admi
n rute(Gost)")
void testSuccessfulReservation() {
    driver.get("https://room-rently-v2.netlify.app");
    driver.findElement(By.tagName("button")).click();
    driver.get("https://room-rently-v2.netlify.app/admin");

    // provjera URL-a
    String trenutniUrl = driver.getCurrentUrl();
    Assertions.assertEquals("https://room-rently-v2.netlify.app/main", tr
enutniUrl, "URL se ne podudara!");
}
}

```

7. Tehnologije za implementaciju aplikacije

Korištene tehnologije i alati

1. Programski jezici

- **Java 21** - [Java](#) je korištena za razvoj serverskog dijela aplikacije zbog svoje čitljivosti, sigurnosti, enkapsuliranosti i široke primjene u razvoju poslovnih aplikacija. Odabir Java omogućio je korištenje Spring Boot radnog okvira te jednostavnu integraciju s bazom podataka, sigurnosnim mehanizmima i vanjskim servisima. Verzija 21 odabrana je zbog svoje dugoročne podrške.
- **JavaScript ES6+** - [JavaScript](#) je korišten za razvoj klijentskog dijela aplikacije jer je standardni jezik za izradu web aplikacija. Omogućuje dinamičko upravljanje korisničkim sučeljem i izravnu komunikaciju s backend REST API-jem.

2. Radni okviri i biblioteke

- **React 19** - [React](#) je korišten za razvoj frontenda zbog svoje komponentne arhitekture, fleksibilnosti i jednostavne ponovne iskoristivosti koda. Omogućuje izradu dinamičnih i responzivnih korisničkih sučelja.
- **Spring Boot 3.5.7** - [Spring Boot](#) je korišten kao glavni radni okvir za razvoj backend aplikacije. Omogućuje brzo postavljanje REST servisa, automatsku konfiguraciju aplikacije i jednostavnu integraciju s drugim Spring modulima. Odabran je zbog svoje stabilnosti i česte primjene u industriji.
- **Spring Data JPA (Hibernate)** - [Spring Data JPA](#) korišten je za rad s bazom podataka kroz objektno-relacijsko mapiranje. Omogućuje jednostavno mapiranje Java entiteta na relacijske tablice i smanjuje potrebu za pisanjem SQL upita.
- **Spring Security i OAuth2 Resource Server** - [Spring Security](#) korišten je za implementaciju autentifikacije i autorizacije korisnika. [OAuth2 Resource Server](#) korišten je za validaciju JWT tokena dobivenih putem Google OAuth2 autentifikacije, čime je osigurana sigurnost aplikacije.

3. Baza podataka

- **PostgreSQL 17.6.1.038** - [PostgreSQL](#) je relacijska baza podataka odabrana zbog svoje stabilnosti, pouzdanosti i skalabilnosti. Svi članovi tima već imaju iskustva s ovom bazom, što ubrzava razvoj. Integracija s Hibernate ORM alatom omogućila je jednostavno upravljanje podacima kroz JPA entitete.

4. Razvojni alati

- **IntelliJ IDEA** - [IntelliJ IDEA](#) je korišten kao primarni razvojni alat za backend zbog odlične podrške za Java i Spring Boot projekte.
- **Visual Studio Code** - [Visual Studio Code](#) je korišten za razvoj frontend dijela aplikacije zbog brzine, fleksibilnosti i odlične podrške za JavaScript i React.
- **Git 2x** - [Git](#) je korišten za verzioniranje koda i suradnju na projektu.
- **Apache Maven 3.8+** - [Apache Maven](#) je korišten za upravljanje ovisnostima i izgradnju backend aplikacije temeljene na Spring Boot radnom okviru.

5. Alati za ispitivanje

- **Selenium 4.16.1** - [Selenium](#) je korišten za automatsko testiranje web sučelja.
- **JUnit 5.10** - [JUnit](#) je korišten za automatizirano testiranje Java koda. Omogućava pisanje jediničnih testova koji provjeravaju ispravnost pojedinačnih metoda ili funkcionalnosti, olakšavajući rano otkrivanje grešaka i održavanje koda.

6. Alati za razmještaj

- **Docker 28.5.1** - [Docker](#) - je korišten za pakiranje, distribuiranje i pokretanje aplikacija u izoliranim kontejnerima. Omogućava da aplikacija zajedno sa svim svojim ovisnostima radi konzistentno na različitim računalnim okruženjima, što olakšava razvoj, testiranje i razmještaj.
- **Npm 11.6.2** - [Npm 11.6.2](#) - je korišten za upravljanje paketima i ovisnostima u JavaScript projektima.

7. Cloud platforma

- **Amazon Web Services** - [AWS](#) je korišten za hostanje backend aplikacije zbog svoje pouzdanosti, skalabilnosti i širokog spektra usluga. Platforma omogućuje jednostavno upravljanje resursima, sigurnosnim postavkama i mrežnom konfiguracijom.
- **Supabase** - [Supabase](#) je korišten za hostanje PostgreSQL baze podataka. Odabran je zbog jednostavne konfiguracije, pouzdanog cloud hostinga i kompatibilnosti s PostgreSQL bazom. Njegova integracija s backend aplikacijom omogućila je sigurno i stabilno udaljeno pohranjivanje podataka, uz minimalnu potrebu za dodatnim administriranjem baze.

8. Upute za puštanje u pogon

Ovaj odjeljak dokumentacije daje detaljne smjernice za instalaciju, konfiguraciju, pokretanje i administraciju aplikacije Room-Rently. Cilj je olakšati postavljanje aplikacije na razvojnom (lokalnom) i produkcijskom (Cloud) okruženju.

1. Instalacija

Ovdje su navedeni koraci potrebni za instalaciju svih potrebnih komponenti kako bi se projekt pokrenuo lokalno.

Preduvjeti

- Java Development Kit (JDK): Verzija 21 (preporučeno Eclipse Temurin ili OpenJDK).
- Node.js: Verzija 18 ili novija (za frontend).
- Docker Desktop: Najnovija verzija (za izgradnju kontejnera).
- Git: Za verzioniranje koda.
- Maven: (Opcionalno, projekt sadrži mvnw wrapper). ### Preuzimanje Klonirajte repozitorij s izvornim kodom:

```
git clone https://github.com/TvojUsername/Room-Rently.git
cd Room-Rently
```

Instalacija ovisnosti

- Backend (Spring Boot): Pozicionirajte se u mapu backenda i preuzmite Maven ovisnosti.

```
cd backend
./mvnw dependency:go-offline
```

- Frontend (React): Pozicionirajte se u mapu klijenta i instalirajte NPM pakete.

```
cd ../client
npm install
```

2. Postavke

Detaljne upute za konfiguraciju aplikacije prije pokretanja. Aplikacija se oslanja na varijable okruženja (Environment Variables) radi sigurnosti.

Konfiguracijske datoteke

Konfiguracija Backenda: Backend očekuje varijable okruženja koje se lokalno mogu postaviti u IDE-u ili putem terminala, a u produkciji se unose u Cloud konzolu.

Sadržaj .env datoteke:

```
# Baza podataka (Supabase)
DATASOURCE_URL=jdbc:postgresql://[SUPABASE_HOST]:5432/postgres
DATASOURCE_USER=postgres.[tvoj-projekt]
DATASOURCE_PASSWORD=[tvoja-lozinka]

# Frontend URL (za CORS)
FRONTEND_URL=http://localhost:3000

# Email postavke
MAIL_USERNAME=room.rently.default@gmail.com
MAIL_PASS=sdtcpuxmfxmedwbh

# Supabase Storage (S3 kompatibilnost)
SUPABASE_S3_ACCESS_KEY=[tvoj-access-key]
SUPABASE_S3_SECRET_KEY=[tvoj-secret-key]
SUPABASE_S3_ENDPOINT=https://[tvoj-id].supabase.co/storage/v1/s3
SUPABASE_S3_BUCKET=units
SUPABASE_S3_REGION=eu-central-1
```

Ova datoteka služi kao lokalno spremište varijabli okruženja. Ona sadrži konkretne, često povjerljive vrijednosti koje se ne bi trebale dijeliti. - Baza podataka: Definira parametre za spajanje na Supabase PostgreSQL bazu (URL, USER, PASSWORD). - FRONTEND_URL: Određuje adresu s koje dolazi frontend (npr. React na portu 3000), što je ključno za rješavanje CORS sigurnosnih pravila. - Email postavke: Sadrži vjerodajnice za Gmail SMTP poslužitelj kako bi aplikacija mogla slati automatske poruke. - Supabase Storage (S3): Ovdje su pohranjeni pristupni ključevi i putanje za rad sa slikama. Sustav koristi S3 protokol, što je standard za pohranu objekata u Cloudu.

Sadržaj application.properties datoteke:

```
spring.application.name=backend

spring.datasource.url=${DATASOURCE_URL}
spring.datasource.username=${DATASOURCE_USER}
```

```

spring.datasource.password=${DATASOURCE_PASSWORD}

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update

frontend.url=${FRONTEND_URL}

spring.security.oauth2.resourceserver.jwt.issuer-uri=https://accounts.google.com

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

# Supabase S3
supabase.s3.access-key=${SUPABASE_S3_ACCESS_KEY}
supabase.s3.secret-key=${SUPABASE_S3_SECRET_KEY}
supabase.s3.endpoint=${SUPABASE_S3_ENDPOINT}
supabase.s3.bucket=${SUPABASE_S3_BUCKET:units}
supabase.s3.region=${SUPABASE_S3_REGION:eu-central-1}

```

Ovo je glavna konfiguracijska datoteka Spring Boota. Ona ne sadrži izravne lozinke, već koristi sintaksu `${IME_VARIJABLE}` kako bi povukla vrijednosti definirane u `.env` datoteci ili na samom poslužitelju. Povezivanje baze podataka: Putem `spring.datasource` parametara, Spring preuzima podatke iz `.env` datoteke i uspostavlja vezu s bazom. - Hibernate postavke: `* dialect`: Govori Hibernateu da koristi specifična pravila za PostgreSQL. - `ddl-auto=update`: Automatski ažurira strukturu tablica u bazi prema promjenama u Java entitetima. - OAuth2 / Google Login: `issuer-uri` postavlja Google kao autoritet za provjeru autentičnosti putem JWT tokena. - Multipart (Slike): Ograničava veličinu datoteka koje se šalju na poslužitelj (ovdje na 10MB), što sprječava preopterećenje memorije. - Mapiranje S3 varijabli: Spring preuzima S3 ključeve i proslijeđuje ih internim servisima koji upravljaju uploadom slika. Primijeti sintaksu `${VAR:default}` — ako varijabla nije definirana, koristit će se zadana vrijednost (npr. `units` ili `eu-central-1`).

Konfiguracija Frontenda: U mapi client kreirajte datoteku .env.

Sadržaj `.env` datoteke:

```

# URL na koji frontend šalje zahtjeve (Lokalno: localhost:8080, Produkcija: AWS URL)
REACT_APP_API_URL=http://localhost:8080

```

Priprema Baze i Bucketa (Supabase)

Baza podataka je hostana na Supabase platformi.

Baza podataka

Za uspješno povezivanje aplikacije s bazom podataka nužan je niz za povezivanje.

Postupak dohvaćanja podataka: - Pristupiti Supabase Dashboardu. - Navigirati na Project Settings -> Database. - Pronaći odjeljak Connection String i odabrati karticu URI. - Kopirati prikazanu vrijednost. Format niza znakova izgleda ovako: jdbc:postgresql://aws-0-eu-central-1.pooler.supabase.com:5432/postgres?user=postgres.[naziv-projekta]&password=[lozinka]

Ovaj podatak koristit će se u kasnijim koracima za konfiguraciju varijable okruženja DATASOURCE_URL.

Storage (Bucket) za slike

Aplikacija koristi varijable okruženja s prefiksom SUPABASE_S3... za komunikaciju sa sustavom za pohranu, koristeći Supabaseovu S3 kompatibilnost.

Postupak konfiguracije pohrane: - Unutar Supabase sučelja odabrati opciju Storage. - Kreirati novi bucket pod nazivom units (kako bi odgovarao definiciji varijable SUPABASE_S3_BUCKET). - Postaviti vidljivost bucketa na Public (kako bi frontend aplikacija mogla dohvaćati i prikazivati slike). - Za dohvaćanje pristupnih ključeva, navigirati na Project Settings -> Storage.

Pridruživanje vrijednosti varijablama: Potrebno je preslikati vrijednosti iz postavki u konfiguraciju aplikacije na sljedeći način: - Access Key ID → unosi se u varijablu SUPABASE_S3_ACCESS_KEY. - Secret Access Key → unosi se u varijablu SUPABASE_S3_SECRET_KEY. - Endpoint (npr. https://[project-id].supabase.co/storage/v1/s3) → unosi se u varijablu SUPABASE_S3_ENDPOINT. - Region (npr. eu-central-1) → unosi se u varijablu SUPABASE_S3_REGION.

3. Pokretanje aplikacije

Upute za pokretanje aplikacije u različitim okruženjima

Razvojno okruženje

- Pokretanje Backenda:

```
cd backend
./mvnw spring-boot:run
```

- Backend će biti dostupan na: http://localhost:8080
- Pokretanje Frontenda: U novom terminalu:

```
cd client
npm start
```

- Frontend će biti dostupan na: http://localhost:3000

Produksijsko okruženje (Priprema):

- Prevođenje Backenda (Docker): Aplikacija se pakira u Docker sliku.

```
docker build -t tvoje_dockerhub_ime/room-rently-backend .
```

- Prevođenje Frontenda: Kreiranje optimiziranih statičkih datoteka.

```
cd client
npm run build
```

Rezultat je mapa build spremna za upload.

4. Upute za administratore

Smjernice za administratore aplikacije nakon puštanja u pogon

Pristup podacima:

- Baza podataka: Administracija se vrši putem [Supabase Dashboarda](#). Tu je moguće pregledavati tablice, ručno mijenjati podatke i upravljati korisnicima.
- Slike (Storage): Pregled i brisanje slika vrši se u Supabase izborniku “Storage” -> bucket “units”.

Redovito održavanje:

- Backup: Supabase automatski radi dnevne sigurnosne kopije (ovisno o planu). Preporučuje se povremeno napraviti “Database Dump” kroz Supabase sučelje.
- Logovi:
 - Backend: Logovi se prate na AWS CloudWatch ili App Runner konzoli (tab “Logs”). Ovdje se vide greške u Java kodu.
 - Frontend: Logovi buildanja i deploja vidljivi su na Netlify Dashboardu.

Ažuriranje aplikacije: Kada napravite promjene u kodu:

- Backend:
 - Re-buildati Docker sliku: `docker build ...`
 - Push na Docker Hub: `docker push ...`
 - Na AWS App Runneru pokrenuti novi deploy (često je automatski ako je tako podešeno).
- Frontend:
 - Ponoviti `npm run build`.
 - Uploadati novi build folder na Netlify (ili `git push` ako je povezano s repozitorijem).

5. Postavljanje na Cloud (AWS App Runner i Netlify)

Ovaj projekt koristi hibridni pristup: Backend je kontejneriziran na AWS-u, a frontend je statički hostan na Netlifyju.

Backend: AWS App Runner App Runner je servis za jednostavno pokretanje kontejneriziranih web aplikacija. - Priprema: Osigurajte da je Docker slika na Docker Hubu (`docker push`). - Kreiranje servisa: - Prijavite se na AWS Konzolu -> App Runner. - Odaberite “Source code repository” -> Docker Hub. - Unesite URI slike: `tvoje_ime/room-`

rently-backend:latest. - Konfiguracija: - Pod Environment variables dodajte sve ključeve iz .env datoteke (DATASOURCE_URL, SUPABASE_S3_ACCESS_KEY, itd.). - Port: 8080. - Deploy: Kliknite "Create & Deploy". Dobit ćete javni URL backenda (npr. <https://xyz.awsapprunner.com>).

Frontend: Netlify Netlify služi za brzu distribuciju React aplikacije. - Konfiguracija: - U datoteku .env u React projektu upišite URL backenda dobiven od AWS-a: REACT_APP_API_URL=<https://xyz.awsapprunner.com> - Deploy: - Izvršite `npm run build`. - Prijavite se na Netlify i povucite build mapu u sučelje za deploy. - Završni korak: - Kopirajte URL nove Netlify stranice (npr. <https://moj-app.netlify.app>). - Vratite se na AWS App Runner i ažurirajte varijablu FRONTEND_URL s tom novom adresom radi CORS pravila.

6. Opis pristupa aplikaciji na javnom poslužitelju

Pristup za korisnike: Korisnici pristupaju aplikaciji putem internetskog preglednika (Chrome, Firefox, Safari, Edge) na javnoj domeni koju je dodijelio Netlify: - URL: [https://\[tvoj-projekt\].netlify.app](https://[tvoj-projekt].netlify.app)

Prilikom prvog učitavanja, aplikacija može raditi malo sporije ("Cold Start") ako AWS App Runner servis miruje (ovisno o konfiguraciji plana), što može potrajati 20-30 sekundi.

Pristup za administratore: Trenutno ne postoji zasebna URL ruta /admin unutar same aplikacije koja je odvojena od korisničkog sučelja. Administratorske ovlasti dodjeljuju se putem uloge korisnika (Role-Based Access Control). - Prijavite se s Google računom koji ima status administratora (definirano u bazi podataka). - Admin funkcionalnosti postaju vidljive unutar standardnog sučelja.

Ograničenja:

- Free Tier (Supabase): Baza podataka pauzira se nakon 7 dana neaktivnosti. Potrebno je logirati se u Supabase dashboard za ponovno aktiviranje.
- Upload slika: Maksimalna veličina slike ograničena je konfiguracijom Supabase Bucketa (standardno 5MB po datoteci).

9. Zaključak i budući rad

Osvrt na izradu projektnog zadatka

- Izrada projekta RoomRently bila je putovanje koje je od našeg tima zahtijevalo znatno veći angažman i trud nego što smo to na samom početku mogli očekivati. Iako smo u proces ušli s velikim entuzijazmom, brzo smo shvatili da put od koncepta do funkcionalne aplikacije podrazumijeva neprestano učenje i rješavanje kompleksnih tehničkih izazova. Budući da se većina članova tima ranije nije bavila profesionalnim razvojem softvera, početna faza bila je obilježena intenzivnim

usvajanjem novih znanja, posebice u radu s React bibliotekom na frontendu te Spring frameworkom na backendu.

- Okosnicu našeg sustava činila je PostgreSQL baza podataka, a najveći izazov, ali i najveća lekcija, bilo je postizanje stabilne komunikacije između tih različitih slojeva aplikacije. Proces povezivanja backenda s korisničkim sučeljem zahtijevao je duboko razumijevanje protoka podataka i rješavanje brojnih bugova koji su se pojavljivali u hodu. Kritički smo procjenjivali svaku zamišljenu značajku te smo uvidjeli da određene funkcionalnosti iako su na papiru možda jednostavne, u praksi zahtijevaju prilagodbu odnosno drugačiji pristup kako bi imale smisla za krajnjeg korisnika.
- Iako smo uspješno realizirali većinu planiranog, vremenski okviri primorali su nas da prioritet damo srži sustava, dok su neke sekundarne ideje morale pričekati buduće verzije. Uz samo programiranje, iznimno važno iskustvo bilo je savladavanje procesa deploymenta, čime smo zaokružili cjelinu razvoja aplikacije. Također, poseban naglasak stavili smo na pisanje kvalitetne dokumentacije, svjesni da bez jasno definiranih korisničkih zahtjeva i opisa arhitekture trud uložen u kod gubi na vrijednosti za buduće korisnike. Ovaj projekt za nas nije bio samo tehnički zadatak već lekcija o programskom inženjerstvu koja nam je pokazala kako se snalaziti u nepredviđenim situacijama i uspješno privesti kompleksan projekt kraju.

Tehnički izazovi

- Opseg projekta i optimizacija značajki - Tijekom rada smo shvatili da smo inicijalno planirali prevelik broj svojstava za našu aplikaciju, što je otežalo postizanje željene razine kvalitete za svaki pojedini detalj. Također, kako smo napredovali u znanju, uviđali smo brojne prilike za poboljšanje ranije napisanog koda.
- Povezivanje sustava i konzistentnost podataka – Velik izazov predstavljalo je usklađivanje React frontenda s našim Spring backendom. Morali smo rješavati česte nekompatibilnosti API poziva i osigurati da se podaci iz PostgreSQL baze ispravno i sigurno prikazuju korisnicima.
- Integracija vanjskih servisa – Kako bismo aplikaciju učinili funkcionalnom za najam, implementirali smo Google mape i mail servis. Početne poteškoće s njihovom integracijom premostili smo detaljnim proučavanjem i iscrpnim debugiranjem koda.
- Strukturiranje dokumentacije – Kroz proces smo naučili važnost kvalitetnog dokumentiranja svakog koraka. Fokusirali smo se na izradu dokumentacije koja će omogućiti nama i korisnicima lako razumijevanje arhitekture i funkcionalnosti naše platforme.
- Deployment – Završna prepreka bila je postavljanje RoomRently aplikacije na internet. Tehničke poteškoće s cloud servisima riješili smo razdvajanjem sustava,

koristeći Netlify za frontend te AWSza backend, čime smo osigurali stabilnost cijele platforme.

Stečena i potrebna znanja

Rad na aplikaciji RoomRently pružio nam je širok spektar novih vještina i kritički uvid u proces razvoja softvera. Kroz praktičan rad, naš tim je usvojio sljedeća znanja:

- Razvoj frontenda i backenda – Ovladali smo radom u React biblioteci za kreiranje dinamičnih sučelja te smo naučili graditi stabilnu poslovnu logiku koristeći Spring framework.
- Upravljanje podacima i deployment – Stekli smo iskustvo u strukturiranju relacijskih baza podataka unutar PostgreSQL sustava te uspješno savladali proces postavljanja cijele aplikacije na cloud platforme.
- Procjena resursa i vremena – Razvili smo znatno bolji osjećaj za planiranje vremena potrebnog za razvoj specifičnih značajki, naučivši kako balansirati između ambicije i realnih mogućnosti.
- Sinergija tima – Unaprijedili smo vještine timskog rada i komunikacije, što se pokazalo ključnim pri rješavanju kompleksnih problema.

Iako smo ponosni na ostvareno, prepoznajemo područja u kojima bi nam dodatno znanje omogućilo još brži i kvalitetniji razvoj:

- Sigurnosni standardi i protokoli – Dublje razumijevanje sigurnosnih aspekata razvoja softvera omogućilo bi nam još ranije prepoznavanje i eliminaciju potencijalnih rizika u sustavu rezervacija.
- Dublje poznavanje backend arhitekture – Naprednije razumijevanje strukture unutar Springa pomoglo bi nam u još boljoj organizaciji koda i optimizaciji performansi sustava.
- Dublje poznavanje deploymenta - Naprednije razumijevanje deployanja aplikacija na servisima poput AWS-a i sličnih kako bi nam pomoglo u budućem odabiru platforme za hosting.

Perspektive za nastavak rada

- Iako smo ponosni na postignuto, svjesni smo da bi pretvaranje aplikacije RoomRently u komercijalno spreman proizvod zahtijevalo značajna tehnička unaprjeđenja. Prije svega, bilo bi nužno provesti dubinsku optimizaciju cijelog sustava, što uključuje pisanje modularnijeg koda radi lakšeg održavanja te implementaciju naprednijih algoritama koji bi korisnicima omogućili preciznije i brže pretraživanje smještaja. Također, podizanje razine sigurnosti i rješavanje svih

preostalih ranjivosti bili bi apsolutni prioritet prije bilo kakvog puštanja aplikacije u stvarno produkcijsko okruženje.

- Što se tiče daljnjeg razvoja, prevladava mišljenje unutar tima da je trenutni opseg projekta sasvim adekvatan zahtjevima i ciljevima kolegija. Smatramo da smo kao tim 3.2 uspješno odgovorili na zadatak i prezentirali kvalitetno rješenje u okviru zadanih parametara. Iako projekt ima potencijal za industrijsku primjenu, takav bi pothvat iziskivao dodatne resurse i vrijeme koje trenutno nemamo, budući da su nam prioritet ostale akademske obaveze i nastavak studija.

Realizacija funkcionalnosti

- Za razliku od mnogih razvojnih procesa gdje se određeni dijelovi moraju izostaviti, naš tim je uspio u potpunosti realizirati sve funkcionalnosti koje smo inicijalno zamislili za RoomRently. Iako je put do finalnog proizvoda bio izazovan zbog rada u Springu i Reactu, nismo radili kompromise s planiranim značajkama sustava. Smatramo da je aplikacija u trenutnom stanju zaokružena cjelina koja nudi sve ključne mogućnosti za najam i upravljanje rezervacijama, čime smo u potpunosti ispunili ciljeve koje smo si postavili na početku semestra.

Ključni izazovi i rješenja

Opis izazova:

- Složenost tehnološkog stoga: Implementacija backenda u Spring Bootu te dinamičnog frontenda u Reactu predstavljala je izazov zbog manjka prethodnog iskustva s tim frameworkom. To je, uz paralelne fakultetske obveze, uzrokovalo sporiju inicijalnu fazu razvoja.
- Arhitektonska integracija i Cloud deployment: Povezivanje mikroservisne logike s bazama podataka (PostgreSQL/Supabase) te konfiguracija AWS servisa pokazali su se kao kritične točke. Sinkronizacija podataka između lokalnog okruženja i produkcije zahtijevala je detaljno poznavanje cloud infrastrukture a i naše aplikacije.
- Sinkronizacija tima i verzija koda: Koordinacija rada na različitim modulima (npr. istovremeni rad na API endpointima i React komponentama) često je dovodila do konflikata u kodu koje je trebalo sustavno rješavati kroz Git i bolju komunikaciju.

Primijenjena rješenja i naučene lekcije

Kako bi se osigurao uspješan završetak projekta, primijenili smo sljedeće strategije:

- Optimizacija deployment strategije: Umjesto kompliciranih ručnih konfiguracija, proces smo ubrzali korištenjem Netlifyja za automatski deployment frontenda, dok smo backend i bazu podataka stabilizirali kroz AWS i Supabase, koristeći njihove prednosti za skalabilnost i sigurnost.

- Iterativno planiranje i realni ciljevi: reorganizirali smo prioritete, fokusirajući se na ključne funkcionalnosti koje donose najveću vrijednost, čime smo smanjili tehnički dug.
- Vodstvo i agilna komunikacija: Kao voditelj, uveo sam sustav redovitih “check-in” sastanaka. Fokus je bio na transparentnosti, rano prepoznavanje blokada omogućilo nam je da preraspodijelimo zadatke prema specifičnim vještinama članova (npr. usmjeravanje onih vještijih u Javi na Spring Boot logiku, frontendašima ostaviti React itd...).
- Održivost koda i dokumentacija: Svjesni da projekt može rasti, inzistirali smo na modularnosti React komponenti i jasnoj strukturi Spring Boot kontrolera. Tehnička dokumentacija pisana je paralelno s kodom, što je olakšalo integraciju Supabase autentifikacije i baze.
- Bolje korištenje umjetne inteligencije: Shvatili smo da nije moguće izdvojiti previše vremena na pojedine stavke projekta, stoga smo za baš te iskoristili umjetnu inteligenciju koja nam je dosta pomogla pri razvoju i učenju.
- Jačanje timske kohezije: U trenucima visokog stresa, ključnim se pokazalo održavanje motivacije kroz jasno komuniciranje vizije projekta. Svaki član tima točno je znao kako njegov doprinos utječe na stabilnost cijelog sustava aplikacije.

Zaključak

- Realizacija ovog projekta predstavljala je intenzivan proces učenja koji je obuhvatio cijeli razvojni ciklus modernog softverskog rješenja od arhitekture baze podataka do cloud deploymenta. Rad na sustavu temeljenom na Reactu i Spring Bootu pružio je timu dragocjeno iskustvo u povezivanju backenda s dinamičnim korisničkim sučeljem.

Smjernice za budući razvoj:

Iako aplikacija u trenutačnom stanju ispunjava sve ključne zahtjeve, za njezinu punu primjenu u realnom poslovnom sektoru bilo bi potrebno implementirati sljedeće nadogradnje:

- Optimizacija i sigurnost: Dodatno učvršćivanje sigurnosnih protokola u komunikaciji između API-ja i PostgreSQL baze te fina optimizacija performansi na AWS infrastrukturi.
- Skalabilnost koda: Daljnja dekompozicija sustava u mikroservise kako bi se povećala modularnost i olakšalo dugoročno održavanje.

Završni osvrt voditelja tima

Kao voditelj tima, iznimno sam zadovoljan finalnim rezultatom. Naš glavni cilj bio je uspješno povezati React na frontend strani sa Spring Boot backendom, što smo ostvarili

kroz stabilnu i logičnu arhitekturu. Iako smo se tijekom rada susreli s tehničkim izazovima, tim je pokazao odličnu snalažljivost u rješavanju problema “u hodu”. Što je bilo lijepo kratko traje ali ovo se odužilo.

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, “Software engineering”, 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langaniere, “Object-Oriented Software Engineering”, 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, Software engineering book“, Department of Electrical and Computer Engineering, Rutgers University, [http://www.ece.rutgers.edu/~marsic/](http://www.ece.rutgers.edu/~marsic/books/SE) books/SE
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new>

A. Popis literature

Baza podataka

7. [Connect a PostgreSQL database to a Spring Boot Application Tutorial AnbuZ HobbieZ](#)

Backend

8. [Spring Boot Tutorial for Beginners \[2025\]](#)
9. [Spring Boot Reference Documentation](#)

Frontend

10. [React docs](#)
11. [React Booking | Reservation App UI Design for Beginners](#)
12. [React Google Login](#)
13. [Google Maps](#)
14. [Launch Site Inspiration](#)

B. Prikaz Aktivnosti Grupe

—Zapisnik 10.10.2025.—

- Jakov pokazuje ideje frontend dizajna i funkcionalnosti koje je pronašao na Youtubeu, itd.
- šifre u bazi podataka treba hashirati nekim pouzdanim algoritmom
- Noa će napraviti glavnu dokumentaciju za funkcionalnosti
- koristiti cemo Trello za lakšu organizaciju, odite na i registrirajte se na Trello imamo boardove za frontend, backend, baze podataka i organizaciju, kako koristiti (napisat ću neki tutorial u chat)

- trebamo Github posložiti za vođenje bilježaka o razvoju aplikacije
- svaki kod mora biti popraćen komentarima u kodu, draftom tjeka misli i problemima s kojima ste se susreli, nakon iz tog drafta napišete dokumentaciju svojeg rada koji će biti jasan nekome tko nije bio uz vas dok ste radili “posao” da može replicirati vaše korake i dobiti isti rezultat
- meet za funkcionalnosti projekta (SVI MORAJU BITI PRISUTNI)

ZA SUTRA @everyone - Noa će započeti dijagram baze podataka koji ćemo sutra pregledati da vidimo što još dodati i mijenjati - Nino i Mateo će pogledati OAuth2, Google Maps linking i slične funkcije da skupe informacije koje će koristiti kao smjernice kod implementacije - Karlo i Jakov će započeti dizajnirati stranice koje ćemo također sutra pregledati i prokomentirati - svi ćemo napisati ideje i todo kartice u TRELLO board što nam padne na pamet da treba napraviti za ubuduće (primjer: frontend zna da mora napraviti Početnu stranicu s get started buttonom i to će napisati u ideje pa prebaciti u todo kad dovrše karticu) - Josip će posložiti Github repo (s Wikiem, branchovima, README.md, itd.), TRELLO porediti da svi imaju potrebne liste, napisati howto za korištenje TRELLO-a i bilježenje napretka u razvoju koji ćemo pratiti

—Zapisnik 11.10.2025.—

- prošli smo HOWTO kanal
- vizualno objašnjavali kako Github Desktop koristiti
- updateali Matea i Nina koji prošli sastanak nisu bili
- gledali napredak u frontendu koji je Jakov napravio
- Jakov mora zapisati svoju implementaciju u Trello karticu i draft
- gledali smo dijagram baze podataka koju je Noa napravio
- treba predati dokument s funkcionalnostima do kraja sljedećeg tjedna (imamo sastanak u srijedu za prokomentirat napisane funkcionalnosti)

DO SRIJEDE 15.10. @everyone - svaka ekipa će napisati u Trello Megakarticu funkcionalnosti i u opis kartice raspisati funkcionalnosti koje istraže online s prioritetima, imajte na umu da moramo to formatirati za predaju do kraja tjedna pa kvalitetno to istražite i zapišite - Jakov će zapisati i urediti svoje kartice na Trello o onome što je implementirao kroz Youtube tutorial, također će linkati Youtube tutorial u komentare kartice - Jakov će raspisati draft što je radio konkretno kroz Youtube kanal kako bi mogao s Karlom napisati profesionalan final dokument sa snippetima i objašnjenjima koda - Jakov i Karlo će napisati final dokument koji je već na Githubu s markdown uređenjem teksta kako bi netko mogao replicirati što je Jakov implementirao, koristit će howto kako bi formatirali taj dokument u obliku problem/rješenje - Josip i tko bude voljan će ići na labos konzultacije u srijedu 15.10. kako bi saznao informacije o formatiranju dokumenta za predaju

—Zapisnik Krhen 15.10.2025.— @everyone

Pitanje: Chat na stranici?

Bilo bi lijepo imati chat. Komentari pri rezervaciji barem. Zahtjevi. Međusobno nema potrebe chatat. Eventualno chat bot, ali ne treba.

Pitanje: Funkcijske jesu dobro zapisane?

Sruk wiki kao primjer.

Pitanje: Prezentacije samo dokument ili uživo prezentirati?

Rok srijeda 22.10. sljedeći termin kreće od 8 ako namjesti Krhen, prezentacija plus uživo pričati. Ono iz zahtjeva treba prezentirati na lijepi način da je interesantno, ne sve raspisati. Preza i govor.

Pitanje: Github username Krhen, demos?

mkrhen, demos ivogabud

Pitanje: Statistika?

Prosjek rezervacija, broj dana, broj ljudi, od kud ljudi dolaze, traže li posebne usluge, ako 80% i više ljudi traže parking da staviš parking kao standardnu opciju, koliko sam imao praznog hoda usred sezone, kad sam imao najviše slobodnih soba. Staviti se u ulogu ugostitelja i razmišljati.

Pitanje: Razlike u administratoru, vlasniku hotela i korisniku? Ovisi kako zamislimo da ćemo to napraviti. Vlasnik može biti i admin. To dajemo hotelu JEDNOM.

BITNO RADIMO APIKACIJU NE BOOKING COM NEGO ZA JEDAN HOTEL DI IMA VLASNIKA, ADMINA ITD. PRIMJER RJEŠENJA:

<https://www.bluesunhotels.com/booking/hotel-elaphusa?adults=2&loyalty=true>

Korisnik ne bira broj sobe, samo pogled tipa prema moru itd.

—Zapisnik 24.10.2025.—

Prisutni: Noa, Mateo, Josip, Nino

- komentiramo kako treba detaljno pisati komentare na Trello s checkistama i opisima kako bi se mogla pratiti aktivnost za završnu tablicu koju će možda trebati pisati (u srijedu 29.10. ćemo saznati od Krhena)
- treba pisati u komentarima na karticama datume kad je tko što završio kako se ne bi desilo da Trello iztrippa i obriše aktivnosti + aktivnosti se jako nakrcaju jer svaki pomak kartice iz liste u listu, tipa Doing u Done se broji kao aktivnost
- gledamo bazu podataka koju je Noa napravio i komentiramo

- stavljamo Noin napredak na Github
- Noa i Mateo syncaju što su napravili kako bi Noa mogao na Github objaviti svoje

Za napraviti sljedeće - @Baza podataka do **nedjelje 26.10. (uključujući)** istražiti kako staviti na Github bazu podataka i napisati kako je mi drugi možemo koristiti - @Backend kad baza bude stavljena na Github treba nastaviti s pretvorbom frontenda u backend što znači da se većina toga povlači sa servera kao na labosu iz WEBa - @Baza podataka i @Backend trebaju komunicirati kako bi sve bilo syncano jedno s drugim i **oba tima rade na branchu bazepod!!!!** - @Baza podataka i @Backend trebaju do **srijede 29.10.** imati gotov login s Google Sign Inom, znači treba sve bitne informacije povući s OAuth2 i spremi u bazu i te informacije se moraju moći funkcijom izvlačiti iz baze (**FUNKCIJE MOGU BITI NAPRAVLJENE IAKO NEMA FRONTEND DIJELA KOJIM IH SE POKREĆE**) - @Baza podataka i @Backend trebaju do **nedjelje 2.11.** postaviti sve funkcije za pretraživanje smještajnih jedinica (filtere i samo pretraživanje mora funkcionirati)

JAKO BITNO @everyone - KOMUNICIRAJTE S DRUGIMA KAD RADITE, POGOTOVO S VAŠIM TIMOM.

- **DOKUMENTIRAJTE SVE KONTINUIRANO I ISPRAVNO POGOTOVO U TRELLO I WIKI “TIJEK IMPLEMENTIRANJA X”, SVA AKTIVNOST U TRELLU ĆE BITI TEMELJ DONOŠENJA BODOVA POJEDINCU NA KRAJU PROJEKTA.**
- **BILO KAKAV PROBLEM PITAJTE KOLEGU IZ TIMA, AKO OBOJE NE ZNATE ILI SAMI RADITE PITAJTE @Vođa/fullstack. POTRUDITE SE PRONAĆI ODGOVOR SAMI UZ CHATGPT, GOOGLE, ITD. PRIJE NEGO DRUGU OSOBU PITATE, ALI ČIM VIDITE DA PREVIŠE VREMENA TROŠITE NA TRAŽENJE BEZ NAPRETKA PITAJTE!**

—Zapisnik Krhen 30.10.2025.— @everyone

Pitanje: Slike u bazi podataka u byteovima ili samo putanja? Odgovor: Putanja/URL uvijek u bazi podataka. Napraviti filesystem u kojem ćemo pohranjivati sve slike, zvukove, itd.

Pitanje: Aktivnost Wiki page treba li aktivno zapisivati ili je okej na kraju prekopirati s Trella? Odgovor: Ne vidi se kad se commita Wiki, okej je Trello i zapisujte kada tko što doda na Wiki jer se ne može pratiti. Napisati snapshot aktivnosti do međuispita na Wiki. Pišite i circa broj sati koliko ste što radili.

Pitanje: Dijagrami za zahtjeve i arhitekturu sustava, možemo li ih na kraju zaljepiti kad budemo sigurni kako će izgledati sve ili treba aktivno ih nadograđivati? Odgovor: Trebalo bi raditi postepeno pa nadodavati nove elemente u dijagrame kako ih razrađujemo.

Pitanje: Dokumentacija i prototip predaja prije međuispita? Odgovor: Sve u Wiki samo mora biti gotovo do roka, ne moramo ekstra raditi pdf ili što već ako se ne promjeni do tada zahtjev. Moramo imati funkcionalnost kao upisivanje smještajne jedinice, da aplikacija otvori prozor tipa i upisuješ informacije o jedinici u prozor i da se to sprema u bazu podataka i mora to iz baze aplikacija povući i prikazati tipa kao search smještajnih jedinica.

Napraviti superusera/admina koji to može napraviti, napraviti fake google mail koji će biti default admin pa se može nadodavati druge.

Pitanje: Bodovanje commitovi? Odgovor: Vođa dodjeljuje bodove nakon prve predaje prototipa (prije međuispita) pa će neradnici biti kažnjeni prikladnim brojem bodova.

—Zapisnik 9.11.2025.—

- diskutiranje svega što smo dokumentirali
- Noa će zapisat što konkretno radi kod ne samo objasniti kod liniju po liniju (relacije stvaraju tablice u bazu podataka kad se backend spoji na bazu)
- treba zapisati u Trello sve dokumentirano i zapisati sate i datume kad je napravljeno
- gledali smo dokumentaciju backenda i dokumentaciju kako postaviti na poslužitelja
- Mateo će zapisati kako lokalno pokrenuti backend da funkcionira s environment varijablama
- Josip će zapisati dokumentaciju postavljanja Supabase poslužitelja
- gledali smo dokumentaciju frontenda
- treba nadodati sve linkove od Youtube tutoriala u Tijek Implementiranja i u Popis Literature
- treba napisati da se koristio ChatGPT
- provjeriti sve dijagrame koje trebamo imati i koje već imamo
- raspodijelili smo dijagrame po osobama, dijagrame treba danas napraviti
- provjerili smo wiki naslove ako imamo sve

@everyone - trebate danas napraviti dijagrame koji su vam dodjeljeni na Trello stranici - provjerite sve dokumentacije - zapišite vrijeme uloženo i datume kad je napravljeno

—Zapisnik 17.12.2025.—

- diskutiranje napravljenog projekta do sada
- ako je rupa u danu npr petak i subota
- Potrebno popraviti greške oko datuma rezervacije da nema preklapanja
- Potrebno dodati funkcionalnost izmjene imena za hotel i prikaz imena glavnog hotela
- Sobe su tipa i imaju od a101 do a110, nama su sada apartmani i sobe poput a101 a102 svaka za sebe što može biti nepregledno, to je optimiranje koje piše u zadatku, da biraš tip A zgrada i da te on automatski smjesti u onaj najpovoljniji čisti ili slično.
- Tipske sobe npr A zgrada i onda prikaze broj primjerice 12 soba i onda te raspoređuje po nekakovom redu.
- Dodati xml u izvoz statistike i rezervacija
- Implementirati pdf confirmationa na mail
- Potrebno spremati slike u bazu

—Zapisnik 14.1.2026.—

-Diskutiranje napravljenog projekta potrebno za napraviti:

- Dokumentaciju dovršiti
- aplikaciju pohraniti na github
- Power point napraviti po templateu
- Potrebno napraviti 6 testiranja komponenti, 4 za cijeli sustav.
- Potrebno dokumentirati testiranja.
- Pdf staviti na točan path

Tablica aktivnosti

Napomena:

Doprinosi u aktivnostima navedeni su u satima po članovima grupe po aktivnosti.

Potrebno je navesti koliko je sati koja osoba uložila u pojedinu komponentu.

Aktivnost	Josip Mrakovčić	Jakov Zekić	Karlo Živković	Noa Rešetar	Mateo Cerčić	Nino Strčić
Upravljanje projektom	70	0	0	0	0	0.001
Opis projektnog zadatka	2	0	0	0	0	0
Funkcionalni zahtjevi	2	3	3	3	3	0
Opis pojedinih obrazaca	4	0	0	2	0	0
Dijagram obrazaca	3	0	0	7	0	0
Sekvencijski dijagrami	0	0	4	0	3	1
Opis ostalih zahtjeva	1	2	2	0	0	0
Arhitektura i dizajn sustava	2	3	3	2	0	0
Baza podataka	0	0	0	20	0	0
Dijagram razreda	0	0	2	2	0	0
Dijagram stanja	0	0	3	0	0	0
Dijagram aktivnosti	0	0	6	0	0	0

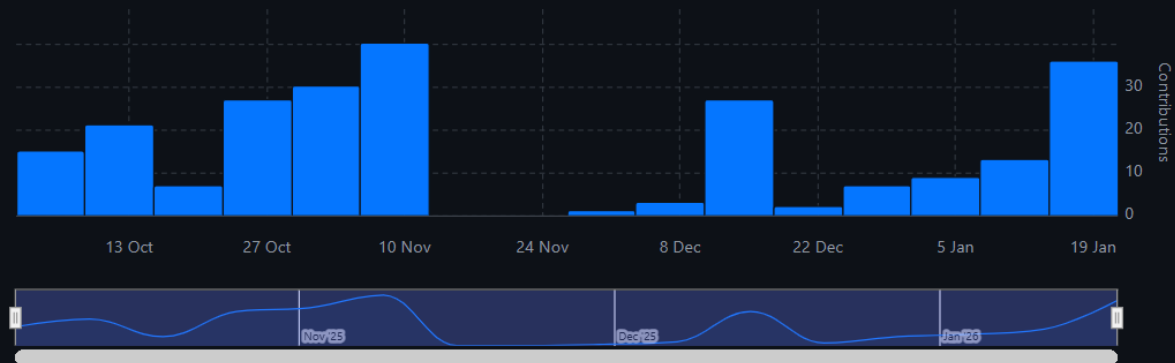
Aktivnost	Josip Mrakovčić	Jakov Zekić	Karlo Živković	Noa Rešetar	Mateo Cerčić	Nino Strčić
Dijagram komponenti	0	6	1	0	0	0
Korištene tehnologije i alati	1	0	0	2	2	0
Ispitivanje programskog rješenja	10	4	12	2	22	0
Dijagram razmještaja	0	3	1	0	0	0
Upute za puštanje u pogon	10	30	83	18	45	0
Dnevnik sastajanja	7	2	0	0	0	0
Zaključak i budući rad	0	2	0	0	0	0
Popis literature	1	1	1	1	1	0
Izrada početne stranice	0	20	0	0	0	0
Izrada baze podataka	0	0	0	35	0	0
Spajanje s bazom podataka	6	0	0	5	5	0
Izrada prezentacije	12	0	0	12	0	0
Frontend	20	109	12	2	35	0
Backend	60	0	0	15	50	0
Hostanje na Cloud	16	0	0	0	7	0

Dijagrami pregleda promjena

Vizualni pregled doprinosa i dinamike rada članova tima vidljiv je u sljedećim dijagramima, koji su izvučeni iz GitHub Contributors statistike.

Commits over time

Weekly from 5. lis 2025. to 18. sij 2026.



JosipMrakovcic

51 commits 9,406 ++ 4,003 --

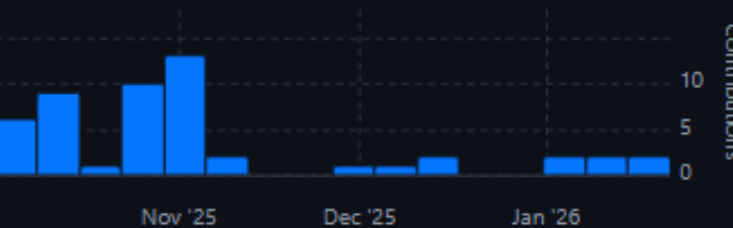
#1



JakovZekic0

51 commits 29,546 ++ 686 --

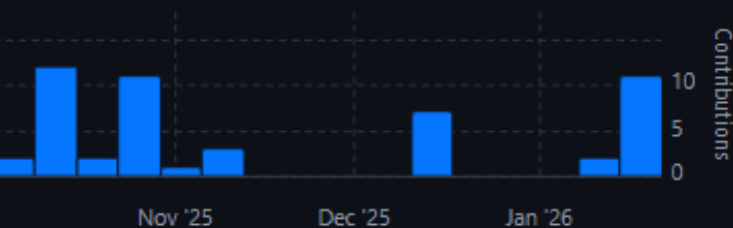
#2

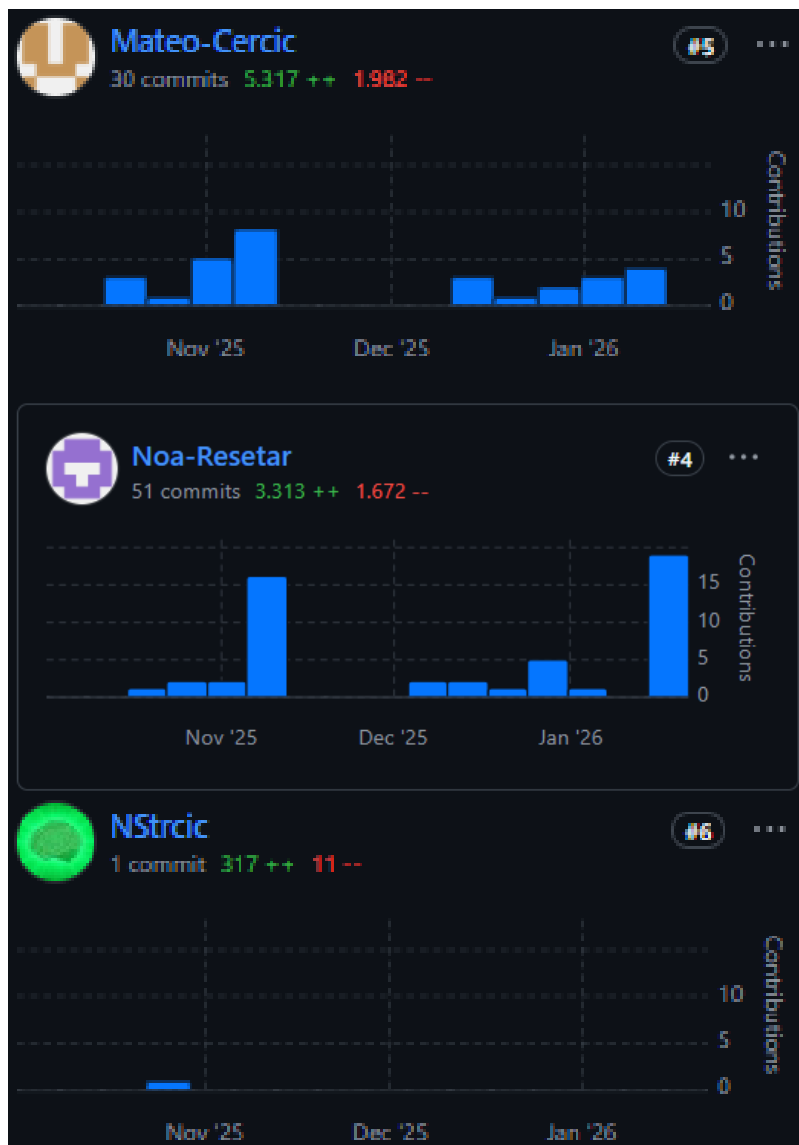


karlo-zivkovic1

51 commits 9,959 ++ 2,708 --

#3





C. Dnevnik promjena dokumentacije

Za praćenje promjena koristimo Trello workspace (<https://trello.com/w/roomrently>).

Dnevnik promjena dokumentacije

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Napravljen početni predložak dokumentacije	Josip Mrakovčić	10.10.2025
0.2	Kreirana Home stranica i prvi sadržaj	Josip Mrakovčić	10.10.– 20.10.2025

Rev.	Opis promjene/dodatka	Autori	Datum
0.3	Dodana poglavlja: 1. Opis projektnog zadatka, 2. Analiza zahtjeva	Josip Mrakovčić	20.10.– 25.10.2025
0.4	Kreirana početna verzija poglavlja 3. Specifikacija zahtjeva sustava	Josip Mrakovčić	24.10.2025
0.5	Višestruka ažuriranja Home, 1. Opis projektnog zadatka, 2. Analiza zahtjeva	Josip Mrakovčić	24.10.– 31.10.2025
0.6	Kreirana početna verzija poglavlja 4. Arhitektura sustava	Josip Mrakovčić	30.10.2025
0.7	Ažuriranja pogl. 3 i 4 – više iteracija	Josip / Karlo / Noa / Jakov	31.10.– 07.11.2025
0.8	Kreirani dokumenti implementacije: Backend, Frontend, Baza podataka, FullStack	Josip / Mateo / Noa / Jakov / Karlo	24.10.– 07.11.2025
0.9	Intenzivna ažuriranja implementacijskih poglavlja (Backend, Frontend, Baza, FullStack) – veliki broj nadogradnji	Mateo / Jakov / Noa / Josip / Karlo	07.11.– 14.11.2025
0.95	Dodana „Dokumentacija o objavljivanju aplikacije na javnom poslužitelju“ i višestruka ažuriranja	Mateo / Josip	07.11.– 11.11.2025
1.0	Finalna konsolidacija svih wiki stranica, uređivanje strukture i priprema dokumentacije za predaju	Svi članovi tima	14.11.2025
1.1	Dijagram stanja	Karlo Živković	19.12.2025
1.2	Dijagram aktivnosti	Karlo Živković	20.12.2025
1.3	Arhitektura komponenata i razmještaja	Karlo Živković , Mateo Cerčić, Noa Rešetar, Jakov Zekić	20.12.2025
1.4	Tehnologije za implementaciju aplikacije	Mateo Cerčić	20.12.2025
1.5	Ispitivanje programskog rješenja	Mateo Cerčić, Karlo Živković	21.12- 29.12.2025

Rev.	Opis promjene/dodatka	Autori	Datum
1.6	Promjena strukture dokumentacije za drugu reviziju	Karlo Živković	4.1.2026
1.7	Ažuriranje informacija dokumentacije za drugu reviziju	Karlo Živković , Mateo Cerčić, Noa Rešetar, Jakov Zekić	4.1.- 17.1.2026
1.8	Ažuriranje Arhitektura komponenata i razmještaja	Jakov Zekić	18.1.2026
1.9	Ažuriranje Dijagrama obrasca uporabe Vlasnik Voditelj	Noa Rešetar	18.1.2026
1.10	Ažuriranje dijagrama stanja	Karlo Živković	18.1.2026
1.11	Ažuriranje dijagrama aktivnosti	Karlo Živković	18.1.2026
1.12	Ispitivanje sustava	Karlo Živković , Mateo Cerčić	10.1.- 19.1.2026
1.13	Zaključak i budući rad	Jakov Zekić	19.1.2026
1.14	Ključni izazovi i rješenja	Jakov Zekić	19.1.2026
1.15	Dijagrami pregleda promjena	Jakov Zekić	19.1.2026
1.16	Upute za puštanje u pogon	Mateo Cerčić	18.1.- 22.1.2026
1.17	Detaljniji opis tehnologija za implementaciju aplikacije	Noa Rešetar	18.1.- 22.1.2026
2.0	Finalna konsolidacija svih wiki stranica, uređivanje strukture i priprema dokumentacije za predaju	Svi članovi tima	1.1- 22.1.2026

Evidencija promjena sadrži popis promjena izvršenih tijekom cijelog životnog trajanja dokumentacije.

Svrha je praćenje napretka svake promjene na temelju njezina preispitivanja, odobrenja (ili odbijanja), provedbe i zaključenja.

Kvalitetan dnevnik promjena sadrži i datum promjene te procjenu njezina utjecaja na projekt u smislu rizika, vremena i troškova.

Promjene su vidljive svim dionicima projekta. U povijest se uključuju i odbačene promjene.

Praćenje promjena

Sve navedene promjene mogu se detaljno pratiti kroz **povijest revizija (Revisions)** svakog pojedinog Wiki dokumenta.

GitHub automatski evidentira svaku izmjenu zajedno s autorom, datumom i pripadajućim commit ID-em, čime se omogućuje jasan uvid u tijek razvoja dokumentacije.

Pregledom revizija na svakom Wiki pageu dionicima je olakšano praćenje svih izmjena, kao i razumijevanje njihovog utjecaja na projekt — uključujući rizike, vrijeme i troškove.

Na taj način osigurava se potpuna transparentnost i mogućnost praćenja kompletne povijesti izmjena sve do revizije **1.0**.

D. Funkcionalnosti u kodu

Frontend sažetak

Glavne rute :

Javne rute (Za sve posjetitelje):

/ (Landing Page): Ulazna stranica aplikacije (LandingScreen). Služi za prvi dojam i usmjeravanje korisnika na glavnu tražilicu.

/main (Početna stranica): Glavni ekran (Home) s istaknutim ponudama, kategorijama i primarnom tražilicom.

/hotels (Rezultati pretrage): Stranica (List) koja prikazuje popis svih dostupnih jedinica nakon što korisnik unese parametre pretrage.

/hotels/:id (Detalji smještaja): Dinamička ruta (Hotel) koja prikazuje sve detalje, galeriju i mogućnost rezervacije za točno određenu jedinicu (identificiranu preko id).

Korisničke rute (Za prijavljene goste)

Rute namijenjene onima koji koriste sustav za rezervacije:

/booked-reservations (Moje rezervacije): Prikaz svih rezervacija koje je trenutno prijavljeni korisnik napravio (UserReservations).

Administrativne rute (Za vlasnike i admina)

Rute za upravljanje objektima i poslovanjem:

/admin (Upravljanje jedinicama): Pregled svih smještajnih jedinica u sustavu (RentalUnits) gdje ih admin može brisati ili odabirati za uređivanje.

/form (Dodavanje novog): Prazna forma (ApartmentForm) za unos potpuno novog apartmana ili sobe.

/form/:id (Uređivanje postojećeg): Ista forma (ApartmentForm), ali se učitava s podacima postojeće jedinice (putem id) radi izmjena.

/owner-dashboard (Vlasnička ploča): Centralno mjesto za statistiku i pregled poslovanja iz perspektive iznajmljivača (OwnerDashboard).

Objašnjenja komponenti Frontenda

Featured

Dohvaćanje podataka (Fetch): Koristi axios za asinkrono dohvaćanje statistike smještajnih jedinica grupiranih po tipu pogleda (sea, village, lake) s backend API-ja.

Upravljanje stanjem (State Management): Pohranjuje podatke o broju jedinica i URL-ovima slika u viewData state.

Dinamički prikaz slika: Implementira logiku koja automatski prepoznaje je li slika pohranjena eksterno (S3/Cloud) ili lokalno na serveru, uz "fallback" sustav koji prikazuje zadanu sliku u slučaju greške.

Navigacija i filtriranje: Omogućuje korisniku klik na kategoriju, što ga preusmjerava na listu hotela s već postavljenim filterom za odabrani tip pogleda pomoću react-router-dom stanja.

Mapiranje UI komponenti: Koristi fiksnu listu kategorija koju uparuje s dinamičkim podacima, osiguravajući konzistentan raspored na sučelju čak i ako backend vrati nepotpune podatke.

Featured properties

Dohvaćanje najbolje ocijenjenih jedinica (Top-Rated Fetch): Poziva backend endpoint /unit/top-rated kako bi prikazao smještaj s najboljim recenzijama.

Hibridno upravljanje lokacijom: Kombinira podatke o općoj adresi objekta iz /api/location s podacima pojedinačnih jedinica, osiguravajući da polje lokacije nikada ne ostane prazno.

Inteligentna obrada slika: Koristi logiku koja detektira je li putanja slike apsolutna (npr. Amazon S3) ili relativna (lokalni server), te implementira onError handler koji automatski mijenja neispravne linkove zadanom slikom.

Korisničko iskustvo (UX): Sadrži stanja za učitavanje (loading) i prazne rezultate, čime se izbjegava "skakanje" elemenata na ekranu dok podaci pristižu.

Dinamičko ocjenjivanje: Automatski formatira numeričke ocjene na jednu decimalu i dodaje tekstualne opise (npr. "Excellent") ovisno o visini ocjene, uz podršku za nove objekte bez recenzija.

Header

Upravljanje parametrima pretrage: Komponenta prati tri ključna unosa: destination (tekst), date (raspon datuma) i options (broj gostiju i soba).

Integracija Kalendara: Koristi react-date-range za vizualni odabir datuma, uz ugrađenu zaštitu (minDate) koja sprječava odabir datuma u prošlosti.

Validacija noćenja: Implementirana je logika koja provjerava razliku u danima (differenceInCalendarDays). Ako je odabrano 0 dana (isti datum dolaska i odlaska), sustav blokira pretragu i upozorava korisnika.

Perzistencija podataka: Prije navigacije, svi parametri pretrage spremaju se u sessionStorage pod ključem lastSearch, što omogućuje aplikaciji da "zapamti" što je korisnik tražio čak i nakon osvježavanja stranice.

Dinamički UI (Conditional Rendering): Header ima dva načina rada (type="list"). Ako je u "list" modu, sakriva veliki pretraživač kako bi oslobodio prostor za rezultate, što je standardna praksa u booking aplikacijama.

Upravljanje stanjem opcija: Koristi kompleksni objekt unutar useState za istovremeno praćenje više brojača (odrasli, djeca, sobe) s ugrađenim limitima (npr. minimalno 1 odrasla osoba).

Maillist

Brendiranje i marketing: Prikazuje ključnu poruku ("Save time, save money") i kratki opis platforme, služeći kao informativni završetak stranice ili sekcije.

Dinamički Copyright: Koristi JavaScript Date objekt za automatsko dohvaćanje trenutne godine, čime se eliminira potreba za ručnim ažuriranjem informacija o autorskim pravima svakog 1. siječnja.

Modularni dizajn: Komponenta je jednostavna i statična, što je čini idealnom za ponovnu upotrebu na dnu različitih stranica (Home, List, Hotel details).

Navbar

Google OAuth integracija: Upravlja kompletnim ciklusom prijave putem Google-a, uključujući dekodiranje JWT tokena i sinkronizaciju s vlastitom bazom podataka (/addPerson, /me).

RBAC (Role-Based Access Control): Dinamički prikazuje gumbе za navigaciju (Admin Panel, Owner Dashboard, Reservations) na temelju korisničkih prava (is_admin, is_owner).

Prisilna dopuna profila (Location Guard): Implementira mehanizam koji prepoznaje nepotpune profile i blokira sučelje modalnim prozorom dok korisnik ne unese državu i grad.

Pametni Dropdown sustav: Sadrži logiku ovisnih polja (gradovi se mijenjaju ovisno o odabranoj državi) te podržava ručni unos ("Other") ako lokacija nije na popisu.

Upravljanje sesijom: Koristi localStorage za perzistenciju korisničkih podataka i access_token za autorizaciju API zahtjeva prema backendu.

OwnerDashboard

Napredna Analitika (BI): Koristi Chart.js za vizualizaciju poslovnih indikatora:

Yearly Traffic: Praćenje broja gostiju kroz mjesece.

Unit Popularity: Pie chart koji pokazuje koji apartmani donose najviše rezervacija.

Amenities: Analiza koji su dodatni zahtjevi (npr. doručak, parking) najpopularniji.

Sustav za generiranje izvještaja:

PDF: Generira profesionalne izvještaje koji kombiniraju tekstualne tablice i slike grafikona.

Excel/XML: Omogućuje izvoz sirovih podataka za daljnju obradu u računovodstvenim programima.

Upravljanje životnim ciklusom rezervacije:

Vlasnik može potvrditi (Confirm) ili odbiti (Reject) nove zahtjeve.

Sigurnosna logika: Gumb Complete je onemogućen dok god datum prijave ne prođe (sprečava prerano zatvaranje rezervacije).

UX optimizacije:

localStorage pamti na kojem je tabu vlasnik stao.

Automatsko detektiranje dostupnih godina u bazi podataka za filtriranje.

Responzivni dizajn (tablica se na mobitelu pretvara u kartice).

PropertyList

Hibridni pristup slikama: Komponenta je dovoljno pametna da prepozna radi li se o vanjskom linku (npr. Cloudinary/S3) ili relativnoj putanji s tvog API-ja, uz dodatan onError osigurač.

Dinamička statistika: Umjesto da pišeš "50 units" fiksno, korisnik vidi stvarno stanje iz baze, što povećava povjerenje u platformu.

UX - Pametna navigacija: Korištenje Maps s state objektom omogućuje da stranica s hotelima (/hotels) odmah primi parametar kreveta bez dugačkih URL upita (Query params).

Reserve modal

Ona omogućuje korisnicima odabir datuma putem interaktivnog kalendara, upravljanje brojem gostiju te odabir dostupnih pogodnosti (amenities).

Upravljanje zauzetim datumima: Koristi `useEffect` za dohvaćanje podataka s backend-a (occupied-dates). Posebno je bitna logika `days.slice(0, -1)` – ona omogućuje tzv. "hotel check-out" sustav, gdje je zadnji dan nečije rezervacije slobodan za check-in novog gosta.

Dinamičke pogodnosti: Umjesto prikazivanja svih mogućih opcija, komponenta filtrira listu (availableAmenities) i prikazuje samo one koje specifična jedinica (unit) nudi (npr. ako jedinica nema klimu, taj se checkbox neće pojaviti).

Validacija kapaciteta: Funkcija `modifyGuests` sprječava korisnika da odabere više odraslih ili djece nego što je definirano u bazi podataka za tu jedinicu.

Searchitem

Napredno upravljanje slikama: Koristi kombinaciju stanja (`imageLoaded`, `isError`) i `useRef` kako bi osigurala glatko učitavanje. Ako slika ne postoji ili dođe do greške pri učitavanju, automatski se prikazuje "No Photo Found" placeholder, izbjegavajući ružne "broken image" ikone u pregledniku.

Dinamičke oznake (Rating Labels): Funkcija `getRatingLabel` pretvara numeričku ocjenu (npr. 9.2) u opisnu kategoriju (npr. "Superb"), što je standard u industriji putovanja (poput Booking.com-a).

Geolokacijski Fallback: Ako jedinica nema definiranu točnu lokaciju, komponenta pokušava dohvatiti globalnu adresu s API-ja ili postavlja zadani "Zagreb, Croatia".

userreservations

Inteligentno sortiranje: Rezervacije nisu poredane nasumično. Sustav prvo odvaja otkazane (Cancelled) rezervacije i šalje ih na dno liste, dok aktivne rezervacije sortira kronološki (najbliži datumi dolaska su na vrhu).

Vremenski ograničeno ocjenjivanje: Funkcija `canUserRate` provodi strogu poslovnu logiku: gumb za ocjenjivanje (1–10) pojavljuje se samo ako je status "Completed" i ako nije prošlo više od 3 dana od datuma odlaska.

Upravljanje stanjima (Modali): Komponenta koristi ugrađene modale za potvrdu otkazivanja, sprječavajući slučajne klikove koji bi mogli rezultirati gubitkom rezervacije.

Rental units

Upravljanje ulogama (Role Management): Administrator može promijeniti ulogu bilo kojeg korisnika (User, Owner, Admin) putem select izbornika. Sustav ima ugrađenu zaštitu koja onemogućuje administratoru da obriše samog sebe ili promijeni vlastitu ulogu, čime se sprječava gubitak pristupa panelu.

Globalna lokacija objekta: Poseban dio panela omogućuje postavljanje adrese cijelog objekta. Ova adresa se koristi kao fallback u Searchitem komponenti ako pojedinačna jedinica nema definiranu lokaciju. Ugrađen je i Google Maps iframe za vizualnu potvrdu adrese.

Perzistencija stanja (Sidebar): Korištenjem localStorage, aplikacija pamti koji je tab (units ili users) bio otvoren, pa čak i nakon osvježavanja stranice administrator ostaje na istom mjestu.

Form

Hibridno upravljanje slikama: Komponenta istovremeno upravlja slikama koje su već na serveru (imaju id) i novim datotekama koje korisnik tek učitava (imaju blob: URL-ove).

"Cover Photo" logika: Korisnik može kliknuti na bilo koju sliku u pretpregledu kako bi je postavio kao naslovnu. Komponenta prati coverIndex i prilikom slanja osigurava da backend točno zna koja slika predstavlja glavni vizual jedinice.

Pametno brisanje: Slike označene za uklanjanje dodaju se u imagesToDelete niz. Tek nakon uspješnog slanja cijele forme, šalju se pojedinačni DELETE zahtjevi za svaku od tih slika, čime se izbjegava slučajno brisanje podataka.

Dinamički tipovi jedinica: Forma se prilagođava ovisno o tome je li odabran "Apartment" (prikazuje broj soba unutar jedinice) ili "Room" (prikazuje broj identičnih jedinica dostupnih za najam).

Home

Resetiranje stanja pretrage: Unutar useEffect kuke (hook), komponenta koristi sessionStorage.removeItem("lastSearch"). Ovo je pametan potez jer osigurava da se prethodni parametri pretrage (datumi, broj osoba) obrišu svaki put kada se korisnik vrati na početnu stranicu, omogućujući mu "svježi" početak.

Modularni dizajn: Umjesto pisanja svog koda na jednom mjestu, stranica koristi specijalizirane komponente. To olakšava održavanje i omogućuje da se npr. Navbar ili Maillist koriste i na drugim stranicama.

Hijerarhija sadržaja: Stranica prati standardni marketinški lijevak za turističke web stranice:

Header: Obično sadrži tražilicu.

Featured: Istaknute destinacije ili ponude.

Propertylist: Kategorizacija (ovdje po kapacitetu).

Featuredproperties: Najbolje ocijenjeni smještaji.

Hotel

ni "lightbox" preglednik slika. Posebna pažnja posvećena je logici učitavanja: sustav prepoznaje jesu li slike pohranjene na vanjskom servisu (poput AWS S3) ili su relativni putovi na lokalnom serveru, osiguravajući da se svaki vizual ispravno renderira.

Vizualni sustav pogodnosti (Amenities): Umjesto običnog popisa teksta, koristi se sustav "tagova" s ikonama (✅, 📅, 👥). To omogućuje korisniku da u par sekundi skenira što smještaj nudi (WiFi, Parking, Klima).

Dinamička lokacija (Google Maps): Stranica integrira interaktivnu kartu. Ako jedinica nema specifično definiranu lokaciju, sustav automatski povlači "globalnu adresu" objekta (npr. glavnu adresu hotela/zgrade), osiguravajući da karta nikada ne ostane prazna.

Modularni proces rezervacije: Gumb "Reserve or Book Now!" otvara ReserveModal. Ovakav pristup omogućuje korisniku da započne proces rezervacije bez napuštanja stranice, što značajno povećava stopu konverzije.

List

Slojevitost podataka (State Management): Komponenta pametno kombinira tri izvora podataka pri učitavanju:

sessionStorage (savedSearch): Omogućuje da se filteri zadrže čak i ako korisnik osvježi stranicu ili se vrati s detaljnog prikaza hotela.

location.state: Prima podatke poslane izravno s Header tražilice na početnoj stranici.

Defaultne vrijednosti: Osiguravaju da aplikacija radi čak i ako korisnik dođe izravno na /list URL.

Dinamički sustav filtriranja: Za razliku od jednostavnih tražilica, ovdje se koristi axios.get s opsežnim params objektom. To znači da se filtriranje događa na backendu (bazi podataka), što je znatno brže i skalabilnije od filtriranja tisuća rezultata unutar samog Reacta.

Interaktivni kalendar (react-date-range): Koristi se za odabir termina, a uključena je i validacija koja spriječava pretragu ako je datum odlaska prije datuma dolaska (pomoću differenceInCalendarDays).

Landingscreen

Jednostavna navigacija: Koristi useNavigate kuku iz react-router-dom biblioteke. Funkcija Mapsmain preusmjerava korisnika na rutu /main, što je u vašem slučaju početna stranica s tražilicom.

Strateški marketing: Naslov (h1) sadrži "Value Proposition" – jasno objašnjava što korisnik dobiva: od malih soba do modernih apartmana, uz naglasak na jednostavnost pretraživanja.

Backend sažetak

Sigurnosna konfiguracija (SecurityConfig)

Glavne rute (API Security Policy):

Javne rute (Dostupne svim posjetiteljima):

Rute koje omogućuju osnovno pregledavanje sadržaja, pretragu i rad kalendara bez potrebe za prijavom:

- **/unit/all, /unit/filter, /unit/top-rated:** Osnovno dohvaćanje i filtriranje smještajnih jedinica.
- **/unit/{id}:** Detaljni prikaz specifične jedinice (korištenjem regex validacije za ID).
- **/unit/counts-by-beds, /unit/counts-by-view, /unit/summary:** Rute za statistiku i prikaz kategorija na početnoj stranici.
- **/unitReservation/occupied-dates/:** Ključna ruta za kalendar koja omogućuje prikaz zauzetosti.
- **/api/location:** Javni pristup općim informacijama o lokaciji objekta (GET metoda).

Korisničke rute (Zahtijevaju validan JWT / Login)

Rute namijenjene prijavljenim gostima za interakciju sa sustavom rezervacija:

- **/addPerson, /me, /updateCountry:** Inicijalna registracija nakon Google prijave i upravljanje profilom korisnika.
- **/unitReservation/add:** Slanje zahtjeva za novu rezervaciju.
- **/unitReservation/my-reservations:** Pregled povijesti i trenutno aktivnih rezervacija korisnika.
- **/unitReservation/cancel/ & /unitReservation/rate/:** Akcije otkazivanja i ocjenjivanja vlastitih rezervacija.

Administrativne i Vlasničke rute (Login + RBAC kontrola)

Iako su zaštićene osnovnom autentifikacijom, ove rute u kontrolerima provjeravaju uloge (Admin/Owner):

- **/unit/add, /unit/update/:** Upravljanje (dodavanje i uređivanje) smještajnim jedinicama i njihovim opisima.
- **/unitImg/:** Potpuna kontrola nad multimedijom (upload novih i brisanje postojećih slika).

- **/allPersons, /deletePerson/, /updateRole/:** Administracija korisnika, brisanje računa i dodjeljivanje prava pristupa.
- **/unitReservation/all:** Centralni pregled svih rezervacija u sustavu za vlasnike i administratore.
- **/unitReservation/update-status/:** Poslovna logika za potvrđivanje (Confirm), odbijanje (Reject) ili zatvaranje rezervacija.

Objašnjenja Security komponenti

SecurityFilterChain (Centralna zaštita)

Izolacija resursa: Koristi requestMatchers za precizno definiranje tko može pristupiti kojem dijelu sustava. Ovo osigurava da kritične funkcije, poput promjene uloga ili brisanja jedinica, budu nedostupne običnim posjetiteljima.

Stateless arhitektura: Sustav je konfiguriran kao .oauth2ResourceServer. Umjesto klasičnih sesija u memoriji, backend validira svaki dolazni zahtjev putem JWT tokena, što omogućuje veću sigurnost i lakše skaliranje aplikacije.

Regex validacija rute: Putanja /unit/{id:[0-9]+} koristi regularne izraze kako bi osigurala da se API poziva isključivo s bročanim identifikatorima, čime se automatski odbacuju neispravni upiti već na razini sigurnosnog filtra.

CORS Configuration (Cross-Origin Resource Sharing)

Dinamička sigurnost podrijetla: Backend strogo kontrolira pristup i dopušta komunikaciju isključivo s adrese definirane u \${frontend.url} varijabli, sprječavajući zahtjeve s neovlaštenih domena.

Sanitizacija URL-a: Implementirana je logika replaceAll("/\$", "") koja automatski čisti URL od kose crte na kraju. Ovo sprječava česte pogreške u konfiguraciji zbog kojih preglednici blokiraju "cross-origin" zahtjeve.

Optimizacija performansi (MaxAge): Postavljanjem setMaxAge(3600L), preglednik sprema dopuštenja (preflight odgovore) na jedan sat. To ubrzava rad frontenda jer se smanjuje broj ponovljenih provjera sigurnosnih polica pri svakom kliku.

OAuth2 & JWT Integracija

Google OAuth kompatibilnost: Integracija s jwt(Customizer.withDefaults()) omogućuje backendu da automatski prepozna, dekodira i validira tokene koje korisnik dobije nakon prijave putem Google-a na frontend aplikaciji.

Defaultna zaštita (AnyRequest): Svaka ruta koja nije izričito navedena u javnoj sekciji automatski se postavlja u authenticated() mod, primjenjujući sigurnosni princip "Zero Trust" – sve je zaključano dok se ne dokaže identitet.

CSRF (Cross-Site Request Forgery)

Onemogućavanje zaštite: CSRF zaštita je isključena (csrf.disable()) jer se aplikacija oslanja na JWT tokene u Authorization headeru. Budući da se autentifikacija ne vrši putem automatskih kolačića (cookies) preglednika, rizik od CSRF napada je eliminiran samim dizajnom komunikacije.

Kontroleri poslovne logike

Ovaj dio sustava upravlja komunikacijom između frontenda i baze podataka, provodi sigurnosne provjere te izvršava kompleksne poslovne procese poput obrade slika i automatizacije rezervacija.

Glavne rute :

Javne rute (Dostupne svim posjetiteljima):

- **/unit/all, /unit/filter, /unit/summary:** Pregledavanje i napredna pretraga smještaja.
- **/unit/top-rated, /unit/counts-by-beds/view:** Statistike za početnu stranicu.
- **/unitReservation/occupied-dates/{unitId}:** Provjera dostupnosti (kalendar).
- **/api/location (GET):** Dohvaćanje službene adrese objekta.

Korisničke rute (Za prijavljene goste):

- **/addPerson, /me, /updateCountry:** Upravljanje korisničkim profilom i registracija.
- **/unitReservation/add:** Slanje zahtjeva za rezervaciju (Inquiry).
- **/unitReservation/my-reservations:** Pregled vlastitih putovanja.
- **/unitReservation/cancel/{id} & /rate/{id}:** Otkazivanje i ocjenjivanje završenih boravaka.

Administrativne rute (Admin / Owner privilegije):

- **/unit/add & /unit/update/{id}:** Kreiranje i kompleksna sinkronizacija apartmana/soba.
- **/unitImg/upload, /delete, /set-cover:** Upravljanje galerijama i Cloud pohranom.

- **/unitImg/delete-full/{id}**: Potpuno uklanjanje objekta i pripadajućih rezervacija.
- **/unitReservation/all**: Centralni pregled poslovanja za vlasnike.
- **/unitReservation/update-status/{id}**: Odobravanje ili odbijanje zahtjeva gostiju.
- **/allPersons, /deletePerson, /updateRole**: Upravljanje korisničkim pravima.

Objašnjenja komponenti Backenda

UnitController & UnitImgController (Upravljanje objektima)

Hibridna struktura (Apartman vs. Soba): Sustav prepoznaje razliku u tipu smještaja. Ako se dodaje "Soba" s više jedinica, backend automatski generira pod-entitete (npr. "Soba 01", "Soba 02"). Prilikom ažuriranja (update), sustav inteligentno provjerava buduće rezervacije prije nego dopusti smanjenje broja soba.

Cloud Storage & Media Management: Slike se ne spremaju na server, već na S3 Cloud.

- **Folder Logic**: Slike su strogo odvojene u /cover/ i /other/ foldere.
- **Atomic Updates**: Promjena naslovne slike (set-cover) fizički premješta datoteku na Cloudu i istovremeno ažurira bazu, osiguravajući da frontend uvijek povlači ispravan URL.
- **Full Cleanup**: Brisanje objekta (delete-full) pokreće lančanu reakciju: brišu se podaci iz baze, brišu se sve slike s Clouda i šalju se email obavijesti gostima čije su rezervacije otkazane.

UnitReservationController (Booking Engine)

Algoritam slobodnih soba: Prilikom rezervacije, sustav ne traži samo "postoji li smještaj", već aktivno traži prvu slobodnu pod-jedinicu koja nema preklapanje datuma (datesOverlap). Koristi se sortiranje po ID-u kako bi se osigurala konzistentna dodjela soba.

Logika zauzetosti (Occupied Dates): Backend izračunava zauzetost u realnom vremenu:

1. Mapira sve rezervacije po danima u HashMap.
2. Uspoređuje broj rezervacija s ukupnim kapacitetom objekta.
3. Tek kada su sve sobe istog tipa zauzete, taj dan se šalje frontendu kao blokiran.

Automatizacija ocjena: Metoda rate vrši strogu validaciju (korisnik mora biti vlasnik rezervacije, status mora biti Completed). Nakon uspješnog ocjenjivanja, pokreće se nativni SQL upit koji trenutno osvježava prosječnu ocjenu cijelog objekta.

PersonController & Security Logic

Google OAuth Sinkronizacija: Prilikom prve prijave, /addPerson izvlači email i ime iz JWT-a. Implementiran je **First User Admin** sustav – prvi korisnik u bazi automatski postaje administrator, dok svi ostali dobivaju User ulogu.

Location Guard: Putem LocationController-a i updateCountry metode, sustav osigurava integritet podataka. Globalna adresa objekta je fiksirana na ID 1, što olakšava frontendu dohvaćanje podataka bez pretraživanja cijele tablice.

ReservationTask (Background Workers)

Cron Automatizacija: Sustav svaku ponoć pokreće autoCloseReservations.

- Pronalazi sve potvrđene rezervacije kojima je prošao datum odlaska.
 - Automatski postavlja status na Completed.
 - Ovime se omogućuje korisnicima da daju recenzije bez ručne intervencije vlasnika, čime se održava dinamika platforme.
-
-

DTO sloj (Data Transfer Objects)

DTO komponente služe kao lagani omotači za podatke koji se šalju između frontenda i backenda. Njihova glavna svrha je optimizacija prometa (slanje samo potrebnih polja) i sigurnost (skrivanje internih struktura baze podataka).

Glavne strukture :

Rezervacijski i kalendarski DTO-ovi:

- **OccupiedDateDTO:** Koristi se za slanje zauzetih termina frontendu. Šalje se kao par startDate i endDate, što omogućuje React kalendaru da vizualno blokira cijele raspone datuma.
- **ReservationRequest:** Objekt koji prima podatke s "Reserve" modala. Sadrži sve potrebne informacije za kreiranje rezervacije, uključujući listu pogodnosti (amenities) koje je korisnik odabrao (npr. doručak, parking).

Pretraga i filtriranje:

- **SearchParams:** Kompleksni objekt koji objedinjuje sve moguće parametre pretrage (datume, kapacitet, cijene, sortiranje i paginaciju). Omogućuje čišći kod u kontroleru jer ne zahtijeva desetke pojedinačnih @RequestParam deklaracija.

- **UnitFilterDTO:** Specifično dizajniran za listu rezultata pretrage (/hotels). Sadrži ključne informacije poput naslovne slike, prosječne ocjene i osnovnih pogodnosti, čime se izbjegava učitavanje teških povezanih objekata iz baze.

Statistika i prikaz:

- **UnitCountDTO:** Koristi Java record (za nepromjenjivost i brzinu). Služi za prikaz kategorija na početnoj stranici (npr. "Sea view: 5 units"), noseći labelu, broj jedinica i URL reprezentativne slike.
- **UnitTopRatedDTO:** Optimizirani record za "Featured" sekciju. Prenosi samo 5 ključnih polja (ID, ime, cijena, slika, ocjena) za najbrže moguće renderiranje najbolje ocijenjenih objekata.
- **UnitSummaryDTO:** Najlakši DTO u sustavu. Sadrži samo osnovnu identifikaciju i tip jedinice, a koristi se za brze "dropdown" izbornike ili administrativne liste gdje detaljni opisi nisu potrebni.

Objašnjenja komponenti Backenda

Arhitektonska optimizacija

Smanjenje mrežnog opterećenja: Umjesto slanja cijelog Unit modela (koji sadrži dugačke opise, liste svih slika i sve povijesne rezervacije), DTO-ovi poput UnitSummaryDTO smanjuju veličinu JSON odgovora i do 90%. To je ključno za brzi odziv aplikacije na mobilnim mrežama.

Korištenje Java Record-a: Za objekte poput UnitCountDTO i UnitTopRatedDTO korišten je moderni Java record tip. Zapisi su po dizajnu nepromjenjivi (immutable), što ih čini savršenim za "read-only" operacije prikaza statistika i najbolje ocijenjenih jedinica.

Integracija s Frontend logikom

Mapiranje na UI komponente:

- Polja u UnitFilterDTO (npr. hasWifi, hasParking) izravno odgovaraju ikonama koje se prikazuju na karticama smještaja u rezultatima pretrage.
- numRooms i numBeds su dodani u ovaj DTO kako bi frontend mogao odmah prikazati specifikacije bez dodatnog API poziva.

Fleksibilnost pretrage: SearchParams podržava napredne značajke poput paginacije (page, size) i sortiranja (sort, order), što omogućuje frontendu da implementira "Beskonačni scroll" ili sortiranje po cijeni bez promjene backend logike.

Sigurnost i validacija

Enkapsulacija logike: ReservationRequest prima listu stringova za pogodnosti, koju backend kasnije u kontroleru spaja (String.join) i sprema u bazu. Ovakav pristup odvaja način na koji frontend šalje podatke (kao niz) od načina na koji ih baza pohranjuje (kao tekstualni niz).

Dosljednost imenovanja: Iako Java koristi standardno imenovanje (npr. isApartment), DTO-ovi su prilagođeni (apartment) kako bi serijalizacija u JSON bila što prirodnija za JavaScript i React na frontendu, izbjegavajući potencijalne probleme s "is" prefiksima u nekim JSON parserima.

Modeli podataka (Database Entities)

Modeli predstavljaju srce aplikacije i preslikavaju se izravno u tablice baze podataka putem Hibernate-a (ORM). Oni definiraju strukturu informacija, od korisničkih profila do kompleksnih veza između smještajnih jedinica i njihovih slika.

Glavne entitetske grupe :

Smještaj i mediji (Unit & UnitImg):

- **Unit (Smještaj):** Centralni entitet s preko 20 polja. Podržava rekurzivnu vezu (Self-reference) preko parent_unit_id, što omogućuje hijerarhiju gdje jedan "Hotel" može imati više "Soba". Sadrži sve tehničke specifikacije (pogled na more, WiFi, klima) i prosječnu ocjenu koja se sinkronizira s recenzijama.
- **UnitImg (Galerija):** Povezan s jedinicom putem ManyToOne veze. Pohranjuje URL-ove slika s Clouda, osiguravajući da podaci o medijima budu odvojeni od same logike smještaja radi boljih performansi.

Korisnici i Sigurnost (Person):

- **Person (Korisnik):** Definira uloge putem tri Boolean zastavice (is_admin, is_user, is_owner). Koristi @JsonProperty kako bi se osiguralo da JSON ključevi na frontendu budu u snake_case formatu, dok su u Javi camelCase. Čuva podatke o lokaciji korisnika (country, city) potrebne za zakonsku evidenciju gostiju.

Poslovanje i Rezervacije (UnitReservation):

- **UnitReservation (Rezervacija):** Ključni transakcijski model. Povezuje osobu (Person) i smještaj (Unit). Sadrži status (Pending, Confirmed, Cancelled, Completed), detalje o broju gostiju, odabrane dodatne usluge (selectedAmenities) te sustav za naknadno ocjenjivanje boravka.

Konfiguracija (LocationSettings):

- **LocationSettings:** Singleton tablica (uvijek id = 1L). Čuva globalne kontakt podatke objekta i Google Maps URL koji se koristi kao fallback na cijeloj platformi.
-

Objašnjenja komponenti Backenda

Relacijska logika i Integritet

Self-Referencing Relationship: Model Unit koristi polje parentUnit i listu listOfRooms. Ova struktura omogućuje da sustav tretira "Apartman" kao samostalnu jedinicu, dok "Hotel" tretira kao roditelja koji u sebi sadrži više identičnih ili različitih soba. Koristi se @OrderBy("unitName ASC") kako bi sobe uvijek bile prikazane kronološki (Soba 01, 02...).

Lazy Loading & JSON kružne reference:

- Kako bi se spriječilo beskonačno ponavljanje podataka u JSON-u (npr. Unit vuče slike, slika vuče Unit natrag), korišteni su @JsonManagedReference i @JsonBackReference.
- Korištenje FetchType.LAZY osigurava da baza ne povlači teške podatke (poput cijele povijesti rezervacija) osim ako to programski kod izričito ne zatraži, što drastično ubrzava odziv API-ja.

Napredne značajke modela

Dinamički Rating sustav: Unutar Unit modela nalazi se polje averageRating, ali i metoda @JsonProperty("rating"). Ova metoda pri serijalizaciji automatski zaokružuje ocjenu na jednu decimalu (npr. s 9.222 na 9.2), osiguravajući čist prikaz na frontendu bez potrebe za dodatnim formatiranjem u Reactu.

Hibernate Proxy Guard: Dodavanjem @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"}), modeli su osigurani od čestih grešaka pri serijalizaciji proxy objekata koje Hibernate koristi za "lijeno" učitavanje podataka iz baze.

Poslovna pravila u bazi

Orphan Removal: Kod veza sa slikama i rezervacijama postavljen je orphanRemoval = true. To znači da ako admin ukloni sliku iz liste unutar koda, Hibernate će je automatski obrisati i iz tablice u bazi podataka, održavajući bazu čistom od nepotrebnih podataka.

Custom Column Definitions: Polja poput mainDescContent ili selectedAmenities koriste columnDefinition = "text". To omogućuje spremanje dugačkih tekstova i opisa koji nadilaze standardno ograničenje od 255 znakova koje nudi obični String tip u bazama podataka.

Repozitorij sloj (Data Access Layer)

Repozitoriji su "mozak" baze podataka. Ovdje se definiraju kompleksni SQL i JPQL upiti koji omogućuju da sustav bude brz i efikasan, čak i kod zahtjevnih operacija poput filtriranja dostupnosti po datumima.

Glavne funkcionalnosti :

UnitRepo (Napredno filtriranje i statistika):

- **filterUnitsDTO:** Najkompleksniji upit u sustavu. On u jednom koraku provjerava desetke parametara (cijena, oprema, broj kreveta) i istovremeno provjerava dostupnost. Upit je inteligentan: ako je objekt "Hotel", on provjerava postoji li *bilo koja* slobodna soba unutar njega za odabrane datume.
- **Rating Engine:** Sadrži refreshUnitRating, nativni SQL upit koji automatski preračunava prosjek ocjena uzimajući u obzir sve recenzije za roditeljsku jedinicu i sve njezine pod-sobe.
- **Dynamic Content:** Upiti poput findFirstImageUrlByBeds automatski pronalaze najljepšu sliku (iz najbolje ocijenjenog apartmana) za određenu kategoriju na početnoj stranici.

UnitReservationRepo (Upravljanje zauzetošću):

- **existsByUnitAndDatesOverlap:** Ključna provjera koja sprječava dvostruke rezervacije (Double Booking) uspoređujući vremenske raspon raspone u bazi.
- **EntityGraph optimizacija:** Koristi @EntityGraph kako bi u jednom pozivu bazi dohvatio rezervaciju, osobu, smještaj i slike. Bez ovoga, sustav bi slao stotine malih upita (tzv. N+1 problem), što bi usporilo aplikaciju.
- **Occupied Dates Lookup:** Dohvaća sve relevantne rezervacije za specifičan objekt kako bi React kalendar mogao ispravno prikazati sive (zauzete) dane.

PersonRepo & UnitImgRepo:

- **findByEmail:** Brzo dohvaćanje korisnika iz Google tokena.
 - **findByUnitIdUnit:** Dohvaćanje cijele galerije slika za specifičan apartman.
-

Objašnjenja komponenti Backenda

Optimizacija performansi (Query Level)

DTO Projection: Umjesto da upiti vraćaju teške entitete, upiti u UnitRepo koriste `SELECT new UnitFilterDTO(...)`. To znači da baza direktno konstruira lagane objekte, prenoseći samo nužne bajtove (npr. samo URL prve slike umjesto cijele liste slika).

Subquery Logic: Unutar `filterUnitsDTO` koristi se podupit (`SELECT img.url ... LIMIT 1`) za dohvaćanje naslovne slike. Ovo eliminira potrebu da frontend radi naknadne pozive za svaku karticu apartmana u rezultatima pretrage.

Logika dostupnosti (Availability Logic)

Double-Booking Zaštita: Upit za preklapanje datuma koristi logiku (`r.startDate < :endDate AND r.endDate > :startDate`). Ovo je matematički najsigurniji način za provjeru preklapanja dva vremenska intervala, osiguravajući da gost ne može rezervirati sobu koja se čisti ili je već zauzeta.

Hijerarhijska provjera: Za objekte koji imaju sobe, sustav ne provjerava roditelja, već koristi `EXISTS` upit nad djecom (`listOfRooms`). Ako barem jedna soba istog tipa nema rezervaciju u tom periodu, cijeli objekt se prikazuje kao "Dostupan".

Integritet i automatizacija

Native SQL za statistiku: Upit `refreshUnitRating` je napisan u nativnom SQL-u (`nativeQuery = true`) jer mora raditi s agregacijskim funkcijama (`AVG`, `COALESCE`) i `DOUBLE PRECISION` tipovima podataka, što osigurava maksimalnu preciznost ocjena od 1.0 do 10.0.

Transactional Safety: Sve operacije koje mijenjaju podatke (poput `refreshUnitRating`) označene su s `@Modifying` i `@Transactional`, što garantira da će baza ostati u konzistentnom stanju čak i ako dođe do prekida u mreži tijekom izvođenja upita.

Servisni sloj (Business Logic & External Integrations)

Servisi izoliraju kompleksnu logiku i vanjske integracije (S3, Brevo API, PDF generiranje) od ostatka aplikacije, osiguravajući stabilnost i asinkrono izvršavanje zadataka.

Glavne komponente :

EmailService (Komunikacijski centar):

- **Brevo API integracija:** Sustav više ne koristi zastarjeli SMTP, već komunicira putem modernog REST API-ja s Brevo servisom.

- **Asinkrono slanje (@Async):** Slanje e-maila može potrajati nekoliko sekundi. Zahvaljujući @Async anotaciji, korisnik na frontendu dobiva odgovor trenutno, dok se e-mail šalje u pozadinskoj dretvi.
- **Dinamički HTML:** Automatski generira profesionalne e-maile s detaljima rezervacije, statusom i brendiranim dizajnom.

PdfService (Službena dokumentacija):

- **iText/OpenPDF generiranje:** Kreira službene potvrde rezervacije u PDF formatu.
- **Automatski izračun:** Samostalno računa broj noćenja (ChronoUnit.DAYS) i ukupnu cijenu na temelju podataka iz baze, uključujući popis svih odabranih pogodnosti.

FileService & FileCleanupService (Cloud Storage):

- **Supabase S3 integracija:** Upravlja uploadom, brisanjem i premještanjem slika. Koristi napredni SSL debugging i TLS protokole za sigurnu komunikaciju.
- **Smart Folder Cleanup:** FileCleanupService se pokreće 10 sekundi nakon starta aplikacije i provjerava konzistentnost. Ako na Cloudu postoji folder za jedinicu koja je obrisana iz baze, servis automatski čisti sav "otpad" s Clouda.

UnitService (Poslovna logika pretrage):

- **Bridge komponenta:** Služi kao most između kontrolera i kompleksnih SQL upita u repozitoriju, osiguravajući da su svi ulazni parametri (poput cijena i datuma) ispravno formatirani prije slanja bazi podataka.

Objašnjenja komponenti Backenda

Arhitektura asinkronih procesa

Korisničko iskustvo bez čekanja: Kada gost klikne "Reserve", backend istovremeno sprema podatke u bazu, pokreće generiranje PDF-a i inicira slanje e-maila. Korištenjem @Async, korisnik vidi poruku "Uspješno" dok se teški procesi još vrte u pozadini.

Upravljanje datotekama i sigurnost

S3 Copy-and-Delete Pattern: Prilikom promjene naslovne slike, FileService koristi moveFile logiku. S3 ne poznaje klasični "Rename", pa servis prvo kopira datoteku na novu lokaciju (/cover/), a zatim briše staru, čime se simulira premještanje bez gubitka podataka.

SSL Truststore Debugging: U FileService je implementiran robusni sustav za provjeru certifikata (javax.net.ssl.trustStore). Ovo je ključno za rad na cloud platformama poput Rendera ili Railwaya, gdje sustav mora potvrditi SSL certifikate Supabase API-ja.

Generiranje dokumenata u realnom vremenu

Financijski integritet: PdfService ne koristi podatke koje šalje korisnik, već podatke izvlači izravno iz baze (preko UnitReservation entiteta) u trenutku generiranja. To sprječava bilo kakve pokušaje manipulacije cijenom na PDF potvrdi.

Resursna efikasnost: Svaki PDF se generira u memoriji (ByteArrayOutputStream) i šalje kao Base64 string kroz API, što znači da se na disku servera nikada ne spremaju privremene datoteke, čime se štedi prostor i povećava privatnost korisnika.

Garbage Collection (Cloud Edition)

Startup Sync: onStartupCleanup je sigurnosni mehanizam. Ako server padne usred brisanja neke jedinice, ovaj servis će pri ponovnom pokretanju otkriti nesklad između baze i Clouda te dovršiti brisanje, sprječavajući nakupljanje nepotrebnih troškova pohrane.